



Zagazig University, Faculty of Engineering,
Department of Electronics and Communications

Integrated Smart Vehicle Management System

“ISVMS”

Enhancing the Driving Experience with an Innovative Vehicle Management System

Team Members

Mahmoud Esmat Abdullah Fahmy
Muhammed Ayman Muhammed Fathy
Mahmoud Ashraf Fathy Abdel-all
Salma Osama Mohamed Abdel-hay
Omnia Wael Ibrahim Abdel-azim
Sara Reda Abdel-gawad Amer
Rehab Nasser Mohamed Abdel-rahman
Hager Saad Eissa Ananny
Hadeer Ehab Mohamed Hassan

Supervised by: Dr. Rania Ahmed, Dr. Azhar Ahmed

A book presented for graduation project

July 2025

Anti-Plagiarism Declaration

We hereby declare that the graduation project titled **ISVMS**, submitted for the completion of a bachelor's degree at Zagazig University, is entirely our original work, which has been conducted **under the supervision of Dr. Azhar Ahmed and Dr. Rania Ahmed**. We have undertaken thorough research and utilized various sources to gather information, duly acknowledging all the resources consulted in the references section.

We affirm that this project is free from any form of plagiarism, including but not limited to copying, paraphrasing, or reproducing the work of others without appropriate attribution. Any direct quotations or ideas borrowed from external sources have been appropriately cited and referenced.

Acknowledgement

We would like to express our deepest gratitude to our esteemed professors, **Dr. Azhar Ahmed and Dr. Rania Ahmed**, for their unwavering guidance, mentorship, and support throughout our academic journey. Their vast knowledge, expertise, and dedication have been instrumental in shaping our intellectual growth and preparing us for future challenges.

Our sincere appreciation also extends to our colleagues, whose collaboration, encouragement, and shared experiences have enriched our learning and made this journey both rewarding and meaningful. Their diverse perspectives have contributed significantly to our academic and personal development.

Finally, we sincerely appreciate everyone who has contributed to this graduation book and our academic journey. The knowledge and memories we have gained will remain with us for a lifetime.

Abstract

The Integrated Smart Vehicle Management System (ISVMS) is an advanced, technology-driven solution aimed at enhancing vehicle automation, monitoring, and control. By integrating state-of-the-art hardware components, including ESP32, Atmega32, GPS (NEO-6M), Pressure Sensor (PMB180), and Temperature Sensor (LM35), the system ensures optimized performance and intelligent vehicle operation.

Designed with a multi-layered architecture, the system is divided into two primary functional domains:

- The first layer focuses on motion control, obstacle detection, and communication, incorporating ultrasonic, PIR, IR, and Bluetooth sensors for real-time responsiveness.
- The second layer is dedicated to environmental monitoring and system maintenance, utilizing a PCB, ESP32, and critical sensors to ensure operational efficiency and data-driven decision-making.

To enhance wireless communication and real-time data transmission, the system is complemented by an IoT-based framework that leverages Node-RED and MQTT protocols for seamless connectivity. Additionally, a custom mobile application (developed using MIT App Inventor) provides remote vehicle control, diagnostics, and intelligent automation features, such as collision avoidance and auto-parking.

By employing dual-microcontroller architecture (Atmega32 & ESP32), the system ensures seamless coordination, high computational efficiency, and enhanced control capabilities. This project represents a significant advancement in smart vehicle technology, offering a scalable, cost-effective, and high-performance solution for autonomous driving, fleet management, and IoT-integrated automotive applications.

Contents

Chapter 1: Introduction	(page 7)
Chapter 2: Car Hardware	(page 13)
2.1 Car Dimensions.....	(page 13)
2.2 Car Components.....	(page 13)
2.2.1 Basic Components.....	(page 13)
2.2.2 First Layer.....	(page 15)
2.2.3 Second Layer.....	(page 20)
2.3 Final version of the car.....	(page 23)
Chapter 3: Maintenance and IOT	(page 24)
3.1 ESP32	(page 24)
3.1.1 Comprehensive Overview of ESP32-WROOM-32	(page 24)
3.1.2 Programming ESP32	(page 26)
3.2 Node-Red	(page 27)
3.3 Sensors	(page 29)
3.3.1 GPS Sensor “NEO 6M”	(page 29)
3.3.2 Pressure Sensor “PMB 180”	(page 32)
3.3.3 Temperature Sensor “LM35”	(page 34)
3.4 Code of maintenance	(page 36)
3.5 Node-red maintenance	(page 42)
Chapter 4: Application (MIT App Inventor)	(page 47)
4.1 Comprehensive Overview of MIT App Inventor	(page 47)
4.2 Car Application	(page 48)
4.2.1 Login page blocks code	(page 49)
4.2.2 Login page blocks explanation	(page 53)
4.2.3 Home page blocks code	(page 58)
4.2.4 Home page blocks explanation	(page 59)
4.2.5 Controller page blocks code	(page 60)
4.2.6 Controller page blocks explanation	(page 64)
4.2.7 About_Us page blocks code.....	(page 69)
4.2.8 About_Us page blocks explanation	(page 73)

Chapter 5: Drivers of Atmega32	(page 74)
5.1 Atmega32	(page 74)
5.1.1 ATmega32 Microcontroller Overview	(page 74)
5.1.2 Programming Atmega32	(page 75)
5.2 Library	(page 77)
5.2.1 LIB drivers	(page 77)
5.2.2 MCAL drivers	(page 79)
Chapter 6: Control	(page 85)
6.1 Motor	(page 85)
6.1.1 Overview about DC Motor	(page 85)
6.1.2 DC Motor Calculation	(page 86)
6.1.3 Code of DC Motor	(page 87)
6.2 H-Bridge “L298”	(page 90)
6.3 Bluetooth module “HC-05”	(page 92)
6.4 Code of control	(page 93)
6.5 Simulation on proteus	(page 95)
Chapter 7: Auto Parking	(page 98)
7.1 Ultrasonic “HC-SR04”.....	(page 98)
7.1.1 Overview of ultrasonic Sensor	(page 98)
7.1.2 Code of Ultrasonic Sensor	(page 101)
7.1.3 Auto Parking Motion calculation	(page 102)
7.1.4 LCD	(page 103)
7.1.5 Auto Parking code	(page 104)
7.1.6 Auto Parking Simulation	(page 107)
Chapter 8: Optimizing Collision Avoidance	(page 108)
8.1 Collision avoidance using PIR.....	(page 108)
8.1.1 Overview of Passive Infrared (PIR) Sensor	(page 108)
8.1.2 Code of PIR	(page 109)
8.1.3 Using Two PIR in our project	(page 109)
8.1.4 Interrupts and Their Importance	(page 109)
8.1.5 Check motion of objects	(page 111)
8.2 Collision avoidance using Ultrasonic.....	(page 113)
References	(page 120)

List of figures

1.1	ISVMS block	(page 9)
2.1	Car Dimensions	(page 13)
2.2	(a) Car Wheel, (b) Hex Coupler	(page 13)
2.3	(a) Battery, (b) Battery Charger	(page 14)
2.4	(a) Battery Box Holder, (b) Digital Battery Monitor	(page 14)
2.5	DC Motor	(page 15)
2.6	Car components First Layer	(page 15)
2.7	Eta32mini AVR Kit	(page 16)
2.8	H-Bridge L298	(page 17)
2.9	HC-SR04	(page 17)
2.10	PIR Sensor	(page 18)
2.11	IR Sensor	(page 18)
2.12	"HC-05" Bluetooth module	(page 19)
2.13	Car Components Second Layer	(page 20)
2.14	Prototype Board 9x7cm	(page 20)
2.15	ESP32 WROOM Devkit	(page 21)
2.16	"NEO 6M" GPS sensor	(page 22)
2.17	PMB180 Sensor	(page 22)
2.18	"LM35" Temperature Sensor	(page 23)
2.19	Final version of the car	(page 23)
3.1	ESP32 Devkit	(page 24)
3.2	Arduino IDE	(page 27)
3.3	Node.js	(page 28)
3.4	Open node-red	(page 29)
3.5	GPS sensor	(page 29)
3.6	GPS Sensor connection	(page 31)
3.7	GPS Sensor connection with ESP32	(page 31)
3.8	PMB180 Sensor	(page 32)
3.9	PMB180 connection to ESP32	(page 33)
3.10	LM35 Sensor	(page 34)
3.11	LM35 Sensor connection to ESP32	(page 36)
3.12	Node-red receiving sensors readings	(page 42)
3.13	Mqtt in server handle in node-red	(page 42)
3.14	Code of PMB180 Pressure function in node-red	(page 43)
3.15	Code of PMB180 Temperature function in node-red	(page 43)
3.16	Code of LM35 Temperature function in node-red	(page 44)
3.17	Editing the gauge	(page 45)
3.18	Reading sensors on "Home" Tab in node-red before burning	(page 45)

3.19	Reading sensors on “Home” Tab in node-red after burning	(page 46)
3.20	Arduino IDE serial terminal after burning	(page 46)
4.1	MIT App. Inventor	(page 47)
4.2	Login Page Application UI	(page 49)
4.3	Home Page Application UI	(page 58)
4.4	Controller Page Application UI	(page 60)
4.5	About Us Page Application UI	(page 69)
5.1	Atmega32	(page 74)
5.2	Layers of Application	(page 77)
6.1	DC Motor	(page 85)
6.2	L298	(page 90)
6.3	L298 connections	(page 91)
6.4	HC-05	(page 92)
6.5	Proteus control	(page 95)
6.6	Setting of Atmega32 in proteus	(page 97)
6.7	Setting of HC-05 in proteus	(page 97)
7.1	HC-SR04	(page 98)
7.2	Wave of ultrasonic	(page 98)
7.3	HC-SR04 Ultrasonic Module Timing Diagram	(page 99)
7.4	LCD 16x2	(page 103)
7.5	Simulation of Car in proteus	(page 107)
8.1	Passive Infrared Sensor	(page 108)
8.2	AND Gate of interrupt service	(page 111)
8.3	Simulation of PIR in proteus	(page 112)
8.4	Total Project Simulation in proteus	(page 119)

List of Tables

3.1	Compare between IOT microcontrollers	(page 25)
3.2	Compare between ESP32 development frameworks	(page 26)
3.3	Connections of NEO 6M pins	(page 31)
3.4	Connections of PMB180 pins	(page 33)
3.5	Connections of LM35 pins	(page 36)
4.1	Summary of Possible Outcomes	(page 57)
4.2	Summary of Controller Page Buttons	(page 68)
5.1	Compare between Atmega32 development frameworks	(page 75)

Chapter 1: Introduction

1.1 Introduction

The Integrated Smart Vehicle Management System (ISVMS) is an innovative project aimed at enhancing the driving experience through advanced vehicle automation, monitoring, and control. With the rapid evolution of automotive technology, the need for smarter, safer, and more efficient vehicles has become paramount. This project leverages cutting-edge hardware and software technologies to create a system that not only automates vehicle operations but also provides real-time monitoring and control capabilities. The ISVMS integrates state-of-the-art components such as the ESP32 microcontroller, Atmega32, GPS, pressure sensors, and temperature sensors to ensure optimized performance and intelligent vehicle operation.

The system is designed with a multi-layered architecture, focusing on two primary functional domains: motion control and environmental monitoring. The first layer handles motion control, obstacle detection, and communication, while the second layer is dedicated to environmental monitoring and system maintenance. By employing dual-microcontroller architecture (Atmega32 & ESP32), the system ensures seamless coordination, high computational efficiency, and enhanced control capabilities. Additionally, the project incorporates an IoT-based framework for wireless communication and real-time data transmission, complemented by a custom mobile application for remote vehicle control and diagnostics.

This project represents a significant advancement in smart vehicle technology, offering a scalable, cost-effective, and high-performance solution for autonomous driving, fleet management, and IoT-integrated automotive applications.

1.2 Problem statement

The modern automotive industry faces several challenges, including the need for enhanced safety, improved fuel efficiency, and reduced environmental impact. Traditional vehicle management systems often lack the integration of advanced technologies that can provide real-time monitoring, automation, and control. This leads to inefficiencies in vehicle operation, increased risk of accidents, and limited capabilities in fleet management.

Key issues include:

- **Lack of Real-Time Monitoring:** Traditional systems do not provide real-time data on vehicle performance, environmental conditions, or potential hazards.

- **Limited Automation:** Most vehicles rely on manual control, which can lead to human error and increased accident rates.
- **Inefficient Fleet Management:** Fleet operators struggle to monitor and control multiple vehicles simultaneously, leading to inefficiencies and increased operational costs.
- **Poor Integration of IoT:** The integration of IoT in vehicles is often limited, preventing the full utilization of data-driven decision-making and remote-control capabilities.

The ISVMS project addresses these challenges by developing a comprehensive system that integrates advanced sensors, microcontrollers, and IoT technologies to provide real-time monitoring, automated control, and efficient fleet management.

1.3 Project Objectives

The primary objectives of the Integrated Smart Vehicle Management System (ISVMS) project are as follows:

- **Enhance Vehicle Automation:** Develop a system that automates key vehicle operations such as collision avoidance, auto-parking, and speed control to reduce human error and improve safety.
- **Real-Time Monitoring:** Implement a system that provides real-time data on vehicle performance, environmental conditions, and potential hazards using sensors such as GPS, pressure sensors, and temperature sensors.
- **IoT Integration:** Create an IoT-based framework that enables seamless communication between the vehicle and a central control system, allowing for remote monitoring and control.
- **Develop a Custom Mobile Application:** Design and implement a mobile application that allows users to remotely control the vehicle, monitor its status, and receive alerts in case of emergencies.
- **Optimize Fleet Management:** Provide fleet operators with tools to monitor and control multiple vehicles simultaneously, improving efficiency and reducing operational costs.
- **Ensure Scalability and Cost-Effectiveness:** Design the system to be scalable and cost-effective, making it suitable for a wide range of applications, from personal vehicles to large fleets.

- **Improve Environmental Monitoring:** Use sensors to monitor environmental conditions such as tire pressure and temperature, ensuring optimal vehicle performance and reducing environmental impact.

1.4 Project Block-Diagram & Operation

The Integrated Smart Vehicle Management System (ISVMS) is designed with a multi-layered architecture, as shown in the block diagram below:

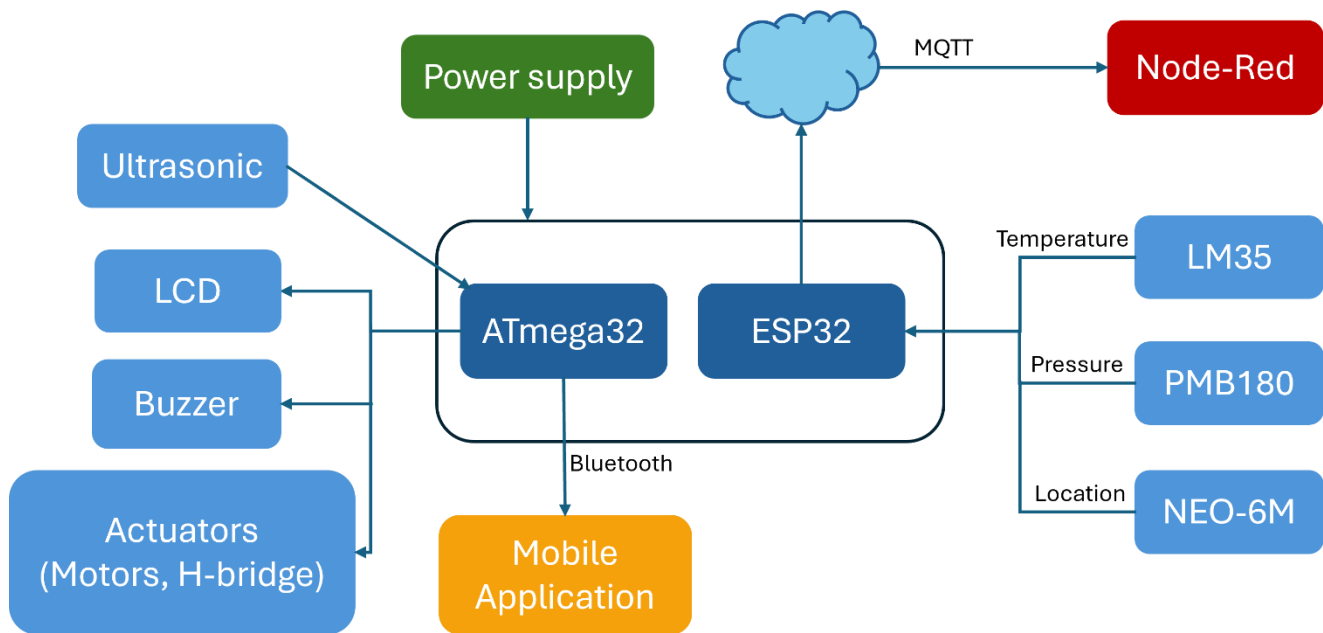


Fig. 1.1. ISVMS block

- **Operation of project**
 - **Mobile Application**
 - **Function:** The user interface for remote control of the vehicle.
 - **Technology:** Developed using MIT App Inventor.
 - **Connections:**
 - **To IoT Framework:** Sends commands (e.g., "F" for forward, "B" for backward, etc.) to the IoT framework via MQTT.
 - **From IoT Framework:** Receives real-time data (e.g., GPS location, sensor readings) from the IoT framework via MQTT.

▪ IoT Framework

- **Function:** Acts as a bridge between the mobile app and the vehicle system.
- **Technology:** Node-RED with MQTT protocol.
- **Connections:**
 - **From Mobile App:** Receives commands (e.g., "F", "B", etc.) via MQTT.
 - **To Vehicle System:** Sends commands to the ESP32 microcontroller via MQTT.
 - **From Vehicle System:** Receives sensor data (e.g., GPS, pressure, temperature) from the ESP32 via MQTT.
 - **To Mobile App:** Sends the received sensor data to the mobile app for display.

▪ Vehicle System

- **Function:** Controls the vehicle's movement and monitors the environment.
- **Main Components:**
 - **ESP32:** Handles IoT communication and sensor data processing.
 - **Atmega32:** Controls the motors and sensors (e.g., PIR, ultrasonic).
- **Connections:**
 - **From IoT Framework:** Receives commands (e.g., "F", "B", etc.) via MQTT.
 - **To IoT Framework:** Sends sensor data (e.g., GPS, pressure, temperature) via MQTT.
 - **To Atmega32:** Sends motor control commands to Atmega32 via UART.
 - **From Atmega32:** Receives sensor data (e.g., PIR, ultrasonic) from the Atmega32 via UART.

▪ Sensors & Actuators

- **Function:** Collects environmental data and executes commands.
- **Main Components:**
 - **Sensors:**
 - **GPS (NEO-6M):** For location tracking.
 - **PIR Sensor:** For motion detection and collision avoidance.
 - **Ultrasonic Sensor (HC-SR04):** For distance measurement and auto-parking.
 - **Pressure Sensor (PMB180):** For tire pressure monitoring.

- **Temperature Sensor (LM35):** For temperature monitoring.
- **Actuators:**
 - **DC Motors:** For vehicle movement.
 - **H-Bridge (L298):** For controlling motor direction and speed.
- **Connections:**
 - **From Atmega32:** Receives motor control commands (e.g., forward, backward, etc.) from the Atmega32.
 - **To Atmega32:** Sends sensor data (e.g., PIR, ultrasonic) to the Atmega32 via GPIO or UART.

1.5 Thesis Organization

The thesis is organized into seven chapters, each focusing on a specific aspect of the Integrated Smart Vehicle Management System (ISVMS) project:

- **Chapter 2: Car Hardware**
This chapter provides an in-depth analysis of the vehicle's hardware components, including the motor, tires, chassis, and sensors. It explains how the vehicle operates and provides technical insights into its structure and functionality.
- **Chapter 3: Maintenance and IoT**
This chapter discusses the maintenance aspects of the system, including how sensor data is read and transmitted to the IoT framework. It also explains the process of visualizing data on a dashboard using Node-RED.
- **Chapter 4: Mobile Application**
This chapter focuses on the custom-built mobile application used to control the vehicle wirelessly. It covers data security, administrator information, and the cloud infrastructure employed in the system.
- **Chapter 5: Drivers and Application Layers**
This chapter delves into the drivers and different layers of the application, including MCAL, HAL, LIB, and Application layers. It also covers modules such as PWM, ICU, GPIO, and more.
- **Chapter 6: Vehicle Control**
This chapter explains the codes and mechanisms used to control the vehicle's movement, including motor speed regulation and directional control.

- Chapter 7: Optimizing Collision Avoidance (Methods and Improvements)

This chapter examines the development and enhancement of a vehicle obstacle avoidance system. Initially, PIR sensors were used but were limited to detecting only moving objects. To overcome this, Ultrasonic sensors were introduced, enabling the detection of both static and dynamic obstacles with accurate distance measurement. The improved system features a two-stage safety mechanism: an early warning buzzer and automatic halting when obstacles reach a critical threshold. These advancements ensure precise detection and collision-free autonomous operation.

- Chapter 8: Auto Parking

The final chapter explains how ultrasonic sensors are used to enable the vehicle to park itself automatically upon receiving a command from the application.

Throughout the thesis, each chapter includes simulation testing before real-world implementation to ensure safety and optimize resource utilization. Additionally, all coding is developed from scratch without relying on pre-built libraries, particularly while working with the ATmega32 microcontroller.

Chapter 2: Car Hardware

2.1 Car Dimensions

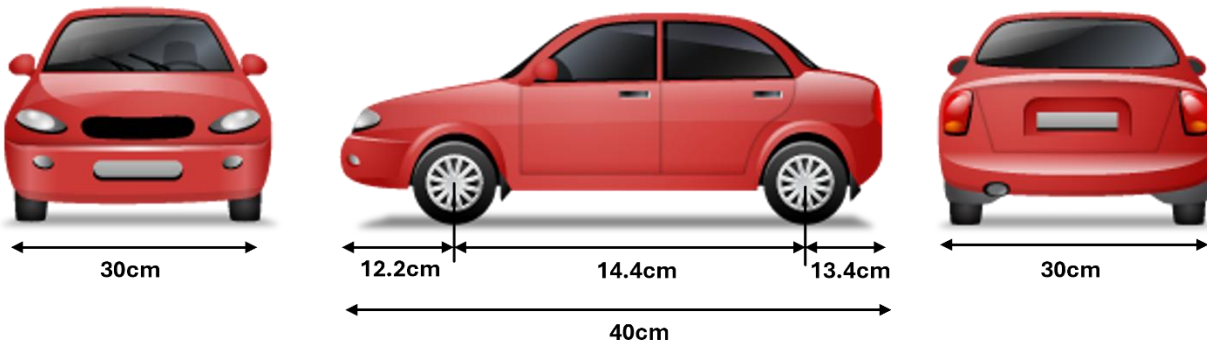


Fig. 2.1. Car Dimensions

We have determined the dimensions of a real car and applied a ratio and proportion method to estimate the dimensions of a smaller vehicle suitable for our project. This ensures that the vehicle remains balanced and stable on the road, can support the required weights, and maintains an appropriate relative speed.

2.2 Car Components

The vehicle is composed of basic components and other components can be summarized in two layers.

2.2.1 Basic Components

- Tires and Hex Coupler



(a)



(b)

Fig. 2.2. (a) Car Wheel, (b) Hex Coupler

Hex coupler 4mm used to connect between 85mm car wheel and DC Geared Motor

■ Batteries

- We use six batteries, each of them is 3.7V.
- Three of them provide enough energy to run L298 driver to control motors. The other three batteries provide enough energy for Atmega32 microcontrollers.
- Three batteries are combined through battery box holder.
- When the battery is empty, we can use a battery charger.
- The level of the battery is measured through digital battery monitor (Operating current: $< 5\text{mA}$, Operating voltage: 8-63V and Operating temperature: 0°C - 40°C).



(a)



(b)

Fig. 2.3. (a) Battery, (b) Battery Charger.



(a)



(b)

Fig. 2.4. (a) Battery Box Holder, (b) Digital Battery Monitor

▪ DC Motors



Fig. 2.5. DC Motor

- The vehicle utilizes DC motors to convert electrical energy into mechanical movement.
- High torque output for improved traction and performance.
- Speed control via PWM signals from the L298 motor driver.
- Works in coordination with the wheels and hex coupler for efficient motion.
- Function in the System: Provides propulsion for the vehicle.

2.2.2 First Layer

The first layer houses the components of the ATmega32 microcontroller and the batteries. The first output from the L298 driver controls the motors on the right side, and the second output controls the motors on the left side, and it contains Ultrasonic, PIR and Bluetooth module.

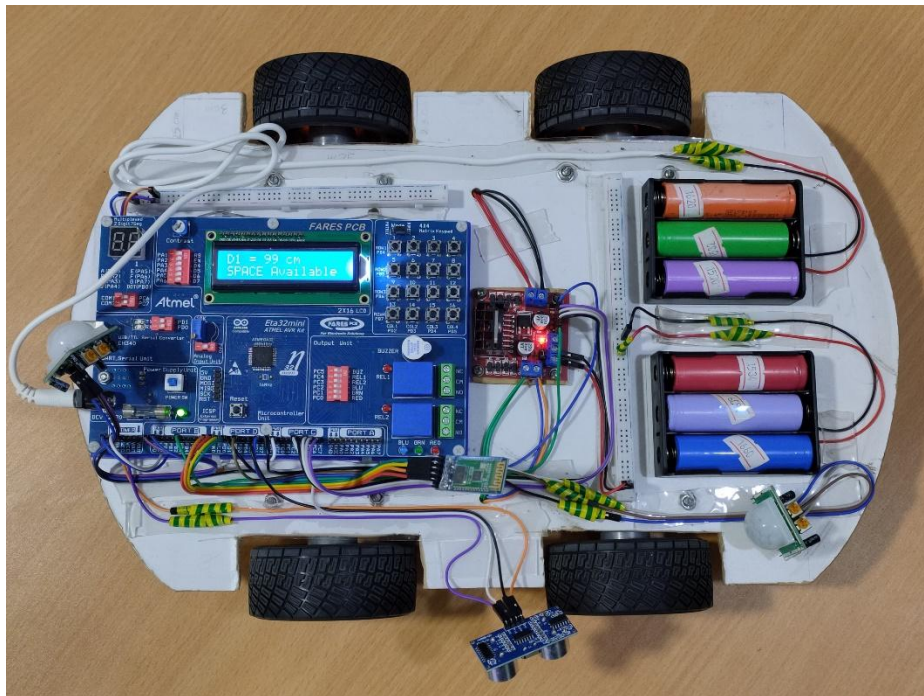


Fig. 2.6. Car components First Layer

■ ETA32Mini AVR Kit

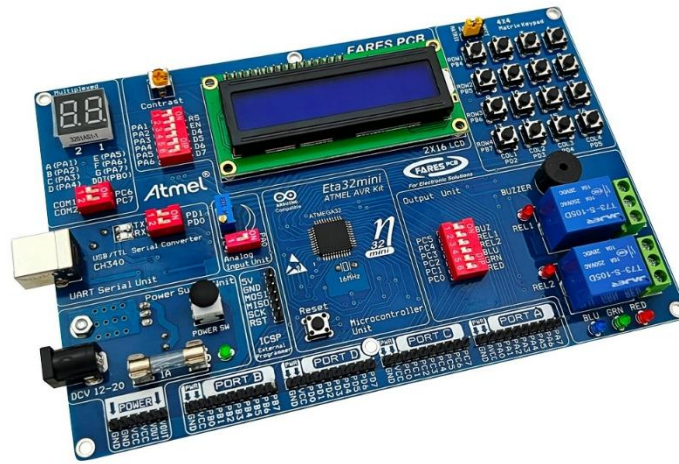


Fig. 2.7. Eta32mini AVR Kit

- The **ETA32Mini AVR Kit** is an advanced microcontroller development board designed for embedded system applications. It is based on the **Atmega32** microcontroller (as we will see in chapter 4), offering a powerful and versatile platform for electronics projects, automation systems, and robotics. The kit provides a structured and integrated environment for beginners and professionals to develop and test microcontroller-based applications efficiently.
- Built-in Peripherals and Components
 - LCD Display Connector: Supports 16x2 LCD for visual output.
 - Push Buttons (can be used as keypad) and Switches: Integrated tactile switches for user input.
 - LED Indicators: Status LEDs for debugging and system indication.
 - Buzzer Module: Used for alert or notification signals.
 - External Crystal Oscillator (16 MHz): Provides stable clock frequency for precise timing operations.
 - Reset Button: Allows microcontroller reboot without powering off.
 - Power Supply Options: Can be powered through USB or an external 9V DC adapter.
- Power Supply and Electrical Considerations
 - Power Input: Accepts USB (5V) or External Adapter (9V-12V DC).
 - Voltage Regulation: Onboard 7805 voltage regulator for stable 5V output.
 - Protection Features: Reverse polarity and overvoltage protection.

▪ L298 driver

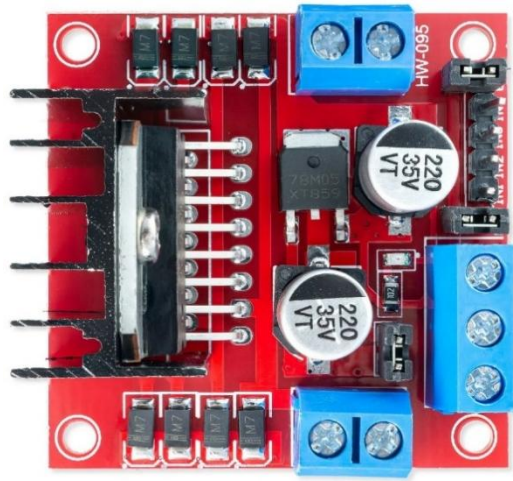


Fig. 2.8. H-Bridge L298

- A high-voltage dual H-Bridge motor driver that controls the speed and direction of DC motors.
- Operates at voltages up to 46V and handles a continuous current of 2A per channel.
- Can control two motors simultaneously with bidirectional movement using PWM.
- Function in the System: Drives the vehicle's motors, ensuring controlled movement and stability.

▪ Sensors in the first layer: -

(a) Ultrasonic “HC-SR04”



Fig. 2.9. HC-SR04

- Detect obstacles and measures distance using high-frequency sound waves (as we will see in ch.7).
- We used three ones in front, right and back of the car.
- Measures the time taken for the echo to return after bouncing off an object.
- We use it to help the vehicle avoid obstacles in real-time.

(b) PIR (Passive Infrared Sensor)

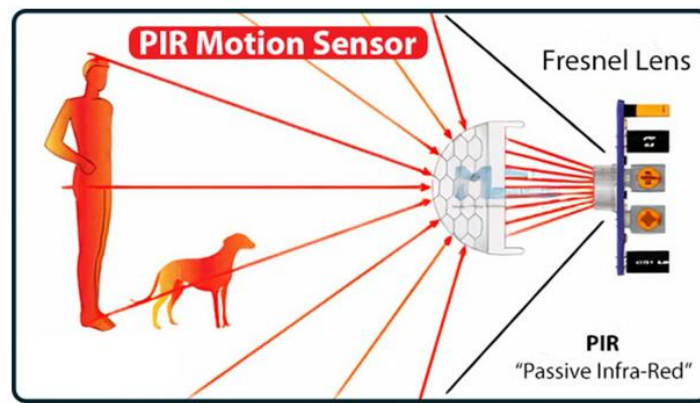


Fig. 2.10. PIR Sensor

- A PIR (Passive Infrared) Sensor detects motion by sensing infrared radiation changes. It consists of:
 - **Pyroelectric Sensor:** Generates a small electrical signal when exposed to infrared radiation.
 - **Fresnel Lens:** Focuses infrared energy onto the pyroelectric sensor to increase detection range.
 - **Differential Detection Circuit:** signals from two pyroelectric elements to detect motion.
 - **Amplifier & Signal Processing:** Amplifies the detected signal and removes noise for accurate motion detection.
 - **Output Stage:** Sends a digital HIGH/LOW signal to trigger external systems like alarms or lights.
- This structure allows the PIR sensor to efficiently detect motion with low power consumption, making it ideal for security and automation applications.

(c) IR (Infrared Sensor)

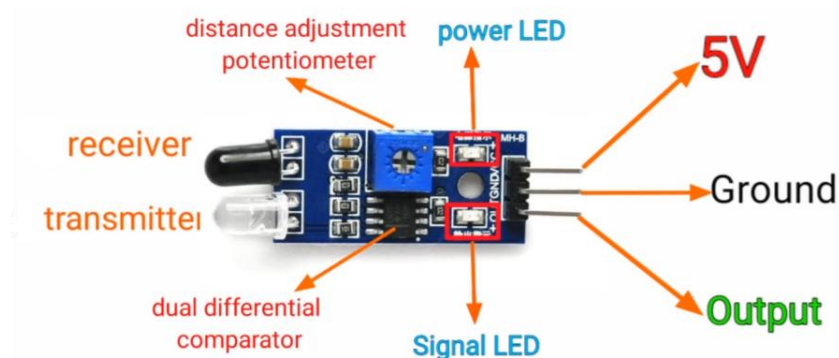


Fig. 2.11. IR Sensor

- An Infrared (IR) Sensor detects objects and measures proximity by emitting and receiving infrared light. It consists of:
 - IR Emitter (LED): Sends out infrared rays.
 - IR Receiver (Photodiode/Phototransistor): Detects reflected IR light and converts it into an electrical signal.
 - Comparator Circuit: Processes the signal and determines object presence.
 - Signal Processing Unit: Sends data to a microcontroller or external system.
- Advantages: Low cost, low power, fast response.
- Limitations: Affected by ambient light, limited range, dependent on surface reflectivity.

(d) Bluetooth module “HC-05”

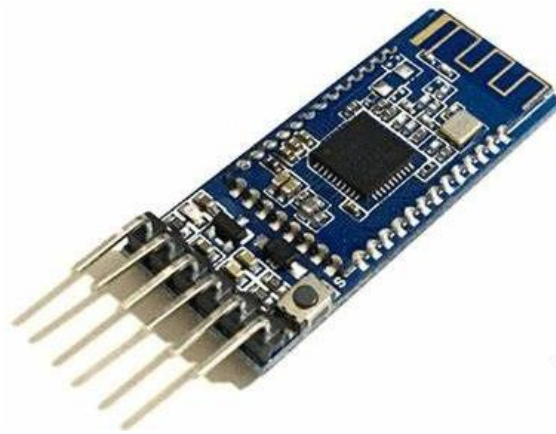


Fig. 2.12. “HC-05” Bluetooth module

- A Bluetooth module operates based on the Bluetooth communication protocol and follows the **IEEE 802.15.1 standard**. It functions using radio frequency (RF) waves in the 2.4 GHz ISM band to establish a wireless link between devices.
- The working process includes:
 - **Pairing**: The module pairs with another Bluetooth-enabled device.
 - **Data Transmission**: It sends or receives data wirelessly through serial communication.
 - **Signal Processing**: The built-in microcontroller processes incoming data.
 - **Wireless Communication**: Uses Bluetooth protocols like Classic Bluetooth (BR/EDR) and Bluetooth Low Energy (BLE) for efficient power usage.

2.2.3 Second Layer

The second layer contains the PCB, where components used for maintenance are soldered. These include the ESP32, NEO M5, LM35, and the PMB180 sensors, with a dedicated battery to power the ESP32.

Fig. 2.13. Car Components Second Layer

▪ Prototype Board

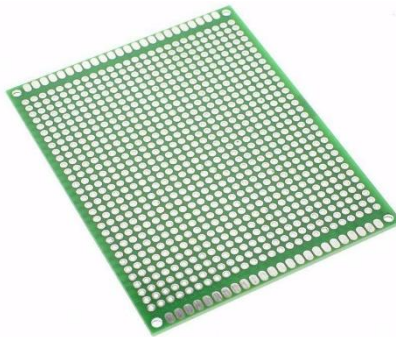


Fig. 2.14. Prototype Board 9x7cm

- It is used in electronics projects to assemble and test electronic circuits in a more permanent way.
- **Specifications and Features:**
 - **Material:** Typically made of epoxy or fiberglass, ensuring durability and strength. The surface is coated with copper pads to facilitate soldering.
 - **Holes:** It contains a grid of evenly spaced holes (usually 2.54 mm apart), making it compatible with standard components like resistors, capacitors, transistors, and integrated circuits.

- **Copper Layer:** The board has small copper pads around each hole on one side, which are used to secure and connect electronic components by soldering.
- **Permanent Use:** Unlike a breadboard, this board is for permanent use, allowing you to assemble circuits that are meant to last.
- **How to Use It:**
 - **Place Components:** Insert components (e.g., resistors, capacitors, transistors) through the holes, so their legs protrude to the other side of the board.
 - **Solder the Legs:** Use a soldering iron to attach the component legs to the copper pads around the holes.
 - **Electrical Connections:** Use wires or solder traces to connect the copper pads according to your circuit design.
 - **Trim Excess:** After soldering, cut off any excess component legs using wire cutters.
 - **Note:** Make sure that excess component legs do not touch each other to prevent short circuits.
- **Advantages of Using a Prototype Board:**
 - **Prototyping and Development:** Ideal for designing and testing circuits before creating a custom printed circuit board (PCB).
 - **Cost-Effective:** Cheaper than manufacturing a custom PCB.
 - **Flexibility:** Allows for modifications and adjustments during the design phase.
 - **Durability:** Once the components are soldered in place, the board becomes sturdy and ready for long-term use.

▪ Esp32

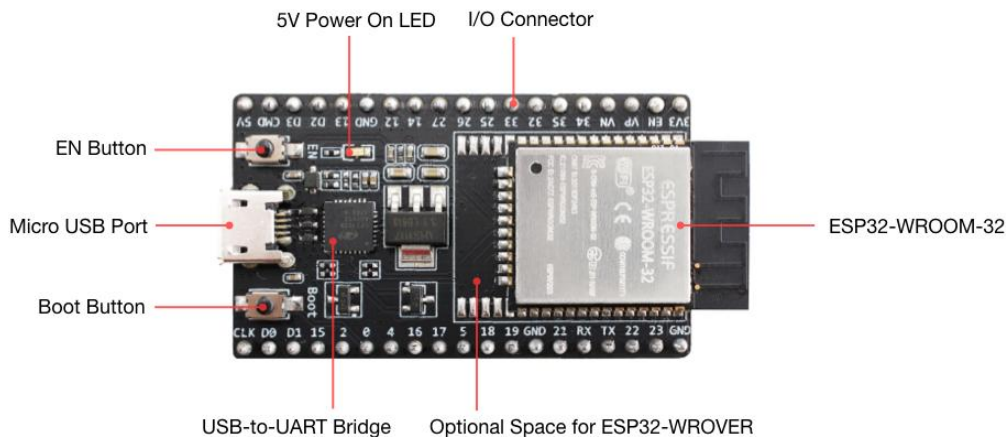


Fig. 2.15. ESP32 WROOM Devkit

- ESP32-WROOM-32 is a powerful System-on-Chip (SoC) developed by Espressif Systems, designed for IoT, embedded systems, and wireless communication applications.
- Processor: Dual-core Ten silica Xtensa LX6 CPU, up to 240 MHz
- Memory & Storage: 448 KB ROM, 520 KB SRAM, supports up to 16 MB SPI Flash.
- Connectivity: Integrated Wi-Fi and Bluetooth (BLE 4.2).
- GPIO & Peripherals: 30 GPIO pins, supports SPI, I2C, UART, PWM, ADC, and DAC.
- Power Efficiency: Dynamic voltage scaling and ultra-low-power (ULP) co-processor for background tasks.

▪ **Sensors in the second layer: -**

(a) GPS “NEO 6M”



Fig. 2.16. “NEO 6M” GPS sensor

- The **NEO-6M GPS module** is a high-performance **Global Positioning System (GPS)** receiver designed for precise location tracking and navigation. It utilizes satellite signals to determine latitude, longitude, altitude, speed, and time synchronization with high accuracy.
- High Sensitivity & Accuracy: Tracks signals at -161 dBm.
- Low Power & Connectivity: UART interface, external antenna, and efficient power usage.

(b) Pressure “PMB 180”



Fig. 2.17. PMB180 Sensor

- PMB180 is a high-precision barometric pressure sensor designed for tires air pressure, weather monitoring, and environmental sensing. It operates using a piezoelectric sensing element to detect minute pressure changes with excellent accuracy and stability.
- Compact & Low Power: Ideal for battery-powered IoT and embedded systems.

(d) Temperature “LM35”



Fig. 2.18. “LM35” Temperature Sensor

- The LM35 is a high-precision analog temperature sensor that provides a linear voltage output proportional to temperature in $^{\circ}\text{C}$, making it ideal for temperature monitoring and control applications.
- High Accuracy: $\pm 0.5^{\circ}\text{C}$ at 25°C , with a wide range of -55°C to $+150^{\circ}\text{C}$.
- Low Power Consumption: Operates on $4\text{V} - 30\text{V}$, making it suitable for battery-powered systems.

2.3 Final version of the car

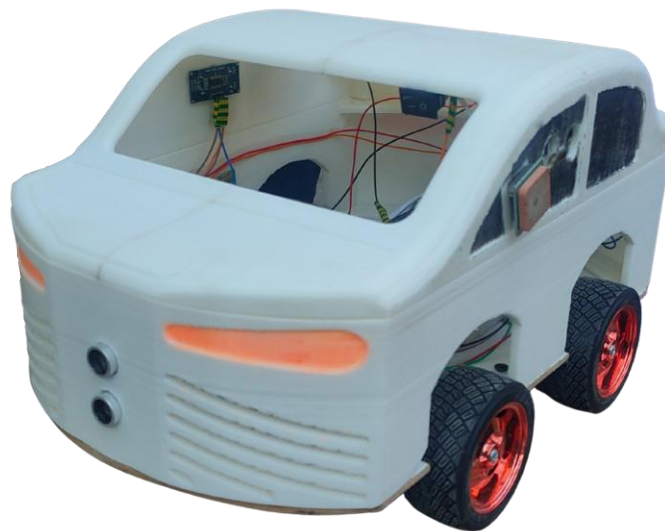


Fig. 2.19. Final version of the car

Chapter 3: Maintenance and IOT

3.1 ESP32

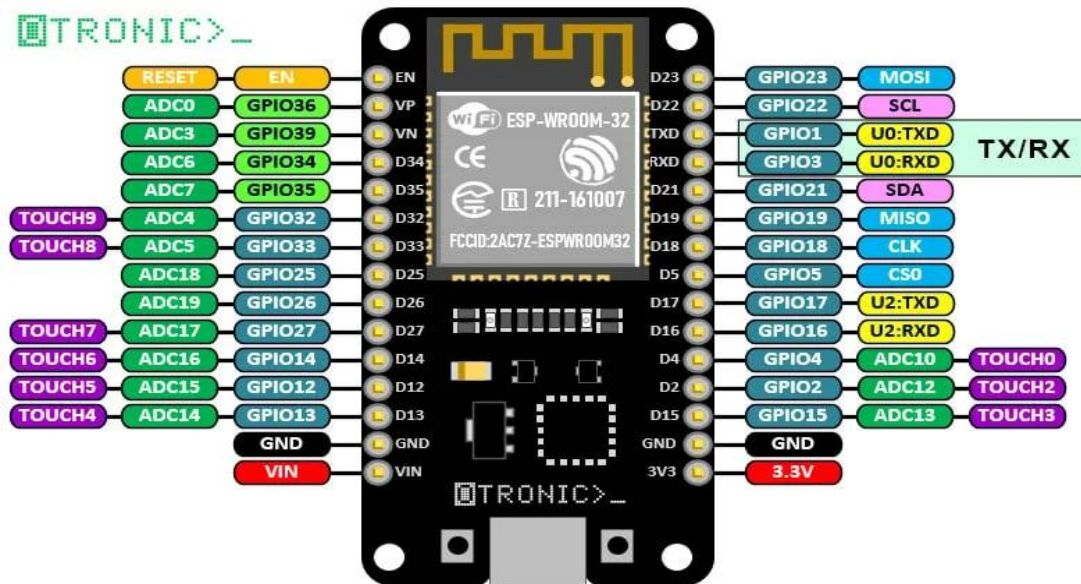


Fig. 3.1. ESP32 Devkit

3.1.1 Comprehensive Overview of ESP32-WROOM-32

ESP32-WROOM-32 is a powerful and highly versatile System-on-Chip (SoC) developed by Espressif Systems. It is designed specifically for applications on the Internet of Things (IoT), embedded systems, and wireless communication. With the integration of Wi-Fi and Bluetooth, along with a wide range of peripherals, it has become a popular choice for various fields, including industrial automation, smart home systems, and wearable devices. ESP32-WROOM-32 is widely recognized for its balance between performance, power efficiency, and cost-effectiveness, making it suitable for both commercial and educational projects.

■ Processing Power:

The ESP32-WROOM-32 is equipped with a Tensilica Xtensa LX6 processor, which comes in dual-core or single-core versions, running at clock speeds of up to 240 MHz. This high processing power enables the module to achieve up to 600 DMIPS (Dhrystone Million Instructions Per Second), making it ideal for computationally demanding applications. Additionally, it features an Ultra-Low Power (ULP) co-processor, which allows for efficient execution of background tasks while consuming minimal energy.

▪ Memory & Storage:

In terms of memory, the ESP32-WROOM-32 includes 448 KB of ROM and up to 520 KB of SRAM, providing sufficient space for code execution and data storage. Additionally, the module supports external SPI flash memory expansion of up to 16 MB, allowing for more complex applications that require additional storage. Certain versions of ESP32 also include embedded PSRAM, which is beneficial for applications involving image processing, artificial intelligence (AI), and complex data handling.

▪ GPIO and Peripherals:

The ESP32-WROOM-32 module provides 30 General-Purpose Input/Output (GPIO) pins, making it highly adaptable for interfacing with various external components such as sensors, displays, and actuators. It supports multiple communication protocols, including SPI, I2C, I2S, UART, PWM, and ADC/DAC, making it ideal for a wide range of embedded applications.

For analog applications, the ESP32 includes a 12-bit ADC (Analog-to-Digital Converter) and an 8-bit DAC (Digital-to-Analog Converter), enabling accurate sensor data acquisition and signal processing. Additionally, the 16-channel PWM (Pulse Width Modulation) feature makes it suitable for motor control, LED dimming, and audio applications.

▪ Operating System and Power Management

The ESP32-WROOM-32 operates on FreeRTOS, which allows for efficient multi-threaded execution and real-time task scheduling. This enables developers to run multiple processes simultaneously while ensuring reliable system performance.

For event-driven applications, the ESP32 supports hardware interruptions, allowing for rapid response times in real-time systems. Additionally, dynamic voltage scaling is implemented to optimize power consumption based on the processing load, making it an energy-efficient solution for battery-powered devices

Feature	ESP32	ESP8266	Arduino UNO	Raspberry
CPU Speed	160-240 MHz	80-160 MHz	16 MHz	1.5 GHz
Wi-Fi	Yes	Yes	No	Yes
Bluetooth	Yes (BLE 4.2)	No	No	Yes (BLE 5)
GPIO pins	30	17	14	40
ADC	12-bit	10-bit	10-bit	12-bit
Power mode	Deep sleep Light sleep	Light sleep	No sleep mode	Low power mode
Cost	Low	Very low	Low	High

Table 3.1. Compare between IOT microcontrollers

From the previous table, the ESP32 stands out as the best choice among other microcontrollers due to its powerful dual-core processor, integrated Wi-Fi and Bluetooth, extensive GPIO support, and efficient low-power modes. Unlike its predecessors and competitors, it offers a perfect balance between performance, connectivity, and cost-effectiveness, making it the ideal solution for IoT, automation, and embedded systems applications. Additionally, its advanced features and connectivity options make it highly suitable for our graduation project, the Smart Vehicle Management System, enabling efficient real-time monitoring and control.

3.1.2 Programming ESP32

ESP32 is supported by multiple development frameworks:

Feature	ESP-IDF (Espressif IoT Development Framework)	Arduino IDE	MicroPython
Programming Language	C, C++	C, C++	Python
Complexity	High (requires expertise)	Low (beginner-friendly)	Low (easy to learn)
Ease of Use	Challenging	Easy	Very Easy
Library Support	Extensive	Large community support	Limited but growing
Memory Usage	Efficient, but requires manual management	Higher than ESP-IDF	Higher due to Python runtime
Connectivity Support	Full support for Wi-Fi, Bluetooth, MQTT, etc.	Supports most features	Supports Wi-Fi, MQTT (but with limitations)

Table 3.2. Compare between ESP32 development frameworks

From these options, Arduino IDE stands out as the best choice for developing with ESP32 due to its ease of use, extensive library support, and strong community, making it ideal for both beginners and professionals working on projects like the Smart Vehicle Management System.

- **To install Arduino IDE:**  [Software | Arduino](#)
- **To add ESP32 Board Support:**
 1. Open Arduino IDE and go to File > Preferences.
 2. In the “Additional Board Manager URLs” field, add the following URL: <https://2u.pw/eSc9rWjl>
 3. Click “OK”.

- **To install ESP32 Board:**

1. Go to Tools > Board > Boards Manager.
2. Search for “ESP32” and install the “ESP32 by Espressif Systems” package.

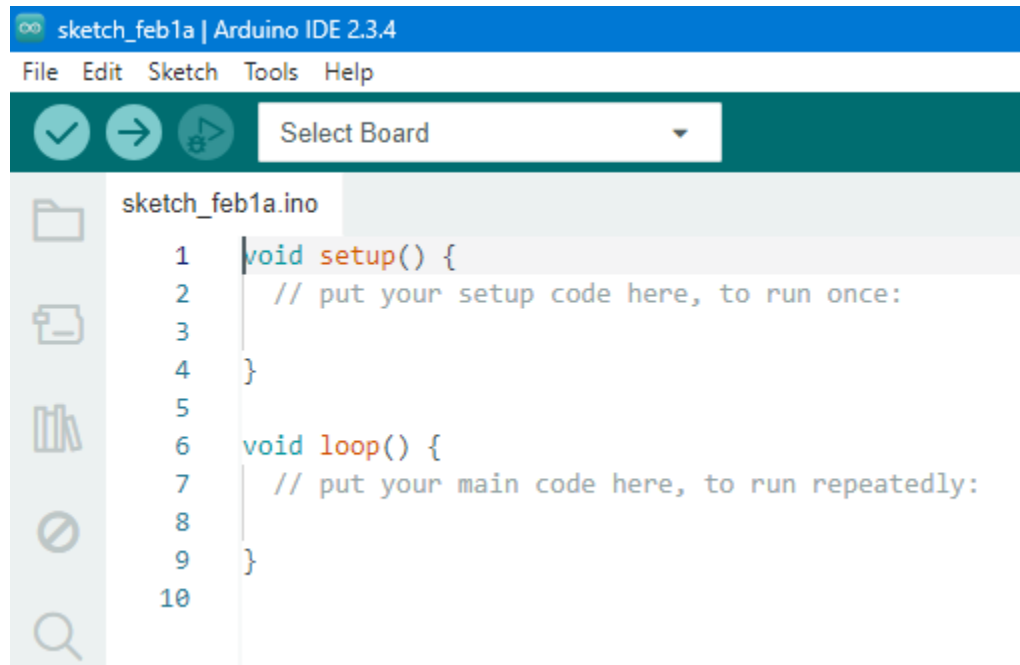


Fig. 3.2. Arduino IDE

3.2 Node-Red

Node-RED is a powerful, flow-based development tool designed specifically for Internet of Things (IoT) applications. Built on Node.js, it provides a simple and intuitive interface for creating workflows by connecting devices, APIs, and online services. With its drag-and-drop interface, users can easily design IoT systems without extensive coding experience.

Node-RED simplifies the integration of various IoT components, including sensors, actuators, cloud platforms, and databases, enabling seamless communication and data processing. Its flexibility and scalability make it an ideal solution for prototyping, automation, and real-time monitoring in IoT projects, helping developers focus on innovation rather than complex system integration.

- **To install Node-red:**

1. Prerequisite install Node.js on your system: <https://nodered.org/docs/getting-started/windows>

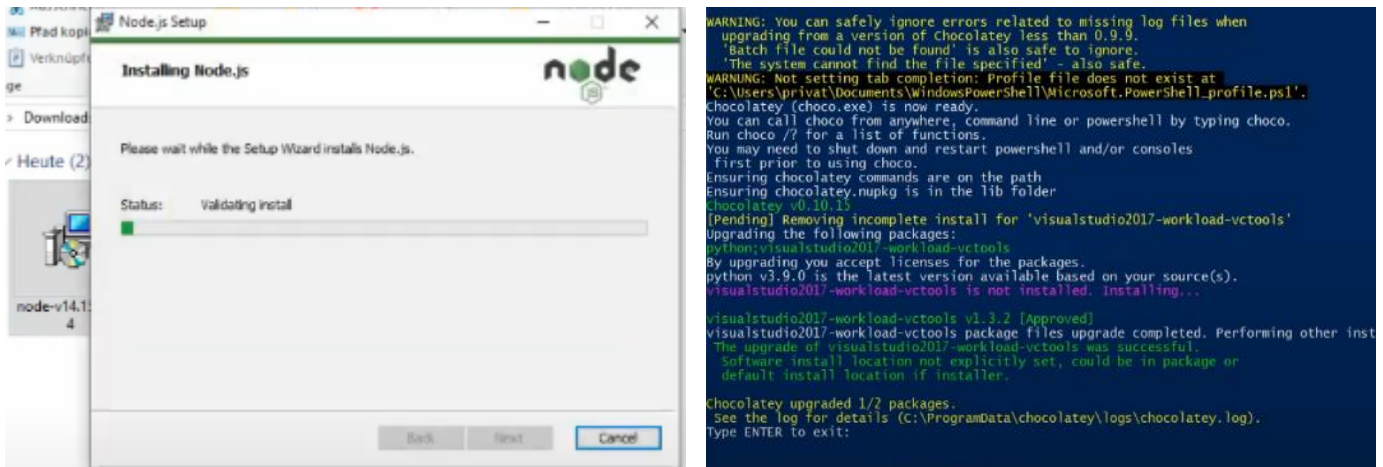


Fig. 3.3. Node.js

Once installed, open a command prompt and run the following command to ensure Node.js and npm are installed correctly.

Using PowerShell: `node --version; npm --version`

Using cmd: `node --version npm --version`

You should receive back output that looks similar to:

```
v18.15.0
9.5.0
```

2. Install Node-RED

Installing Node-RED as a global module adds the command node-red to your system path. Execute the following at the command prompt:

```
npm install -g --unsafe-perm node-red
```

3. Run Node-red

Once installed, the simple way to run Node-RED is to use the node-red command in a command prompt: If you have installed Node-RED as a global npm package, you can use the node-red command:

```
C:>node-red
```

```
node-red
Welcome to Node-RED
=====
1 Feb 22:01:49 - [info] Node-RED version: v4.0.5
1 Feb 22:01:49 - [info] Node.js version: v22.12.0
1 Feb 22:01:49 - [info] Windows_NT 10.0.22631 x64 LE
1 Feb 22:01:50 - [info] Loading palette nodes
1 Feb 22:01:55 - [info] Dashboard version 3.6.5 started at /ui
1 Feb 22:01:56 - [info] Settings file : C:\Users\Mahmoud 3smat\.node-red\settings.js
1 Feb 22:01:56 - [info] Context store : 'default' [module=memory]
1 Feb 22:01:56 - [info] User directory : \Users\Mahmoud 3smat\.node-red
1 Feb 22:01:56 - [warn] Projects disabled : editorTheme.projects.enabled=false
1 Feb 22:01:56 - [info] Flows file : \Users\Mahmoud 3smat\.node-red\flows.json
1 Feb 22:01:56 - [info] Server now running at http://127.0.0.1:1880/
1 Feb 22:01:56 - [warn]

-----
Your flow credentials file is encrypted using a system-generated key.

If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
-----

1 Feb 22:01:56 - [info] Starting flows
1 Feb 22:01:56 - [info] Started flows
1 Feb 22:01:57 - [info] [mqtt-broker:aa12e9813344faa2] Connected to broker: mqtt://Test.mosquitto.org:1883
```

Fig. 3.4. Open node-red

Copy <http://127.0.0.1:1880/> and paste it in google search to open node-red page

The broker of MQTT we use is: `mqtt://Test.mosquitto.org:1883`

3.3 Sensors

Sensors play a vital role in modern technology, acting as the bridge between the physical world and digital systems by collecting essential data for accurate monitoring and control.

3.3.1 GPS Sensor “NEO 6M”



Fig. 3.5. GPS sensor

The GPS (Global Positioning System) sensor is a critical component in modern IoT projects, enabling accurate real-time location tracking and navigation. It works by receiving signals from satellites to determine geographical coordinates.

▪ What is a GPS Sensor?

A GPS sensor is a device that receives signals from satellites orbiting the Earth, allowing it to determine its exact location (latitude, longitude, and altitude) on the globe. This sensor uses a network of at least 24 satellites, operated by the U.S. Department of Defense, to triangulate the sensor's position with high accuracy.

▪ How GPS Works?

- **Satellite Communication:** GPS sensors require signals from at least four satellites to calculate a device's position with high accuracy using trilateration.
- **Time Synchronization:** GPS systems are synchronized with atomic clocks, ensuring highly accurate time and position data.
- **Data Output:** Most GPS modules provide data in the standard NMEA (National Marine Electronics Association) format, which includes coordinates, altitude, speed, and time.

▪ Why Use GPS in the Project?

In our project, the GPS sensor plays a crucial role in enhancing functionality by adding location-based capabilities and it allows Data Logging to record movement or travel paths.

▪ Key Features of GPS Sensors

- **High Accuracy:** Pinpoint locations within a few meters.
- **Wide Coverage:** Operates globally, independent of internet connectivity.
- **Low Power Consumption:** Suitable for battery-operated systems.
- **Ease of Integration:** Compact modules and standard communication protocols make it simple to integrate GPS with ESP32.

- **GPS Sensor Connections**

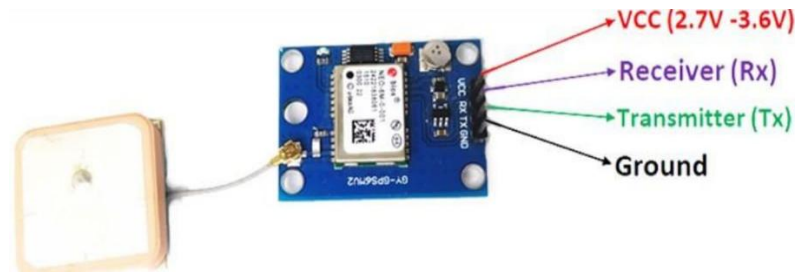


Fig. 3.6. GPS Sensor connection

- **Pins Already Use in project:**

Neo 6M Pins	ESP32 Pins
VCC	2.7-3.6 Volt
GND	GND
Tx	D17
Rx	D16

Table 3.3. Connections of NEO 6M pins

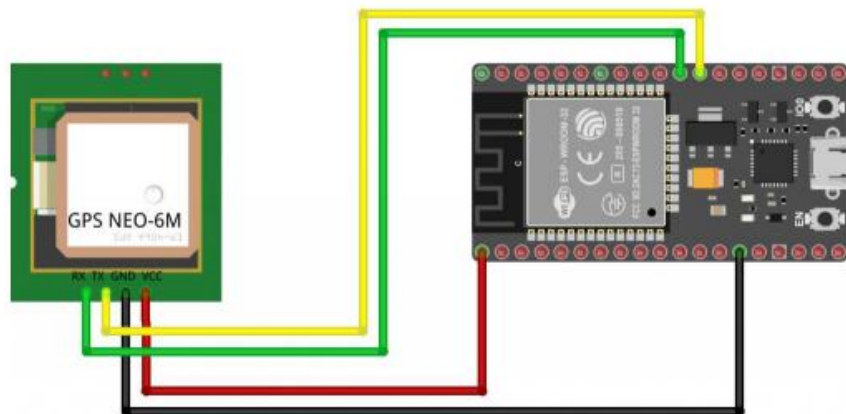


Fig. 3.7. GPS Sensor connection with ESP32

UART (**Universal Asynchronous Receiver-Transmitter**) is a serial communication protocol used to transmit and receive data between devices. It operates asynchronously, meaning it does not require a clock signal, relying instead on start and stop bits for synchronization.

When connecting a **GPS module** to a microcontroller or computer, **UART** is commonly used for communication. The GPS module continuously sends NMEA sentences (data strings) containing location, time, and velocity information. The microcontroller receives this data through the **RX (Receive) pin**, while the GPS module transmits data via its **TX (Transmit) pin** with a baud rate of 9600 bps.

3.3.2 Pressure Sensor “PMB 180”



Fig. 3.8. PMB180 Sensor

■ Pressure Sensor Overview

A pressure sensor is a critical component in many IoT and embedded systems projects, enabling the measurement of force exerted by a fluid (liquid or gas) on the surface. In our project, the pressure sensor plays a vital role in monitoring and controlling environmental or system conditions. Below is a detailed explanation of its features, working principles, and applications.

■ How does Pressure Sensors Work?

- **Sensing Element:** Pressure sensors use materials like piezoelectric crystals, capacitive plates, or strain gauges to measure pressure changes.
- **Conversion:** The sensor converts the physical pressure into an electrical signal proportional to the applied force.
- **Signal Processing:** An onboard circuit processes the signal and outputs it as a voltage, current, or digital data (I2C, SPI, or UART protocols).

■ Advantages of Using Pressure Sensors

- **High Sensitivity:** Can detect minute changes in pressure.
- **Wide Range:** Suitable for applications requiring low-pressure (e.g., weather systems) or high-pressure (e.g., industrial systems) monitoring.
- **Low Power Consumption:** Ideal for battery-operated IoT devices.

■ BMP180 Sensor Connections

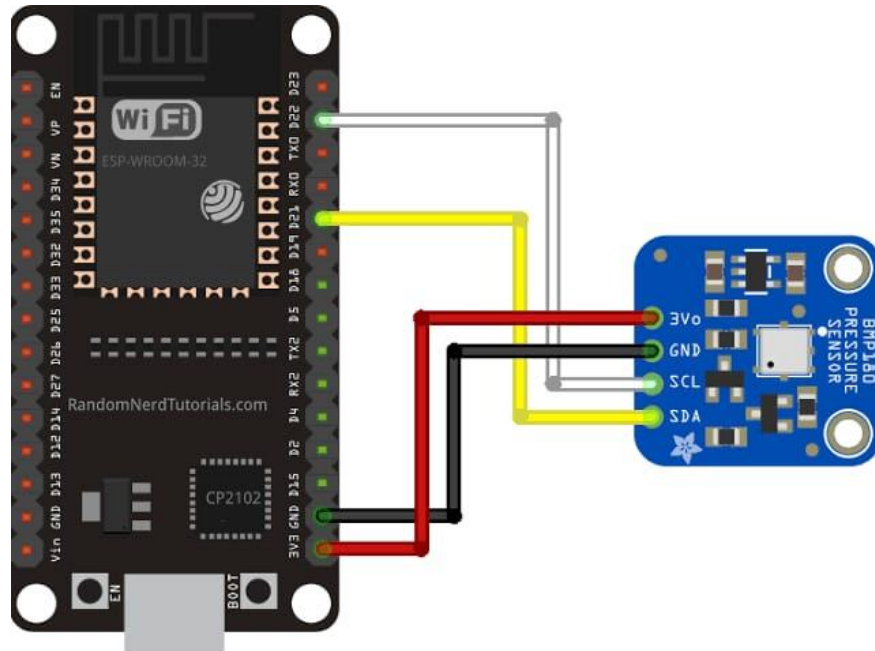


Fig. 3.9. PMB180 connection to ESP32

■ Pins Already Use in project:

PMB180 Pins	ESP32 Pins
3V0	3V3
GND	GND
SCL	D22 (SCL)
SDA	D21 (SDA)

Table 3.4. Connections of PMB180 pins

SCL (Serial Clock Line) and **SDA (Serial Data Line)** are the two main communication lines used in the **I²C (Inter-Integrated Circuit) protocol**. SCL carries the clock signal, while SDA transmits data between the master (e.g., a microcontroller) and slave devices (e.g., sensors).

The **PMB180**, a barometric pressure sensor, uses the **I²C interface** for communication. To connect it:

1. **The SCL of PMB180** should be connected to the **SCL of the microcontroller**.
2. **The SDA of PMB180** should be connected to the **SDA of the microcontroller**.
3. Pull-up resistors (typically **4.7k Ω –10k Ω**) should be used on both SCL and SDA lines to ensure proper signal levels.
4. A common **GND** connection is required.

Once connected, the microcontroller can communicate with the **PMB180** by sending commands over the I²C bus to read pressure and temperature data.

3.3.3 Temperature Sensor “LM35”



Fig. 3.10. LM35 Sensor

▪ LM35 Temperature Sensor

The LM35 Temperature Sensor is a precision analog sensor widely used in a variety of applications, from embedded systems to environmental monitoring. Known for its accuracy, simplicity, and reliability, the LM35 is an essential part of the Smart Vehicle Management System, where it plays a critical role in monitoring the temperature of various components within the vehicle.

■ What is the LM35 Temperature Sensor?

The LM35 is an analog temperature sensor designed by Texas Instruments. It provides an accurate and linear output, with an output voltage directly proportional to the centigrade temperature. For every 1°C change in temperature, the output voltage changes by 10 mV. This feature makes the LM35 highly suitable for use in precise temperature measurement applications.

■ Key Features of the LM35 Sensor

- **High Precision:** The LM35 is known for its high accuracy. Unlike other sensors that may drift or have higher tolerances, the LM35 provides accurate measurements with a typical error of $\pm 0.5^{\circ}\text{C}$ at 25°C . This precision makes it ideal for applications that require fine temperature control and monitoring.
- **Wide Operating Range:** The LM35 operates over a wide temperature range, typically from -55°C to $+150^{\circ}\text{C}$, allowing it to be used in both low and high-temperature applications. Whether you're monitoring the temperature of a vehicle's engine or the interior climate of the cabin, the LM35 can handle a broad spectrum of temperature readings.
- **Linear Output:** The LM35 has a linear output characteristic, meaning that the relationship between the temperature and the output voltage is direct and predictable. For every degree Celsius change in temperature, the output voltage changes by 10 mV. This linearity ensures easy calibration and straightforward interpretation of the data.
- **Low Power Consumption:** The LM35 operates on a very low supply voltage, typically between 4V and 30V, making it suitable for battery-operated systems and low-power embedded applications. Its low power consumption contributes to the overall energy efficiency of the Smart Vehicle Management System.
- **Simple to Use:** The LM35 is an analog output sensor, making it incredibly easy to interface with microcontrollers like Arduino, ESP32, or Raspberry Pi. Its simplicity and direct voltage output make it a popular choice for embedded system developers, especially when temperature data needs to be processed in real-time.

- LM35 Sensor Connections

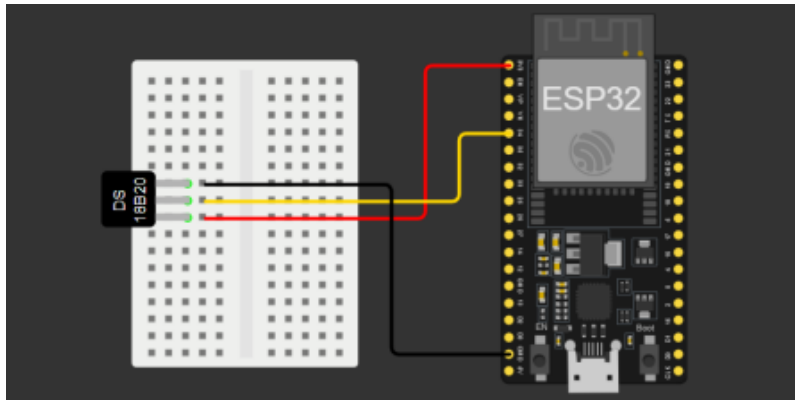


Fig. 3.11. LM35 Sensor connection to ESP32

- Pins Already Use in project:

LM35 Pins	ESP32 Pins
VIN	3.3-5 V
GND	GND
VOUT	D34

Table 3.5. Connections of LM35 pins

3.4 Code of maintenance

These are the necessary libraries for WiFi, MQTT, GPS, I2C communication, and the BMP085 sensor.

```
#include <WiFi.h>           // Include WiFi library
#include <PubSubClient.h>    // Include PubSubClient library for MQTT
#include <TinyGPSPlus.h>     // Include TinyGPS++ library for GPS
#include <Wire.h>            // Include Wire library for I2C communication
#include <Adafruit_Sensor.h> // Include Adafruit Sensor library
#include <Adafruit_BMP085_U.h> // Include Adafruit BMP085 library
```

These are the WiFi credentials (SSID and password) and MQTT broker settings, including the broker IP, port, and topics for publishing and subscribing.

```
const char* ssid = "A71a Team";           // WiFi network name
const char* password = "0123456789";      // WiFi password
const char* mqtt_server = "Test.mosquitto.org"; // MQTT Broker IP address
const char* outTopic = "/kdi/inTopic";     // Topic to publish messages
const char* inTopic = "/kdi/outTopic";     // Topic to subscribe to
const char* sensorTopic = "esp32/Sensors/Graduation Project"; // Topic to publish sensor data
const int mqtt_port = 1883;               // MQTT Broker port
```

Defines the RX and TX pins for the GPS module and its baud rate, along with the analog input pin for the LM35 temperature sensor.

```
const int RXPin = 16, TXPin = 17;        // GPS module RX and TX pins
const uint32_t GPSBaud = 9600;           // GPS module baud rate
const int lm35Pin = 34;                  // Analog input pin for LM35 sensor
```

Creates objects for WiFi, MQTT, GPS, and the BMP085 sensor to be used later in the code.

```
WiFiClient espClient;                    // Create WiFi client
PubSubClient client(espClient);          // Create MQTT client
TinyGPSPlus gps;                          // Create TinyGPS++ object
Adafruit_BMP085_Unified bmp = Adafruit_BMP085_Unified(10085); // Create BMP085 sensor object
```

Declares global variables to store the MQTT client name, message buffer, GPS coordinates, and a timer for publishing GPS data every 10 seconds.

```
char clientName[50];                     // Buffer for MQTT client name
char msg[100];                           // Buffer for MQTT messages
float lat, lon;                           // Variables for latitude and longitude
unsigned long previousMillis = 0;         // Store last time GPS data was published
const long interval = 10000;             // Interval at which to publish GPS data (10 seconds)
```


The `setup()` function initializes the serial monitor, connects to WiFi, sets up MQTT, initializes the BMP085 sensor, and connects to the MQTT broker.

```
void setup()
{
    Serial.begin(9600);                // Start Serial Monitor at 9600 baud rate
    Serial1.begin(GPSBaud, SERIAL_8N1, RXPin, TXPin); // Initialize GPS hardware serial
    setup_wifi();                      // Connect to WiFi

    // Auto-generate a unique client name based on ESP32 MAC address
    uint64_t chipid = ESP.getEfuseMac(); // Get unique chip ID
    String sChipID = String((uint16_t)(chipid >> 32), HEX) + String((uint32_t)chipid, HEX);
    sChipID.toCharArray(clientName, sChipID.length() + 1);
    Serial.println(clientName);

    client.setServer(mqtt_server, mqtt_port); // Set MQTT server and port
    client.setCallback(callback);             // Set callback function for incoming messages

    // Initialize BMP085 sensor
    if (!bmp.begin())
    {
        Serial.print("Could not find a valid BMP085 sensor, check wiring!");
        while (1); // Stay here if sensor initialization fails
    }

    // Connect to MQTT Broker
    while (!client.connected())
    {
        if (client.connect(clientName)) // Unique MQTT client ID
        {
            Serial.println("Connected to MQTT Broker");
            client.subscribe(inTopic);    // Subscribe to topic
        }
        else
        {
            Serial.println("Retrying MQTT connection...");
            delay(2000);                  // Wait 2 seconds before retrying
        }
    }
}
```

This function connects the ESP32 to the WiFi network and prints the assigned IP address.

```
void setup_wifi()
{
    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);

    WiFi.begin(ssid, password);           // Start WiFi connection

    while (WiFi.status() != WL_CONNECTED) // Wait until connected
    {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi connected");
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
}
```

This function handles incoming MQTT messages and prints them to the serial monitor.

```
void callback(char* topic, byte* payload, unsigned int length){
    Serial.print("Message arrived [");
    Serial.print(topic);
    Serial.print("] ");
    for (int i = 0; i < length; i++){
        Serial.print((char)payload[i]);           // Print message payload
    }
    Serial.println();
}
```

This function attempts to reconnect to the MQTT broker if the connection is lost. It retries every 5 seconds until it succeeds.

```
void reconnect(){
    while (!client.connected()) {
        Serial.print("Attempting MQTT connection...");
        if (client.connect(clientName)){           // Try to connect
            Serial.println("connected");
            client.publish(outTopic, "Reconnected"); // Publish reconnect message
        }
    }
}
```

```

        client.subscribe(inTopic);                // Subscribe to topic
    } else {
        Serial.print("failed, rc = ");
        Serial.print(client.state());
        Serial.println(" try again in 5 seconds");
        delay(5000);                            // Wait before retrying
    }
}
}
}

```

This function checks if the GPS data is valid. If valid, it retrieves latitude and longitude and prints them to the serial monitor.

```

void displayInfo() {
    if (gps.location.isValid()){                // Check if GPS location is valid
        lat = gps.location.lat();
        lon = gps.location.lng();
        Serial.print("Latitude: ");
        Serial.print(lat, 6);
        Serial.print(", Longitude: ");
        Serial.println(lon, 6);
    } else {
        Serial.println("Invalid GPS data.");
    }
}
}

```

Ensures MQTT connection is maintained.

Reads and processes GPS data.

Publishes GPS coordinates to the MQTT broker every 10 seconds.

```

void loop()
{
    if (!client.connected()){
        reconnect();                            // Ensure MQTT connection
    }
    client.loop();                             // Handle MQTT communication

    while (Serial1.available() > 0)
    {
        gps.encode(Serial1.read());            // Feed GPS data to TinyGPS++
    }

    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval)
    {

```

```

previousMillis = currentMillis;
displayInfo(); // Update latitude and longitude
if (gps.location.isValid() && client.connected())
{
    sprintf(msg, "{\"lat\":%.5f,\"lon\":%.5f}", lat, lon);
    Serial.println(msg); // Print GPS data
    client.publish(outTopic, msg); // Publish GPS data to MQTT broker
}
}
.
.
.

```

Reads temperature from the LM35 sensor by converting the analog input to voltage and then to temperature.
 Reads pressure and temperature from the BMP085 sensor.
 Publishes both temperature and pressure data to the MQTT broker.
 Delays for 1 second before taking new readings.

```

.
.
.
// Read temperature from LM35 sensor
int adcValue = analogRead(lm35Pin); // Read analog value from LM35
float voltage = adcValue * 3.3 / 4095.0; // Convert analog value to voltage
float lm35Temperature = voltage * 100.0; // Convert voltage to temperature

sensors_event_t event; // Create sensor event object

// Read pressure from BMP085 sensor
bmp.getEvent(&event);
if (event.pressure)
{
    float bmpTemperature;
    bmp.getTemperature(&bmpTemperature); // Get temperature from BMP085 sensor

    // Create payload with both temperature and pressure data
    String payload = "Pressure: " + String(event.pressure) + " Pa\n";
    payload += "LM35 Temperature: " + String(lm35Temperature) + " °C\n";
    payload += "BMP085 Temperature: " + String(bmpTemperature) + " °C";

    // Publish combined data to MQTT topic
    client.publish(sensorTopic, payload.c_str());
    Serial.println(payload); // Print combined data to Serial Monitor
}

delay(1000); // Wait 1 second before next reading
}

```

■ Summary:

- The ESP32 connects to a WiFi network and an MQTT broker.
- It retrieves GPS data and publishes it every 10 seconds.
- It reads temperature from the LM35 sensor and both temperature and pressure from the BMP085 sensor, then publishes them via MQTT.
- It ensures stable MQTT connectivity by reconnecting if the connection is lost.

3.5 Node-red maintenance

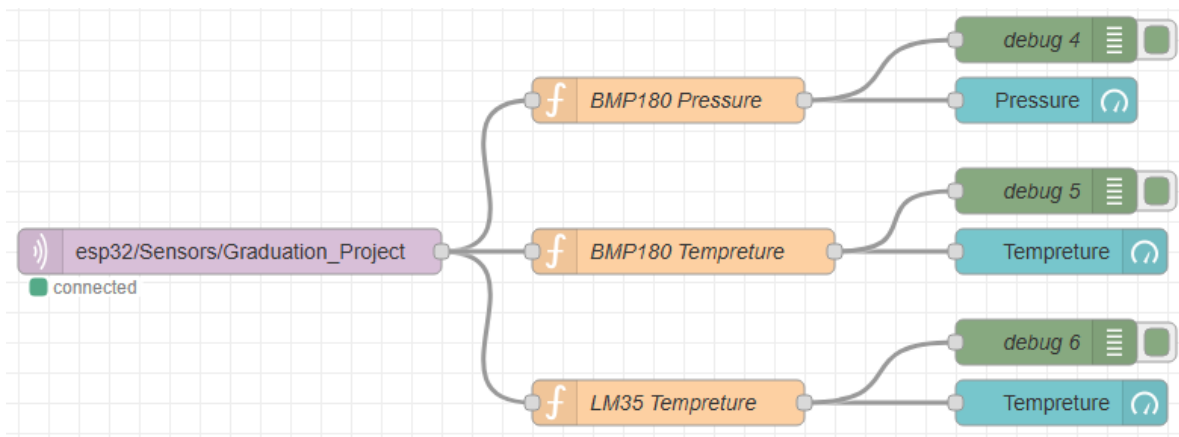


Fig. 3.12. Node-red receiving sensors readings

First, I select **mqtt in**, then I double-click on it. After that, I entered the Topic as written in the code “esp32/Sensors/Graduation_Project”.

Next, I click on the ‘+’ icon in the Server field.

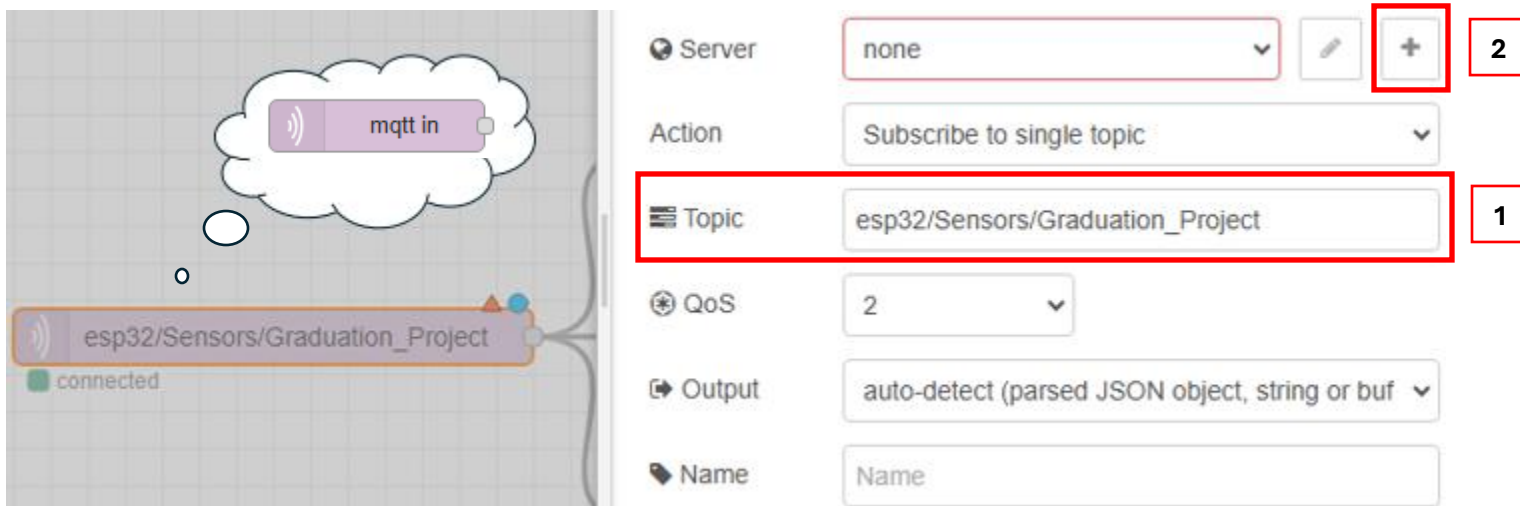
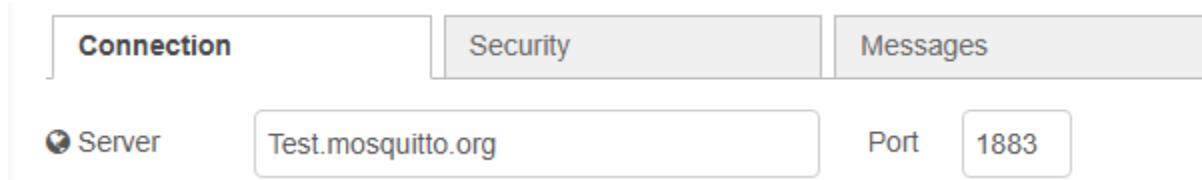


Fig. 3.13. Mqtt in server handle in node-red

Remember it from cmd:

```
2 Feb 16:00:02 - [info] [mqtt-broker:aa12e9813344faa2] Connection failed to broker: mqtt://Test.mosquitto.org:1883
2 Feb 16:00:17 - [info] [mqtt-broker:aa12e9813344faa2] Connected to broker: mqtt://Test.mosquitto.org:1883
```

The server is “Test.mosquitto.org” and the port is “1883” and it written in code.



I previously sent all the values together as a string through the code. To separate them and convert them into readable values, I needed three functions. I wrote the following code in each function:

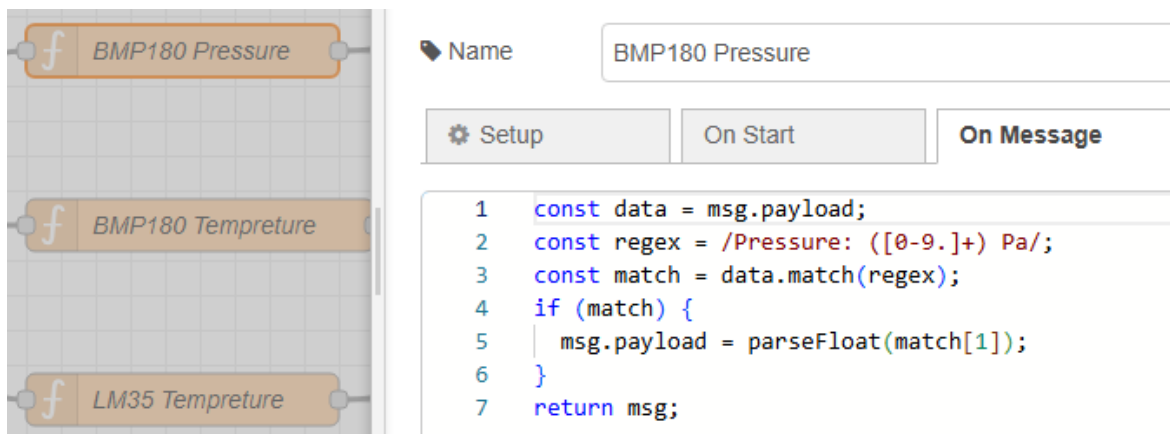


Fig. 3.14. Code of BMP180 Pressure function in node-red

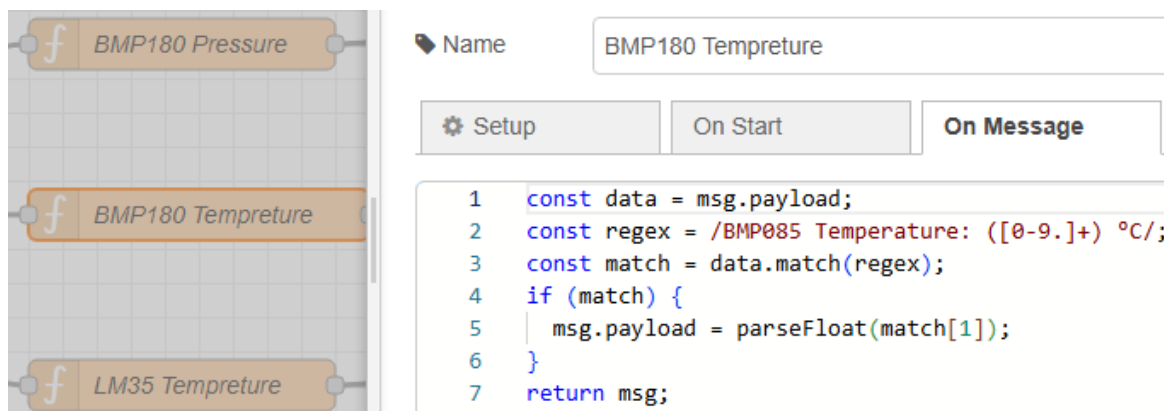


Fig. 3.15. Code of BMP180 Temperature function in node-red

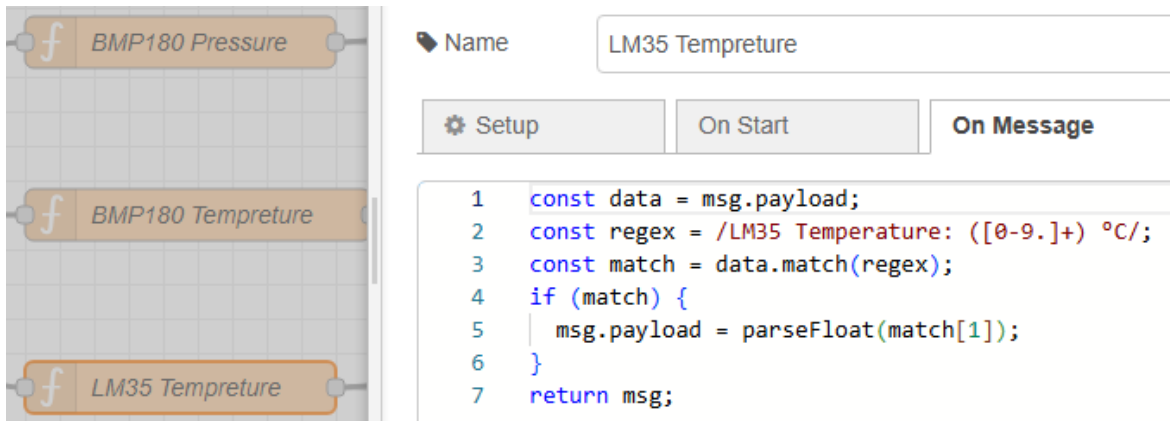


Fig. 3.16. Code of LM35 Temperature function in node-red

Select gauge and handle it by double clicking, then choose ‘+’ to create new group

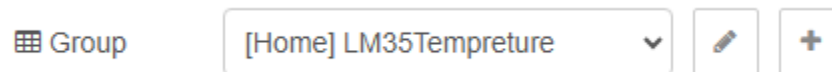


and then do it



This creates a vertical section where I can add everything related to the “BMP180 sensor”. Similarly, I can do the same for the “LM35 sensor” or any other sensors if needed. Each vertical section will be displayed horizontally on the Tab.

Choose the same Tab but different groups for “BMP180” and “LM35”:



Then, enter any name you want and make sure the “Value format” contains {{value}}. Choose the appropriate unit and sensor range and change the color if needed.

Label

Value format

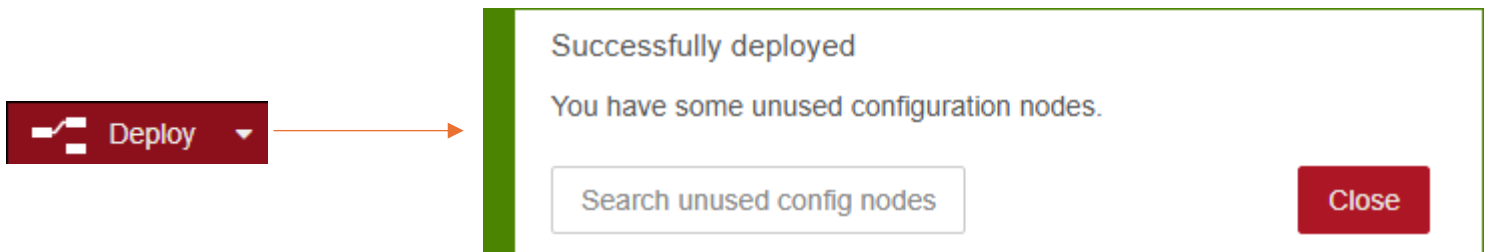
Units

Range min max

Colour gradient 

Fig. 3.17. Editing the gauge

Don't forget to click on Deploy after changing.



To go to the dashboard, write this link in a new Tab on any browser.

 Node-RED Dashboard - <http://127.0.0.1:1880/ui>

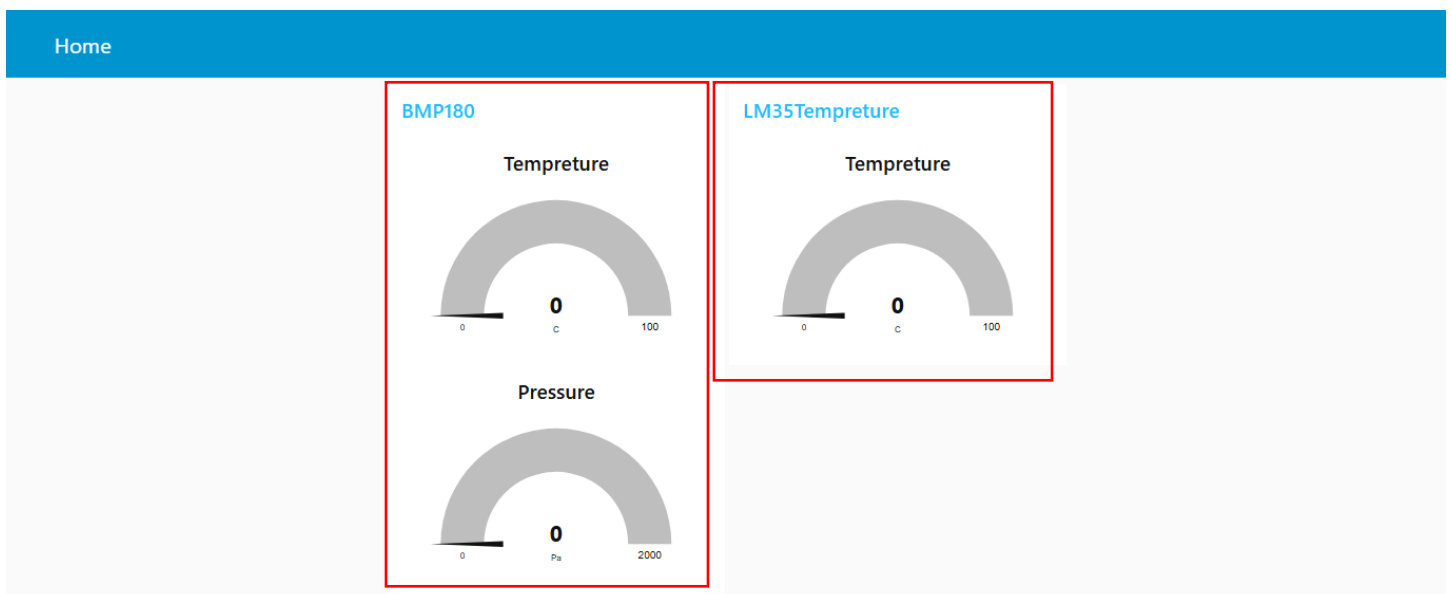


Fig. 3.18. Reading sensors on “Home” Tab in node-red before burning

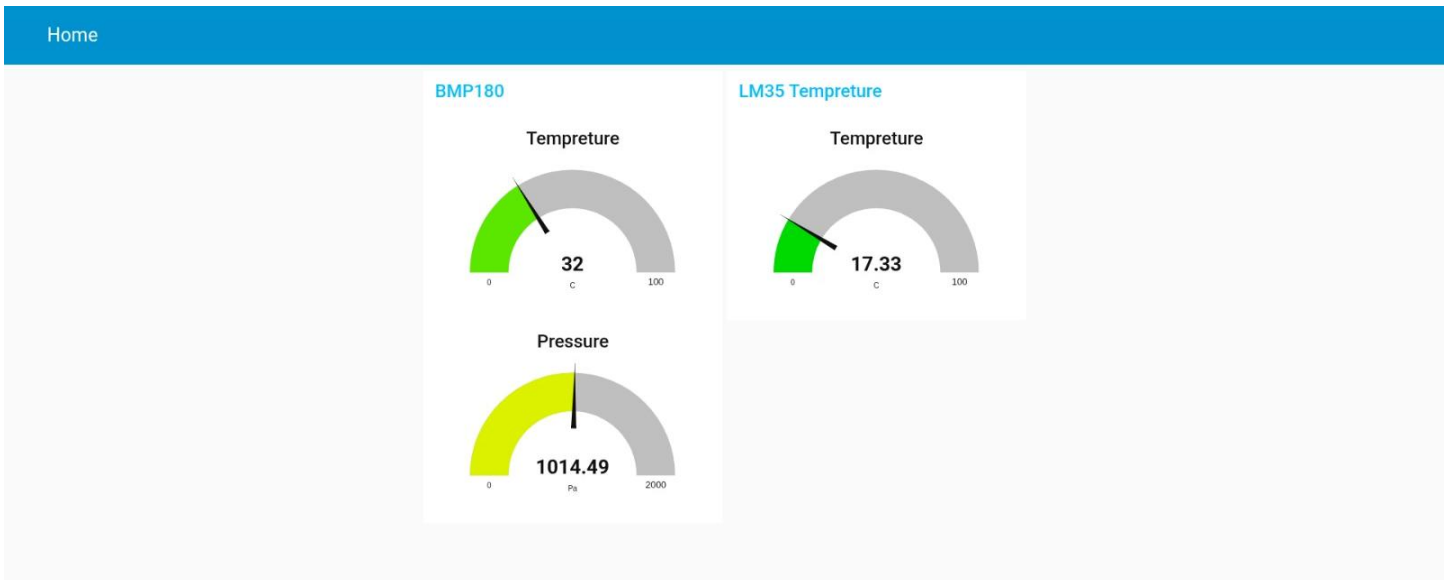


Fig. 3.19. Reading sensors on “Home” Tab in node-red after burning

```
/dev/ttyACM0
Connecting to TP-Link
....
WiFi connected
IP address: 192.168.1.103
783f4876efd0
Connected to MQTT Broker
Pressure: 1014.66 Pa
LM35 Temperature: 15.96 °C
BMP085 Temperature: 30.20 °C
Pressure: 1014.60 Pa
LM35 Temperature: 16.04 °C
BMP085 Temperature: 30.20 °C
Pressure: 1014.63 Pa
LM35 Temperature: 16.52 °C
BMP085 Temperature: 30.20 °C
Pressure: 1014.66 Pa
```

Autoscroll ☐ Show timestamp Newline 9600 baud Clear output

Fig. 3.20. Arduino IDE serial terminal after burning

- Serial baud rate must be equal to code baud rate.

Chapter 4: Application (MIT App Inventor)



Fig. 4.1. MIT App. Inventor

4.1 Comprehensive Overview of MIT App Inventor

MIT App Inventor is a **visual, block-based programming environment** that allows users to create **Android applications** without needing extensive coding knowledge. It was developed by the **Massachusetts Institute of Technology (MIT)** to make app development accessible to beginners, students, and educators.

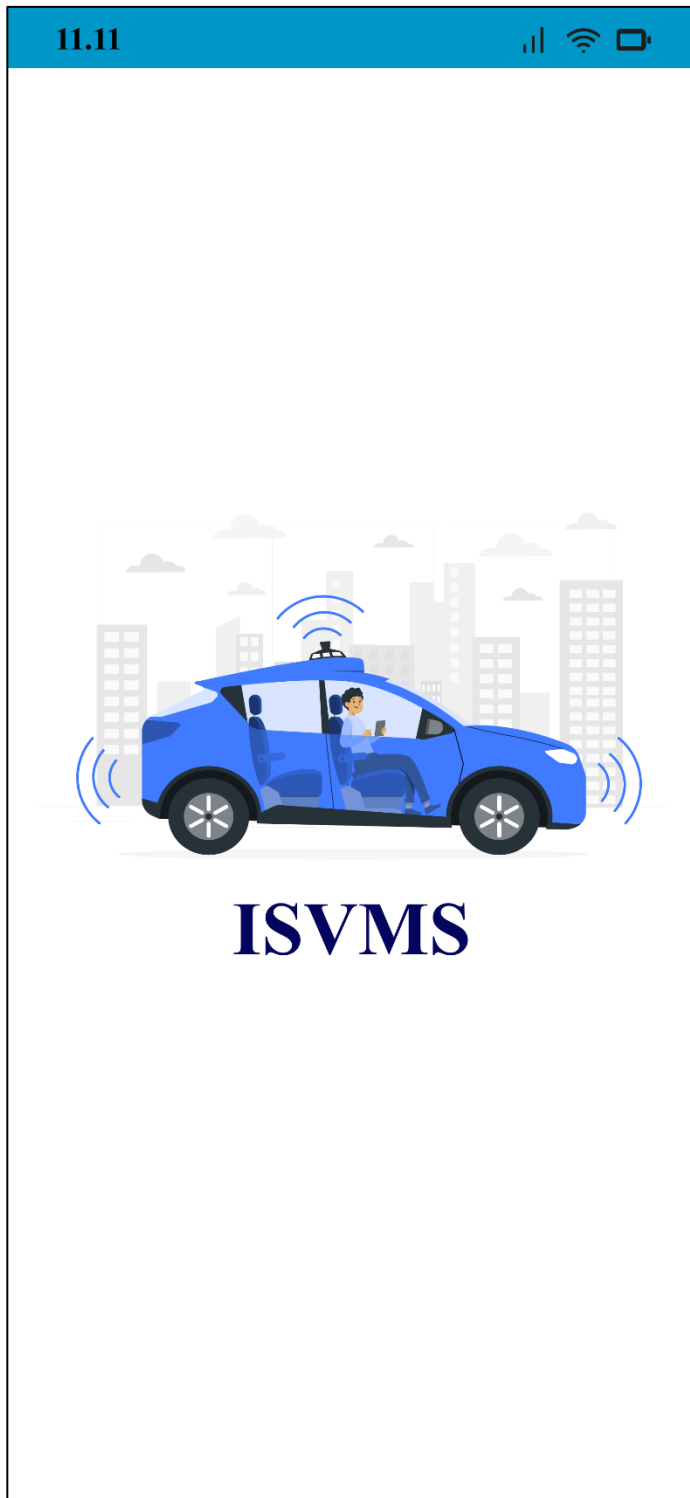
▪ Key Features of MIT App Inventor:

- **Drag-and-Drop Interface:** Uses a block-based coding system like Scratch, making programming easy.
- **Live Testing:** Apps can be tested in real-time on a connected device or emulator.
- **Cloud-Based Development:** Works in a web browser, so no software installation is needed.
- **Database & Cloud Support:** Can connect to databases like Firebase, TinyDB, and CloudDB (Redis-based)
- **Sensors & Hardware Integration:** Supports GPS, accelerometer, camera, and Bluetooth for interactive apps

4.2 Car Application

This app is a smart Bluetooth-based car control system that allows users to operate a vehicle remotely. It includes secure login, easy navigation controls, speed selection, and an automatic parking mode. With a simple and user-friendly design, the app provides smooth and efficient vehicle control for a better driving experience.

4.2.1 Splash screen page blocks code



```
when Screen1.Initialize
do set Clock1.TimerEnabled to true

when Clock1.Timer
do set Clock1.TimerEnabled to false
  open another screen screenName Login
```

4.2.2 Login page blocks code

11.11



ISVMS

Log In

Username

jason_ali

Password

[Forget The Password?](#)

Log In

Don't Have An Account?

[Sign Up](#)

Fig. 4.2. Login Page Application UI

```

when Login .BackPressed
do close application

initialize global User_Enter_Error to false

initialize global button_pressed to ""

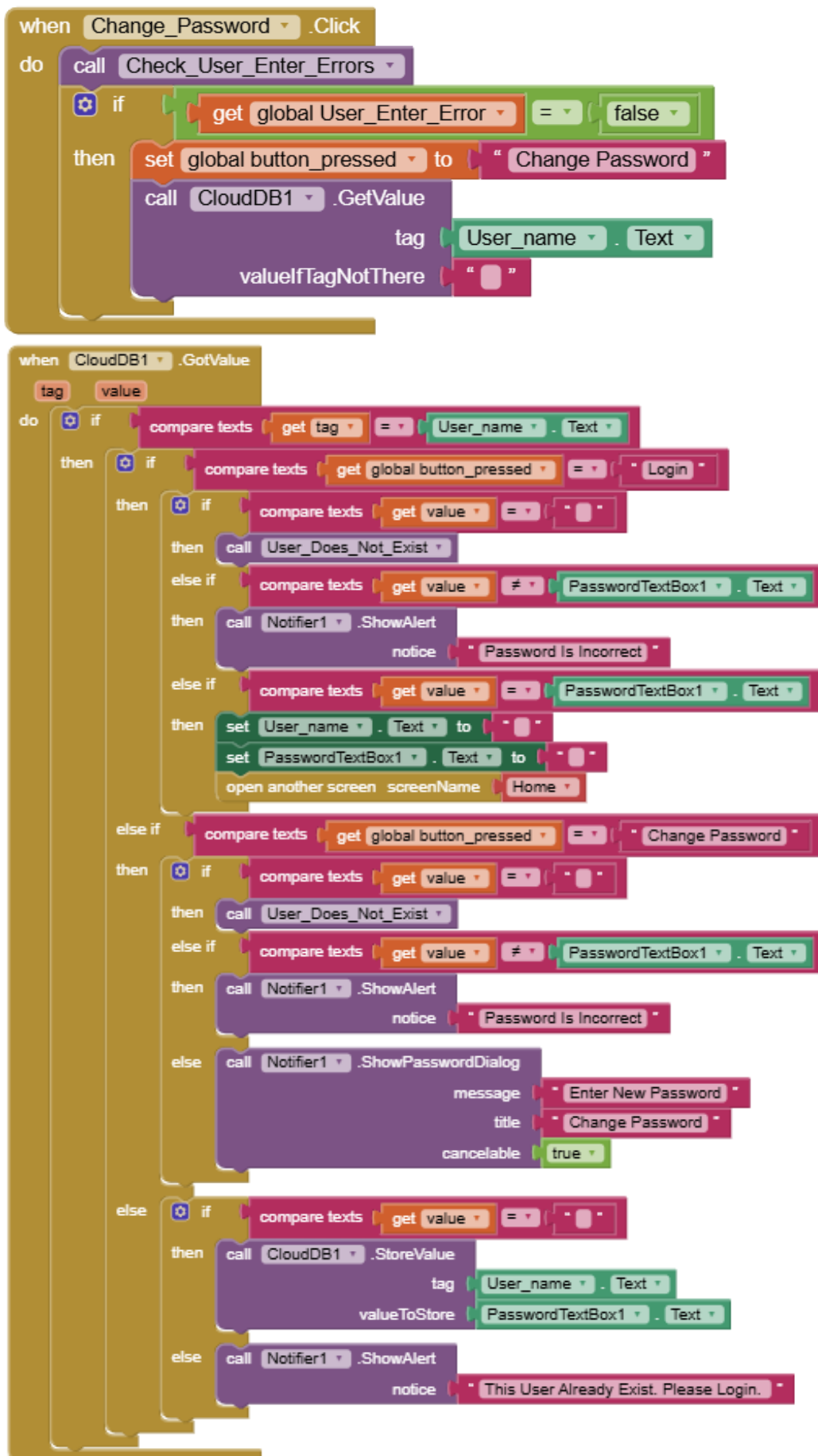
to Check_User_Enter_Errors
do
  set global button_pressed to ""
  set global User_Enter_Error to false
  if
    is empty User_name . Text or is empty PasswordTextBox1 . Text
  then
    call Notifier1 .ShowAlert
    notice "Please Enter Username And Password "
    set global User_Enter_Error to true

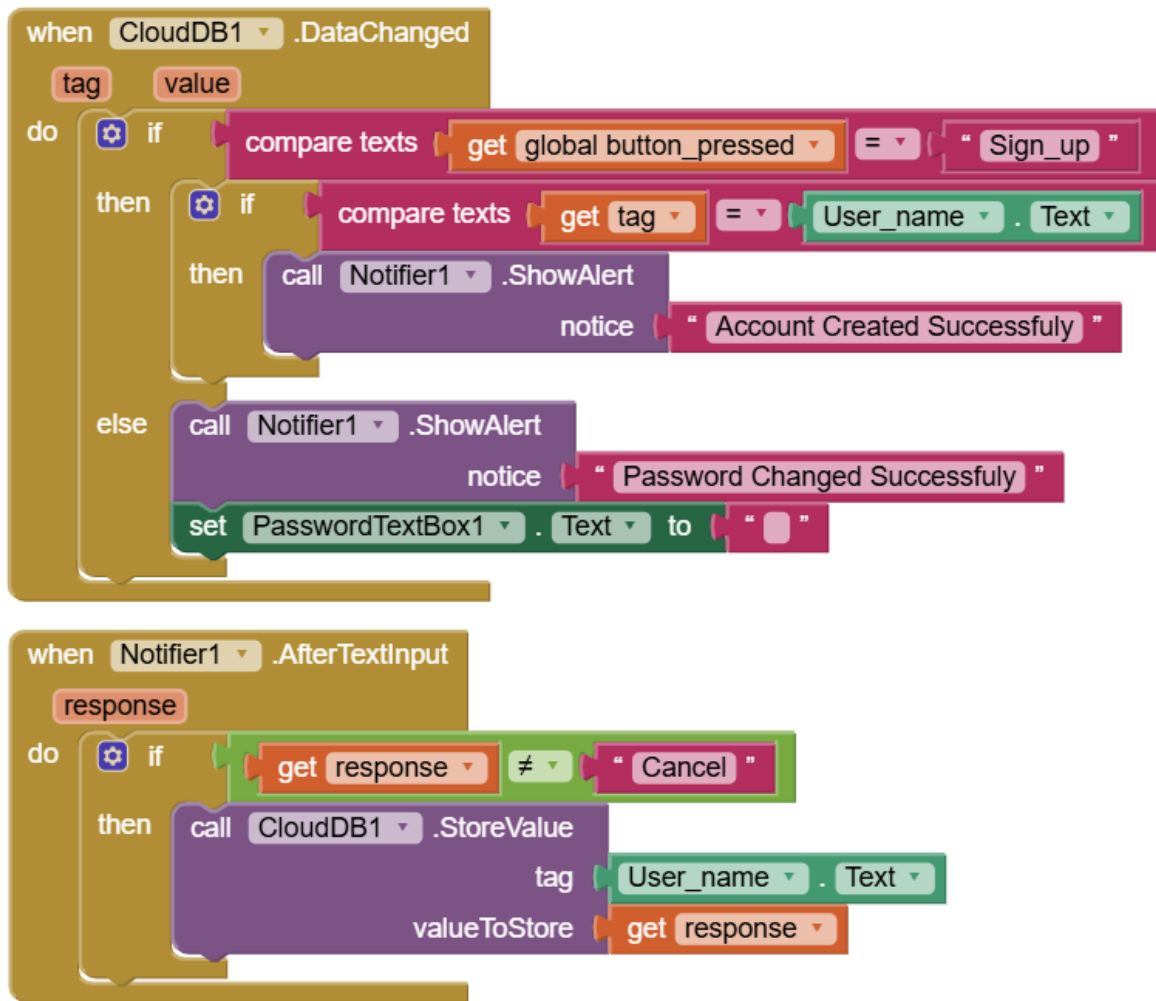
to User_Does_Not_Exist
do
  call Notifier1 .ShowAlert
  notice "This User Does Not Exist. Please Sign Up First. "
  set User_name . Text to ""
  set PasswordTextBox1 . Text to ""

when Login2 .Click
do
  call Check_User_Enter_Errors
  if
    get global User_Enter_Error = false
  then
    set global button_pressed to "Login "
    call CloudDB1 .GetValue
    tag User_name . Text
    valueIfTagNotThere ""

when Sign_up .Click
do
  call Check_User_Enter_Errors
  if
    get global User_Enter_Error = false
  then
    set global button_pressed to "Sign_up "
    call CloudDB1 .GetValue
    tag User_name . Text
    valueIfTagNotThere ""

```





4.2.3 Login page blocks explanation:

▪ Global Variables

1. User_Enter_Error:

- A boolean variable initialized to false.
- Used to track whether the user has made an error (e.g., leaving the username or password fields empty).

2. button_pressed:

- A string variable initialized to an empty string (" ").
- Used to store which button was pressed (e.g., "Login", "Change Password" or "Sign up").

▪ Procedure: Check_user_Enter_Errors

This procedure checks if the user has left the username or password fields empty. If either field is empty, it displays an error message and sets the User_Enter_Error variable to true, **Steps in the Procedure:**

1. Reset Variables:

- Set button_pressed to an empty string (" ").
- Set User_Enter_Error to false.

2. Check for Empty Fields:

- If either the User_name Text or PasswordTextBox Text is empty:
 - Display an alert using the Notifier Show Alert block with the message: "Please Enter Username and Password."
 - Set User_Enter_Error to true.

▪ Procedure: Check_User_Does_Not_Exist

executed when a user tries to log in or change their password, but their username is not found in the database, **Steps in the Procedure:**

1. **Show an Alert Message:** A notification alert appears, informing the user that their account does not exist, and they need to sign up first
2. **Clear the Username Field:** The username input field is cleared to prompt the user to enter a valid username
3. **Clear the Password Field:** The password input field is also cleared to prevent unnecessary attempts with incorrect credentials.

▪ **Event: when Login Clicked**

This event is triggered when the user clicks the "Login" button, **Steps in the Event:**

1. Call Check_user_Enter_Errors:

- Check if the username and password fields are filled

2. Check for Errors:

- If User_Enter_Error is false (no errors):
 - Set button_pressed to 'Login'.
 - Call CloudDB1 GetValue to retrieve data associated with the User_name Text from the cloud database.
 - If the tag (username) is not found, it returns an empty string (“ ”).

▪ **Event: when Sign Up Clicked**

This event is triggered when the user clicks the "Sign Up" button, **Steps in the Event:**

1. Call Check_user_Enter_Errors:

- Check if the username and password fields are filled

2. Check for Errors:

- If User_Enter_Error is false (no errors):
 - Set button_pressed to 'Sign Up'.
 - Call CloudDB1 GetValue to retrieve data associated with the User_name Text from the cloud database.
 - If the tag (username) is not found, it returns an empty string (“ ”).

▪ **Event: when Change_Password Clicked**

This event is triggered when the user clicks the "Change Password" button, **Steps in the Event:**

1. Call Check_user_Enter_Errors:

- Check if the username and password fields are filled.

2. Check for Errors:

- If User_Enter_Error is false (no errors):
 - Set button_pressed to 'Change Password'.

- Call CloudDB1 GetValue to retrieve data associated with the User_name Text from the cloud database.
- If the tag (username) is not found, call Check_user_Enter_Errors again (likely to handle the error).

▪ **Event: CloudDB1.DataChanged**

This event is triggered whenever a value in CloudDB1 is changed.

- **tag:** The key (e.g., username) that was updated.
- **value:** The new value stored in the database.

• **Event Explanation:**

➤ **Checking if the Action Was a Signup:**

This checks whether the global button_pressed variable is set to "Sign_up", meaning the user was creating an account.

➤ **Confirming the Account Creation:**

This ensures that the updated data in the database matches the entered username. If true, it confirms that the account has been successfully created

➤ **Handling Password Change (Else Condition):**

- If button_pressed was not "Sign_up", it assumes that the user was changing their password.
- A successful message "Password Changed Successfully" is shown.
- The password field (PasswordTextBox1.Text) is cleared to enhance security.

▪ **Event: CloudDB1.DataChanged**

This block-based program is built using MIT App Inventor or Kodular, handling user authentication (Login, Password Change, and Registration) using CloudDB, **Steps in the Event:**

1. **Trigger CloudDB1.GotValue:**

- This event is triggered when the app retrieves data from CloudDB1.
- It takes two inputs:
 - **tag** → The key used to store/retrieve the value (Username).
 - **value** → The retrieved value (Password).

2. Checking if the Retrieved tag Matches User_name. Text:

- The program first compares if the retrieved tag (username) matches the input from the user (User_name.Text).
- If the usernames match, it proceeds further

3. Checking User Action (global button_pressed):

- The program checks which action the user wants to perform (global button_pressed) **Login** or **Change Password** or **Register (New User Creation)**.
- **Login Process (global button_pressed = "Login")**
 - If the username does not exist (value = " "), it calls User_Does_Not_Exist.
 - If the username exists but the password is incorrect (value ≠ PasswordTextBox1.Text), it shows a notification "Password is Incorrect".
 - If the username exists and the password matches, the user is authenticated:
 - Clears User_name and PasswordTextBox1 fields.
 - Opens the Home Screen.
- **Change Password (global button_pressed = "Change Password")**
 - The user does not exist (value = ""), it is called User_Does_Not_Exist.
 - If the password is incorrect, it displays "Password is Incorrect".
 - If the password is correct, it prompts the user with a password input dialog to enter a new password.
- **Registering a New User (If the user is not found in the database)**
 - If the user does not exist (value = ""), it stores the new username and password in CloudDB1.
 - If the user already exists, it displays "This User Already Exists. Please Login."

Scenario	Condition	Action Taken
User tries to log in but doesn't exist	value = ""	Calls User_Does_Not_Exist
User enters incorrect password during login	value $\neq$ PasswordTextBox1.Text	Displays "Password is Incorrect"
User enters correct password during login	value = PasswordTextBox1.Text	Logs in, clears fields, and opens "Home" screen
User tries to change password but doesn't exist	value = ""	Calls User_Does_Not_Exist
User enters incorrect password during password change	value $\neq$ PasswordTextBox1.Text	Displays "Password is Incorrect"
User enters correct password and wants to change it	value = PasswordTextBox1.Text	Prompts a password change dialog
New users try to register and do not exist	value = ""	Stores username and password in CloudDB1
New users try to register but already exists	value $\neq$ ""	Displays "This User Already Exists. Please Login."

Table 4.1. Summary of Possible Outcomes

▪ **Event: Notifier1.AfterTextInput**

- Checks if the response is not equal to the string "Cancel".
- If it's not "Cancel," it stores the response in a cloud database using the CloudDB1 component.
- The data is stored with a tag set to the text property of the User_name component.

4.2.4 Home page blocks code

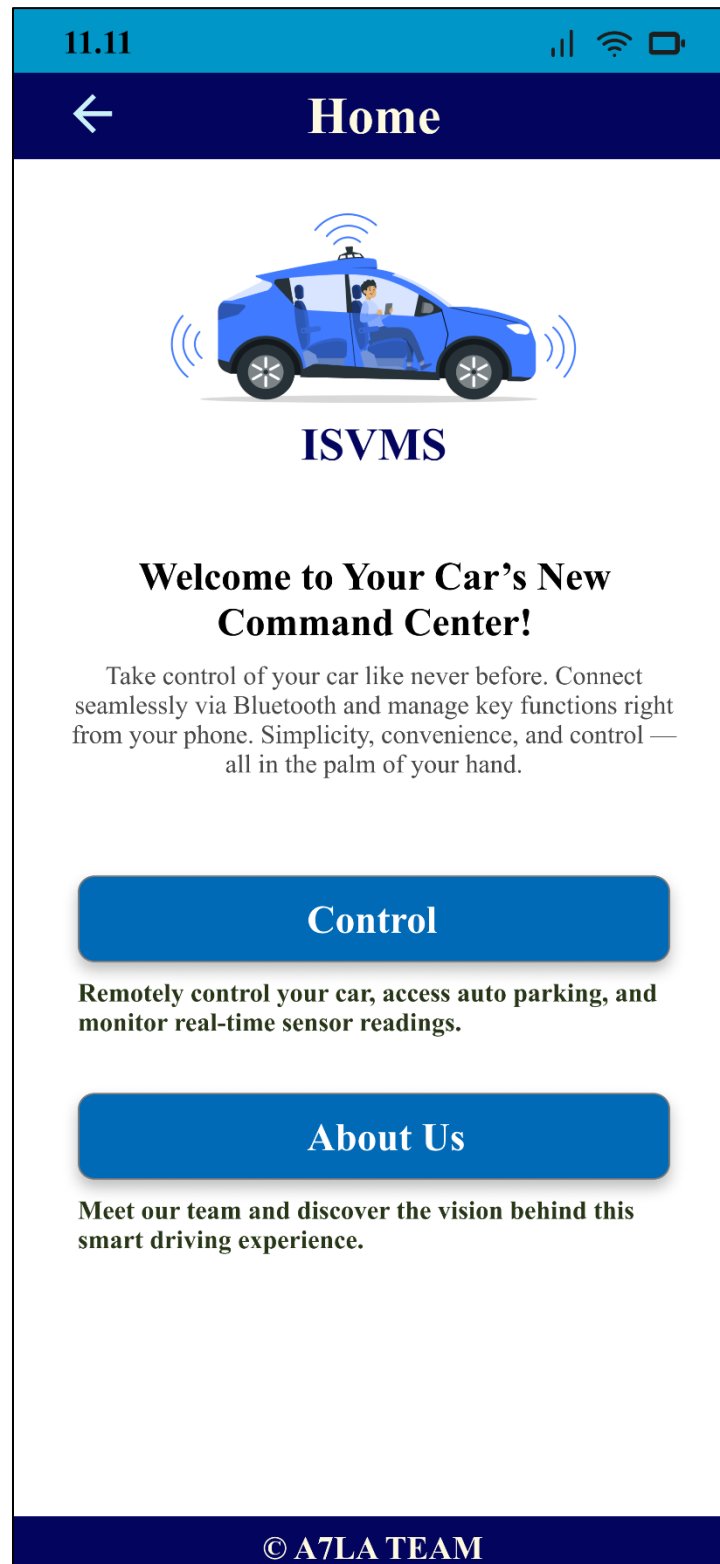
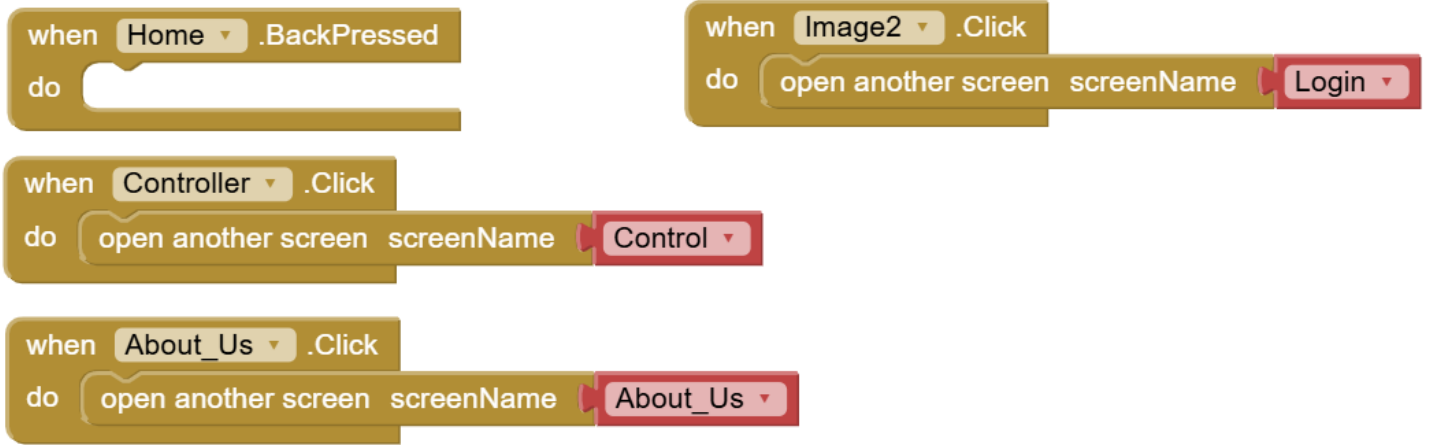


Fig. 4.3. Home Page Application UI



4.2.5 Home page blocks explanation:

▪ Block 1: Controller Button Click Event

- **Trigger:** When the button named Controller is clicked.
- **Action:** The app opens another screen named Control.

Explanation:

- o Controller.Click: This event is triggered when the Controller button is clicked.
- o open another screen: This function is used to switch to a different screen within the app.
- o screenName Control: This specifies that the app should navigate to the screen named Control.

▪ Block 2: About_Us Button Click Event

- **Trigger:** When the button named About_Us is clicked.
- **Action:** The app opens another screen named About_Us.

Explanation:

- o Controller.Click: This event is triggered when the About_Us button is clicked.
- o Open another screen: This function is used to switch to another screen.
- o screenName Control: This specifies that the app should navigate to the screen named Control.

4.2.6 Controller page blocks code

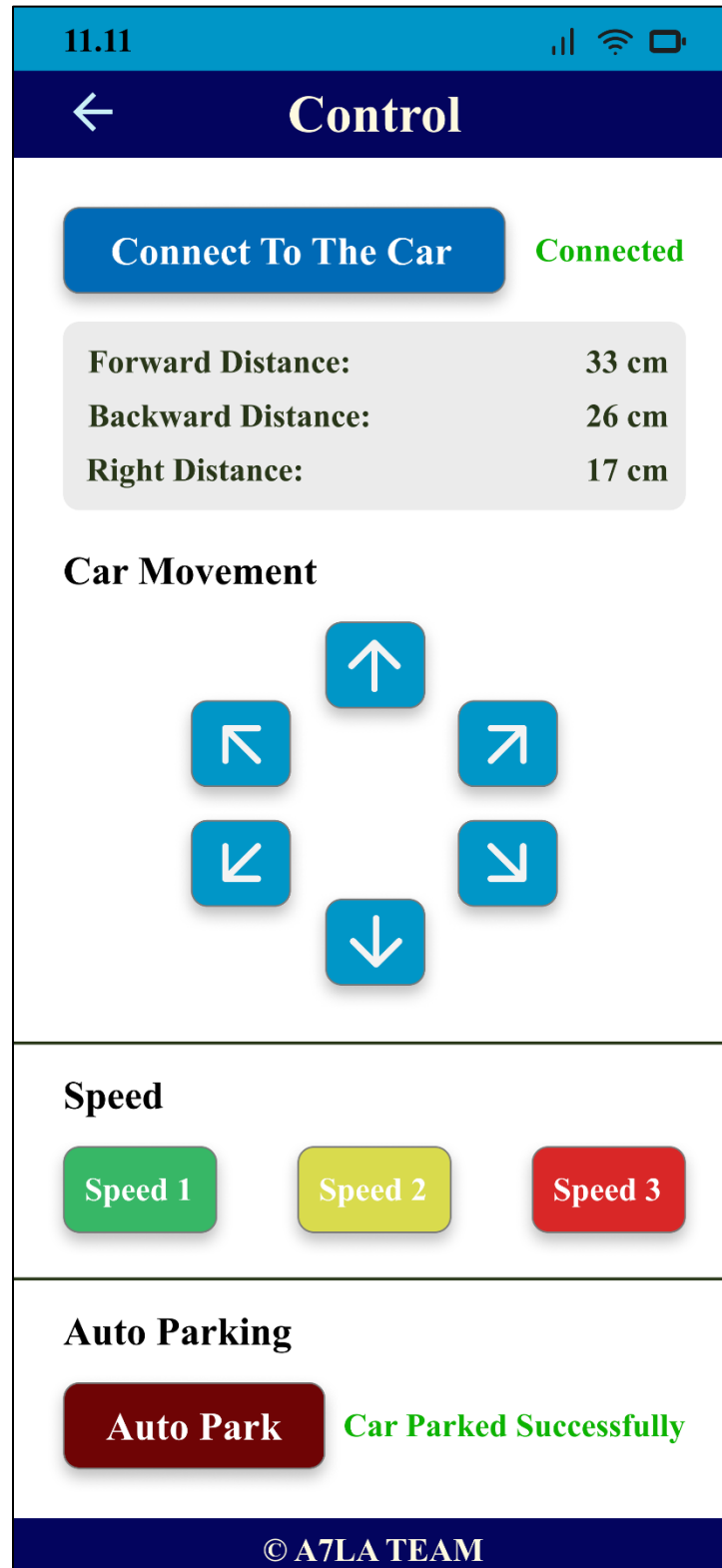


Fig. 4.4. Controller Page Application UI

when **Control** .BackPressed
do

when **Image2** .Click
do open another screen screenName **Home**

when **Connect** .BeforePicking
do set **Connect** . Elements to **BluetoothClient1** . AddressesAndNames

when **Connect** .AfterPicking
do if call **BluetoothClient1** .Connect
address **Connect** . Selection
then set **Sataus** . Text to **"Connected"**
set **Sataus** . TextColor to 
else set **Sataus** . Text to **"Not Connected"**
set **Sataus** . TextColor to 

when **Forward** .TouchDown
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"F"**

when **Forward** .TouchUp
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"S"**

when **Backward** .TouchDown
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"B"**

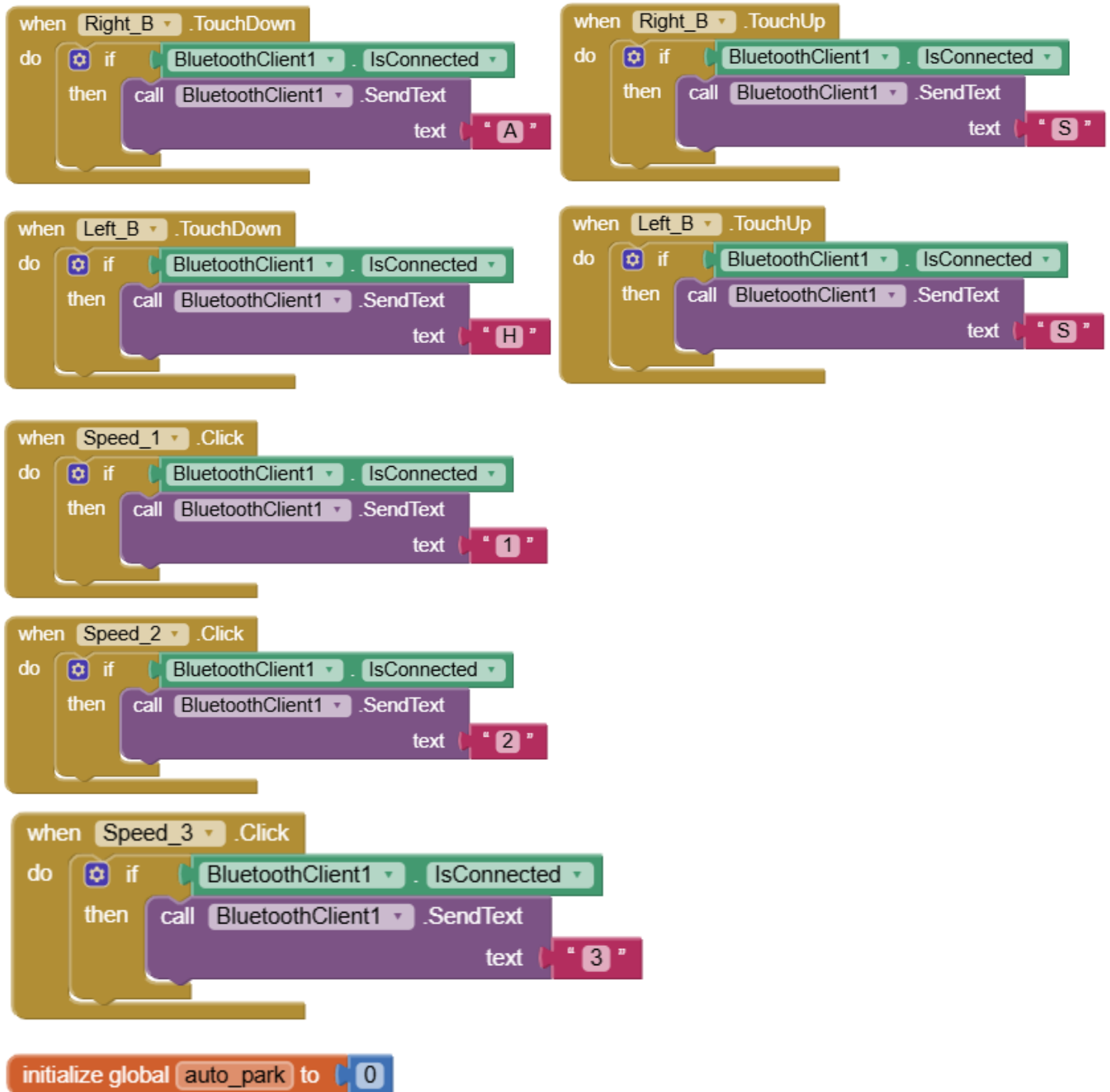
when **Backward** .TouchUp
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"S"**

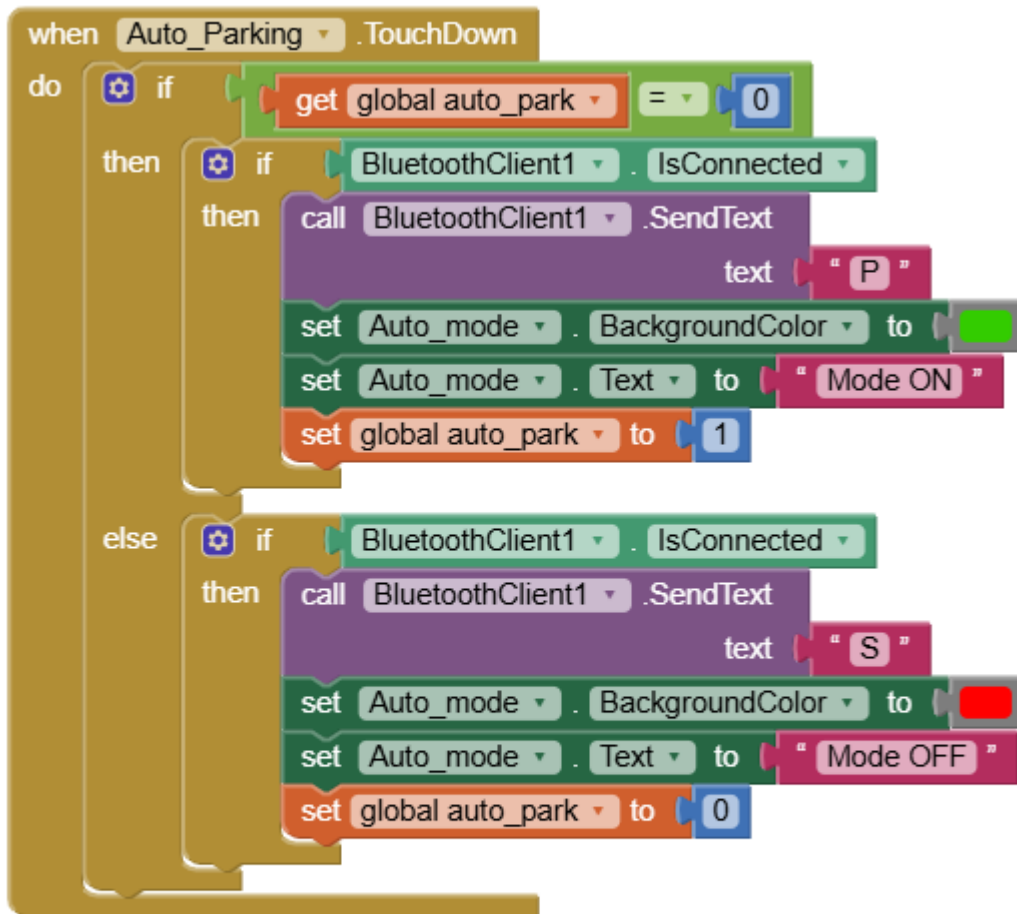
when **Right_F** .TouchDown
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"R"**

when **Right_F** .TouchUp
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"S"**

when **Left_F** .TouchDown
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"L"**

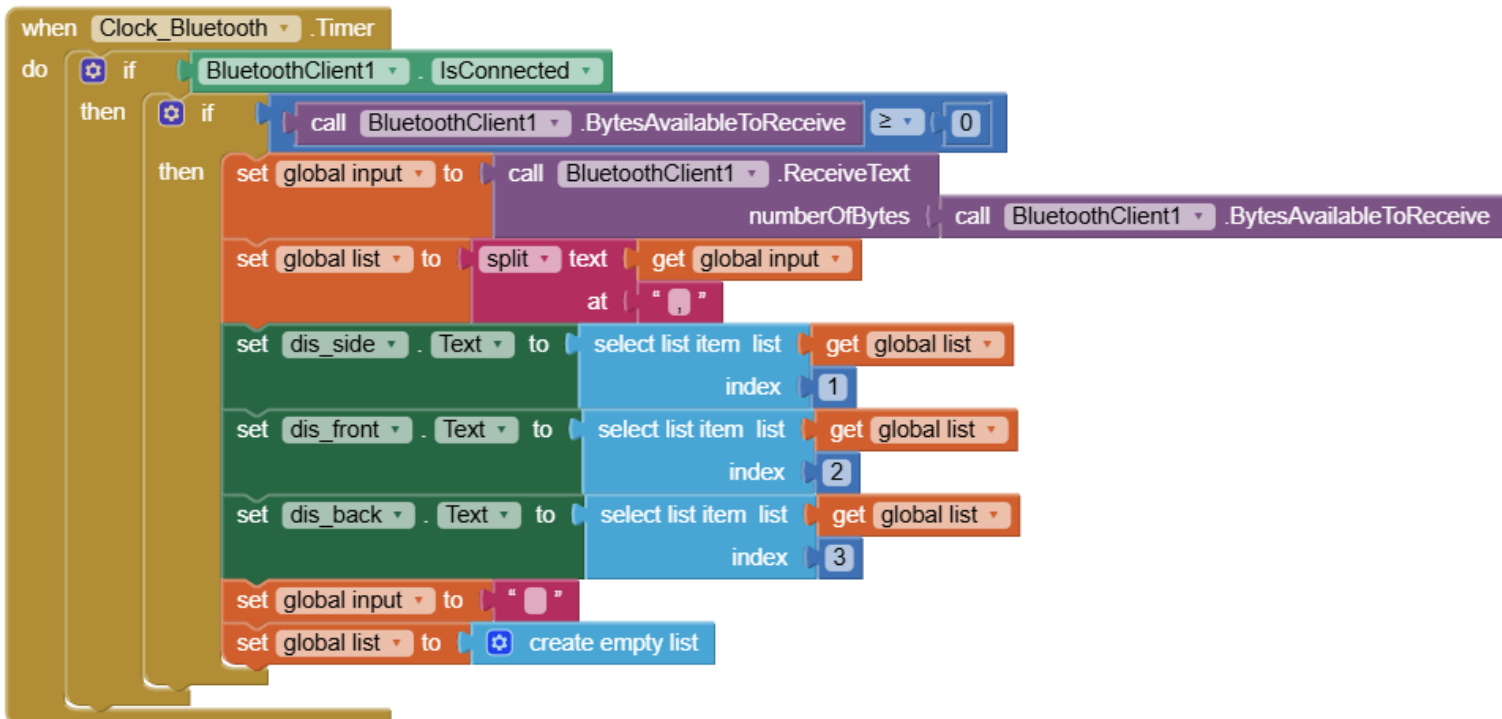
when **Left_F** .TouchUp
do if **BluetoothClient1** . IsConnected
then call **BluetoothClient1** .SendText
text **"S"**





initialize global list to create empty list

initialize global input to " "



4.2.7 Controller page blocks explanation:

a) Directions Buttons:

▪ General Functionality

- The application sends specific **command characters** (F, B, R, L, S, A, H, M, 1, 2, 3, P) over Bluetooth when buttons are pressed (TouchDown) and released (TouchUp).
- The car interprets these characters to move in the respective direction or stop.

▪ Detailed Breakdown of Blocks

Each button (Forward, Backward, Right_F, Left_F, Right_B, Left_B) has two associated event handlers:

- **TouchDown Event** (when the button is pressed)
- **TouchUp Event** (when the button is released)

▪ Forward Button

- **When Pressed (TouchDown):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "F" (indicating "move forward").
- **When Released (TouchUp):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "S" (indicating "stop").

▪ Backward Button

- **When Pressed (TouchDown):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "B" (indicating "move backward").
- **When Released (TouchUp):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "S" (indicating "stop").

- **Right Forward Button (Right_F)**
 - **When Pressed (TouchDown):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "R" (indicating "turn right").
 - **When Released (TouchUp):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "S" (indicating "stop").

- **Left Forward Button (Left_F)**
 - **When Pressed (TouchDown):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "L" (indicating "turn left").
 - **When Released (TouchUp):**
 - Checks if **BluetoothClient1** is connected.
 - Sends the text "S" (indicating "stop").

- **Right Backward Forward Button (Right_B)**
 - **When Pressed (TouchDown):**
 - If **BluetoothClient1** is connected, the app sends "A" (indicating a **right backward turn**).
 - **When Released (TouchUp):**
 - If **BluetoothClient1** is connected, the app sends "S" (indicating **stop**).

- **Left Backward Button (Left_B)**
 - **When Pressed (TouchDown):**
 - If **BluetoothClient1** is connected, the app sends "H" (indicating a **left backward turn**).
 - **When Released (TouchUp):**
 - If **BluetoothClient1** is connected, the app sends "S" (indicating **stop**).

b) Speed Control Buttons:

- **Select_Speed.Click**
 - Checks if BluetoothClient1 is connected.
 - Send the letter "M" via Bluetooth when clicked.

- **Speed_1. Click**
 - Checks if BluetoothClient1 is connected.
 - Send the number "1" via Bluetooth when clicked.

- **Speed_2. Click**
 - Checks if BluetoothClient1 is connected.
 - Sends the number "2" via Bluetooth when clicked.

- **Speed_3. Click**
 - Checks if BluetoothClient1 is connected.
 - Send the number "3" via Bluetooth when clicked.

c) Bluetooth Connection Handling:

- **Connect.BeforePicking**
 - Populate the Connect list with available Bluetooth addresses and names before the user picks a device.

- **Connect. AfterPicking**
 - Try to connect to the selected Bluetooth device.
 - If the connection is successful, it updates the Status text to "Connected" and sets the text color to green.
 - If the connection fails, it updates the Status text to "Not Connected" and sets the text color to red.

d) Auto Parking Feature:

▪ Global Variable Initialization

- initialize global auto_park to 0
- This sets the auto_park variable to 0, meaning that auto parking mode starts in an "OFF" state.

▪ Event: Auto_Parking.TouchDown

- This block triggers when the Auto_Parking button is pressed.

▪ Checking the auto_park State

- If auto_park = 0, the system will turn on Auto Parking mode.
- If auto_park = 1, the system will turn off Auto Parking mode.

▪ How the Toggle Works:

(A) Turning Auto Parking Mode ON

- **If auto_park = 0:**
 1. Check if BluetoothClient1 is connected.
 2. Send the text "P" via Bluetooth to activate auto parking.
 3. Change Auto_mode label background color to green.
 4. Change Auto_mode text to "Mode ON".
 5. Set auto_park to 1, indicating that auto parking mode is now ON.

(B) Turning Auto Parking Mode OFF

- **If auto_park = 1:**
 1. Check if BluetoothClient1 is connected.
 2. Send the text "S" via Bluetooth to stop auto parking.
 3. Change Auto_mode label background color to red.
 4. Change Auto_mode text to "Mode OFF".
 5. Set auto_park to 0, indicating that auto parking mode is now OFF.

Button	TouchDown (Pressing)	TouchUp (Releasing)
Forward	"F" (Move Forward)	"S" (Stop)
Backward	"B" (Move Backward)	"S" (Stop)
Right_F	"R" (Turn Right Forward)	"S" (Stop)
Left_F	"L" (Turn Left Forward)	"S" (Stop)
Right_B	"A" (Turn Right Backward)	"S" (Stop)
Left_B	"H" (Turn Left Backward)	"S" (Stop)
Select_Speed	"M" (Select the car speed)	-----
Speed_1	"1" (Select the Minimum speed)	-----
Speed_2	"2" (Select the Moderate speed)	-----
Speed_3	"3" (Select the Maximum speed)	-----
Auto_Parking	"P" (Auto Parking Mode)	-----

Table 4.2. Summary of Controller Page Buttons

4.2.8 About_Us page Blocks code

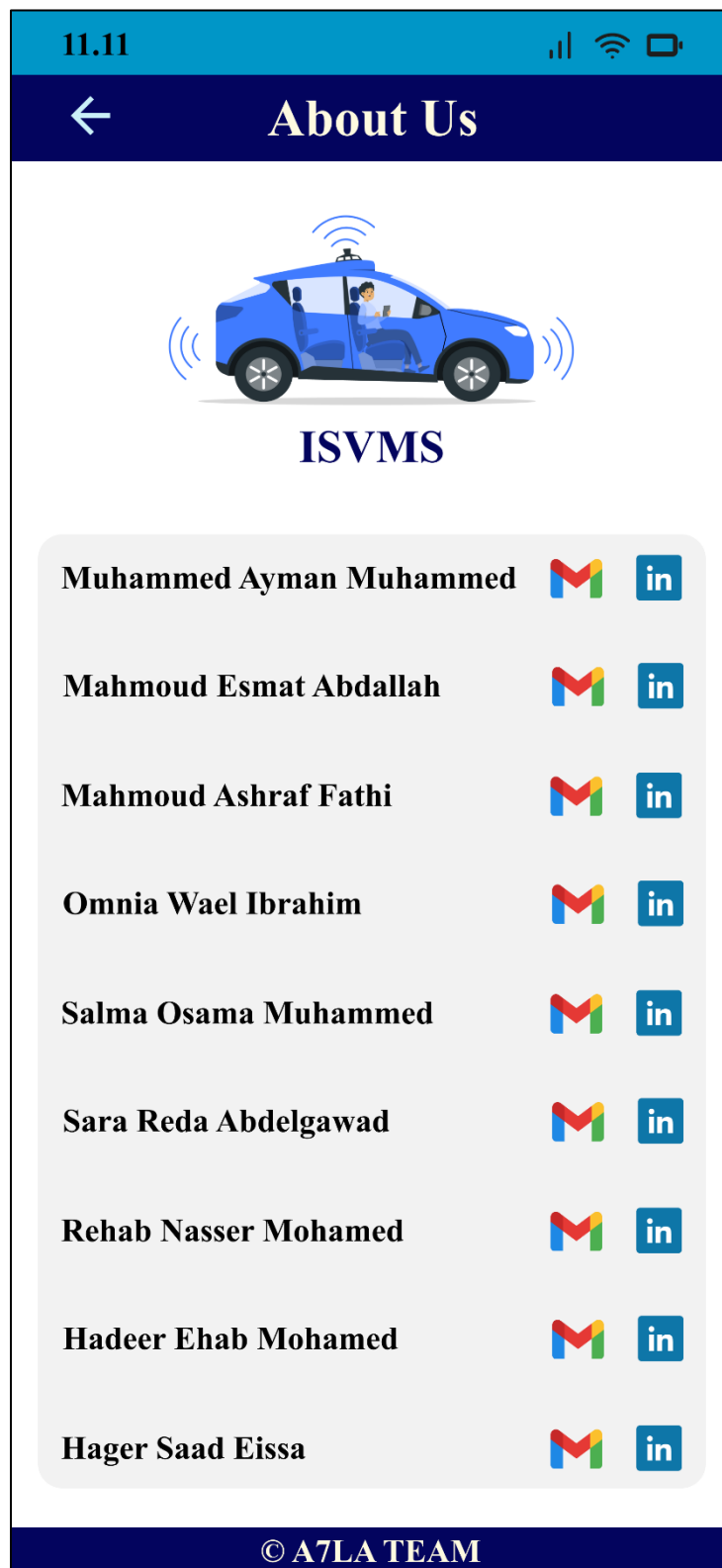


Fig. 4.5. About Us Page Application UI

when About_Us ▾ .BackPressed
do

when Image2 ▾ .Click

do open another screen screenName Home ▾

when Muhammed_Email ▾ .Click

do set ActivityStarter1 ▾ .DataUri ▾ to "mailto:eng.muhammed.ayman@gmail.com"
set ActivityStarter1 ▾ .Action ▾ to "android.intent.action.VIEW"
call ActivityStarter1 ▾ .StartActivity

when Mahmoud_A_Email ▾ .Click

do set ActivityStarter1 ▾ .DataUri ▾ to "mailto:mashrag4@gmail.com"
set ActivityStarter1 ▾ .Action ▾ to "android.intent.action.VIEW"
call ActivityStarter1 ▾ .StartActivity

when Mahmoud_E_Email ▾ .Click

do set ActivityStarter1 ▾ .DataUri ▾ to "mailto:Mahmoudesmat20029@gmail.com"
set ActivityStarter1 ▾ .Action ▾ to "android.intent.action.VIEW"
call ActivityStarter1 ▾ .StartActivity

when Omnia_Email ▾ .Click

do set ActivityStarter1 ▾ .DataUri ▾ to "mailto:omniawail9@gmail.com"
set ActivityStarter1 ▾ .Action ▾ to "android.intent.action.VIEW"
call ActivityStarter1 ▾ .StartActivity

when Salma_Email ▾ .Click

do set ActivityStarter1 ▾ .DataUri ▾ to "mailto:salmaosamamohamed74@gmail.com"
set ActivityStarter1 ▾ .Action ▾ to "android.intent.action.VIEW"
call ActivityStarter1 ▾ .StartActivity

when Sara_Email ▾ .Click

do set ActivityStarter1 ▾ .DataUri ▾ to "mailto:sarahamer562@gmail.com"
set ActivityStarter1 ▾ .Action ▾ to "android.intent.action.VIEW"
call ActivityStarter1 ▾ .StartActivity

when Rehab_Email .Click

do set ActivityStarter1 . DataUri to "mailto:zayedrehab1@gmail.com"
set ActivityStarter1 . Action to "android.intent.action.VIEW"
call ActivityStarter1 .StartActivity

when Hadeer_Email .Click

do set ActivityStarter1 . DataUri to "mailto:hadeerehap754@gmail.com"
set ActivityStarter1 . Action to "android.intent.action.VIEW"
call ActivityStarter1 .StartActivity

when Hager_Email .Click

do set ActivityStarter1 . DataUri to "mailto:hagersaad552002@gmail.com"
set ActivityStarter1 . Action to "android.intent.action.VIEW"
call ActivityStarter1 .StartActivity

when Muhammed_LinkedIn .Click

do set ActivityStarter1 . DataUri to "https://www.linkedin.com/in/muhammed-ayman-63a02..."
set ActivityStarter1 . Action to "android.intent.action.VIEW"
call ActivityStarter1 .StartActivity

when Mahmoud_A_LinkedIn .Click

do set ActivityStarter1 . DataUri to "https://www.linkedin.com/in/mahmoud-ashraf-13104..."
set ActivityStarter1 . Action to "android.intent.action.VIEW"
call ActivityStarter1 .StartActivity

when Mahmoud_E_LinkedIn .Click

do set ActivityStarter1 . DataUri to "https://www.linkedin.com/in/mahmoud-esmat-120a0b..."
set ActivityStarter1 . Action to "android.intent.action.VIEW"
call ActivityStarter1 .StartActivity

when Omnia_LinkedIn .Click

do set ActivityStarter1 . DataUri to "https://eg.linkedin.com/in/omnia-wael-2a84b7230"
set ActivityStarter1 . Action to "android.intent.action.VIEW"
call ActivityStarter1 .StartActivity

when Salma_LinkedIn .Click

do
 set ActivityStarter1 . DataUri to " https://www.linkedin.com/in/salma-abdelhy?utm_so... "
 set ActivityStarter1 . Action to " android.intent.action.VIEW "
 call ActivityStarter1 .StartActivity

when Sara_LinkedIn .Click

do
 set ActivityStarter1 . DataUri to " https://www.linkedin.com/in/sara-amer-6645992b5?... "
 set ActivityStarter1 . Action to " android.intent.action.VIEW "
 call ActivityStarter1 .StartActivity

when Rehab_LinkedIn .Click

do
 set ActivityStarter1 . DataUri to " https://www.linkedin.com/in/rehab-zayed-4b9a8a24... "
 set ActivityStarter1 . Action to " android.intent.action.VIEW "
 call ActivityStarter1 .StartActivity

when Hadeer_LinkedIn .Click

do
 set ActivityStarter1 . DataUri to " https://www.linkedin.com/in/hadeer-ehap-bb0b0423... "
 set ActivityStarter1 . Action to " android.intent.action.VIEW "
 call ActivityStarter1 .StartActivity

when Hager_LinkedIn .Click

do
 set ActivityStarter1 . DataUri to " https://www.linkedin.com/in/hager-saad-93a78a324... "
 set ActivityStarter1 . Action to " android.intent.action.VIEW "
 call ActivityStarter1 .StartActivity

4.2.9 Blocks Explanation:

- **Each name in the list has:**

1. A Gmail icon that allows users to send an email.
2. A LinkedIn icon that redirects users to the respective LinkedIn profile

- **Email Buttons**

Each email button, when clicked:

1. Sets the ActivityStarter1's DataUri to a mailto: link with the specific email.
2. Sets the Action to android.intent.action.VIEW (this tells the system to open the mail app).
3. Calls StartActivity to launch the default email application.

- **LinkedIn Buttons**

Each LinkedIn button, when clicked:

1. Sets ActivityStarter1's DataUri to the specific LinkedIn profile URL.
2. Sets the Action to android.intent.action.VIEW (this tells the system to open the web browser or LinkedIn app).
3. Call StartActivity to open the LinkedIn profile.

QR code of application:



<https://ai2.appinventor.mit.edu/b/5gzoc>

Chapter 5: Drivers of Atmega32

5.1 Atmega32

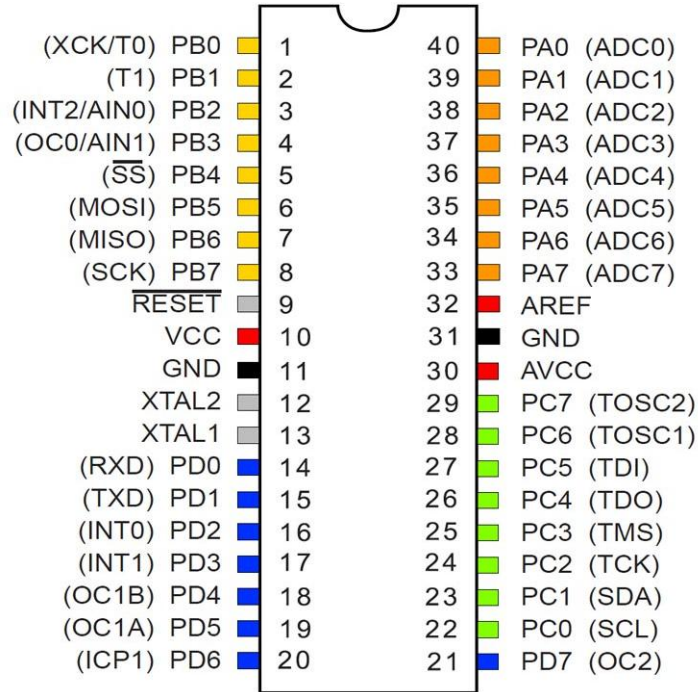


Fig. 5.1. Atmega32

5.1.1 ATmega32 Microcontroller Overview

The ATmega32 is an 8-bit microcontroller from Microchip Technology (formerly Atmel), based on the AVR RISC architecture. It is widely used in embedded systems due to its versatility, low power consumption, and ease of programming.

■ CPU & Memory:

- 8-bit RISC architecture with 32 general-purpose registers.
- 32 KB Flash memory for program storage.
- 2 KB SRAM for data storage.
- 1 KB EEPROM for non-volatile storage.

■ Clock & Power:

- Supports internal (1 MHz) and external (up to 16 MHz) clock sources.
- Low power consumption with multiple sleep modes.

▪ **Peripherals & Interfaces:**

- **Timers:** Three timers (Timer0 & Timer2 - 8-bit, Timer1 - 16-bit) for time-related operations and PWM generation.
- **ADC:** 10-bit, 8-channel Analog-to-Digital Converter (ADC) for sensor interfacing.
- **Communication:**
 - UART for serial communication.
 - I2C (TWI) for multi-device communication.
 - SPI for high-speed data transfer.
- **I/O Ports:** 32 configurable pins across four ports (PORTA, PORTB, PORTC, PORTD).
- **External Interrupts:** Supports INT0, INT1, and INT2 for handling real-time events.

▪ **Other Features:**

- **PWM (Pulse Width Modulation):** Used for motor control, LED dimming, and more.
- **Watchdog Timer (WDT):** Ensures system reliability by resetting the microcontroller in case of failure.
- **Brown-out Detection (BOD):** Prevents the microcontroller from operating under low voltage conditions.

5.1.2 Programming Atmega32

Feature	Eclipse IDE	Atmel studio	Code::Blocks	Platform (Vs.Code)
Installation	Requires manual setup of AVR Plugin & Toolchain	One-click installation with full tool support	Requires manual setup like Eclipse	Easy installation with Platform IO
Integration with AVR-GCC	Supported via AVR Plugin	Natively supported	Supported via external plugins	Fully supported
Code Completion & IntelliSense	Good, but depends on plugin	Excellent with built-in support	Decent but requires extra configuration	Excellent with IntelliSense
Debugger Support	Limited, requires external GDB setup	Best debugging support for AVR	Limited, requires additional tools	Good, supports GDB
User Interface	Complex but customizable	Modern and feature-rich	Simple and lightweight	Modern (VS Code-based)
Supported Operating Systems	Windows, Linux, macOS	Windows only	Windows, Linux, macOS	Windows, Linux, macOS
Toolchain Management	Manual	Built-in	Manual	Automated with PlatformIO
Extensibility	High (via plugins)	Limited (mostly for AVR)	High (but fewer plugins than Eclipse)	Very high (supports multiple platforms)

Table 5.1. Compare between Atmega32 development frameworks

Although Atmel Studio offers a simple and user-friendly experience compared to other IDEs, it is limited to AVR only. We wanted an IDE that supports multiple microcontroller architecture and is widely used in industry. Therefore, we chose Eclipse IDE.

- **To install Eclipse IDE:**  <https://linksshortcut.com/qjReS>

Follow the simple routine: Install, Extract, then Open.

- **We need to take the following steps to configure eclipse IDE to support AVR Microcontroller:**

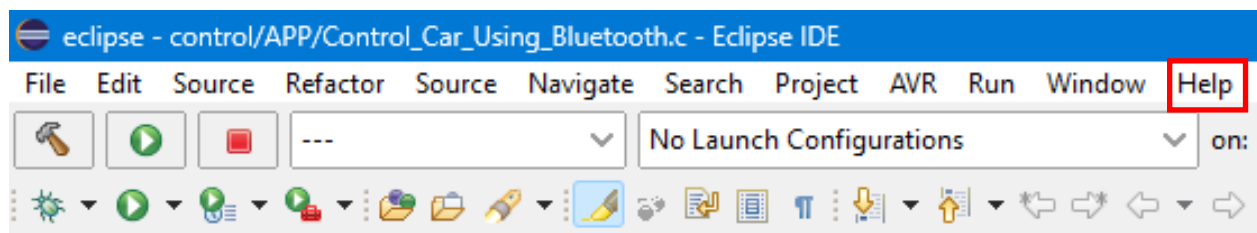
1. Download and Install AVR-GCC compiler: <https://linksshortcut.com/buBaK>

It is normal installation steps just press next, next ... finish

2. Download AVR Eclipse Plugin from the below link: <https://linksshortcut.com/nySbm>

Installation steps:

- Don't extract .zip file
- Open eclipse
- Choose help --> install new software
- Choose Add --> Archive --> browse to the .zip file (check the attached pic)
- Continue by press next, next, next then restart eclipse.



- **To burn the code, we must need burner:**  <https://linksshortcut.com/KmWGT>

Need file with extend .Hex, it can create by eclipse by:

- Right click on the project inside the project.
- Choose properties.
- Choose AVR → setting and choose extract .Hex file

- **To install Proteus for simulation:**  <https://linksshortcut.com/oYTth>

5.2 Library

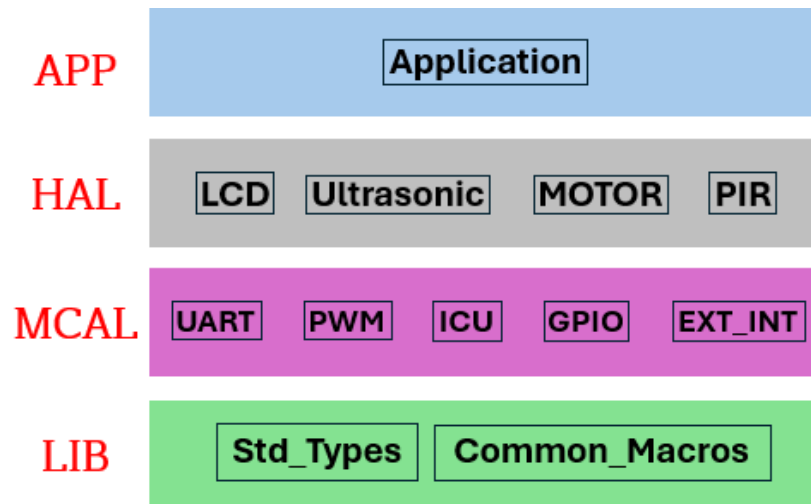


Fig. 5.2. Layers of Application

5.2.1 LIB drivers

- **Std_Types:**

This code ensures that the file is included only once during the compilation by using the included guard. If the file has already been included, the code inside will be ignored.

```
#ifndef STANDARD_TYPES_H_
#define STANDARD_TYPES_H_
.
.
.
#endif /* STANDARD_TYPES_H_ */
```

A new data type boolean is defined as an unsigned character, which can store values like TRUE or FALSE.

The FALSE and TRUE values are defined if they haven't been defined earlier. FALSE is set to 0, and TRUE is set to 1.

```
typedef unsigned char boolean;          /* Boolean Data Type */

#ifndef FALSE
#define FALSE          (0u)          /* Boolean false value */
#endif
#ifndef TRUE
#define TRUE           (1u)          /* Boolean true value */
#endif
```


An enum type `STD_Value` is defined with two possible values: `LOGIC_LOW` (0) and `LOGIC_HIGH` (1), representing logical low and high values.

```
typedef enum
{
    LOGIC_LOW,          /* 0 */
    LOGIC_HIGH          /* 1 */
}STD_Value;
```

An enum `Timer_ClockType` defines different clock types for the timer: no clock, CPU clock, and various prescalers like `F_CPU_8`, `F_CPU_64`, etc.

```
/* Enum to define timer clock types */
typedef enum
{
    NO_CLOCK,           /* No clock source (Timer is stopped) */
    F_CPU_CLOCK,        /* CPU clock as the timer clock source */
    F_CPU_8,            /* CPU clock divided by 8 as the timer clock */
    F_CPU_64,           /* CPU clock divided by 64 as the timer clock */
    F_CPU_256,          /* CPU clock divided by 256 as the timer clock */
    F_CPU_1024          /* CPU clock divided by 1024 as the timer clock */
} Timer_ClockType;
```

Various data types are defined to handle unsigned and signed integers of different sizes (8-bit, 16-bit, 32-bit, 64-bit), and floating-point types (`float32` and `float64`).

```
/* Standard Types */
typedef unsigned char    uint8;      /* 0 .. 255 */
typedef signed char      sint8;      /* -128 .. +127 */
typedef unsigned short   uint16;     /* 0 .. 65535 */
typedef signed short     sint16;     /* -32768 .. +32767 */
typedef unsigned long     uint32;     /* 0 .. 4294967295 */
typedef signed long      sint32;     /* -2147483648 .. +2147483647 */
typedef unsigned long long uint64;    /* 0 .. 1844E16 */
typedef signed long long  sint64;    /* -922E16 .. 922E16 */
typedef float            float32;     /* Single precision floating point */
typedef double           float64;     /* Double precision floating point */
```

A macro `NULL_PTR` is defined to represent a null pointer, which is set to 0.

```
#define NULL_PTR          ((void*)0)          /* Null pointer definition */
```

▪ Common_Macros:

This header file (`COMMON_MACROS_H_`) defines macros for bit manipulation in registers, commonly used in embedded systems. It includes macros to set, clear, toggle, and check bits, as well as bitwise rotations (left and right). These macros simplify low-level operations, making the code more efficient and readable.

```
#ifndef COMMON_MACROS_H_
#define COMMON_MACROS_H_

#define SET_BIT(REG,BIT) (REG |= (1<<BIT))      /* Set a certain bit */
#define CLEAR_BIT(REG,BIT) (REG &= ~(1<<BIT))    /* Clear a certain bit */
#define TOGGLE_BIT(REG,BIT) (REG ^= (1<<BIT))    /* Toggle a certain bit */

/* Rotate right the register value with specific number of rotates */
#define ROR(REG,NUM) (REG = (REG>>NUM) | (REG<<(8-NUM)))

/* Rotate left the register value with specific number of rotates */
#define ROL(REG,NUM) (REG = (REG<<NUM) | (REG>>(8-NUM)))

/* Check if a specific bit is set in any register and return true if yes */
#define BIT_IS_SET(REG,BIT) (REG & (1<<BIT))

/* Check if a specific bit is cleared in any register and return true if yes */
#define BIT_IS_CLEAR(REG,BIT) (!(REG & (1<<BIT)))
#define GET_BIT(REG,BIT) ((REG & (1<<BIT)) >> BIT) /* Get a specific bit value */

#endif /* COMMON_MACROS_H_ */
```

5.2.2 MCAL drivers

Each driver consists of a “**header file (.h)**” and a “**source file (.c)**”. In our implementation, we will create the **header file (File.h)**, which should be included in any driver that requires it. Additionally, every driver should include the “**std_types.h**” file to ensure consistent data type definitions and improve code portability.

To achieve our goal, we utilized several essential peripherals, including **GPIO (General Purpose Input/Output)**, **ICU (Input Capture Unit)**, **External Interrupts**, **PWM (Pulse Width Modulation)**, and **UART (Universal Asynchronous Receiver-Transmitter)**. These modules play a crucial role in handling input/output operations, capturing signals, managing external events, generating pulse signals, and enabling serial communication.

- General Purpose Input/Output (GPIO):

```
#include "../LIB/std_types.h" /* Include standard types header file */

/***** Definitions *****/
#define NUM_OF_PORTS          4 /* No. of ports (PORTA, PORTB, PORTC, PORTD) */
#define NUM_OF_PINS_PER_PORT  8 /* No. of pins per port (Pins 0 to 7) */

#define PORTA_ID              0 /* Port A ID */
#define PORTB_ID              1 /* Port B ID */
#define PORTC_ID              2 /* Port C ID */
#define PORTD_ID              3 /* Port D ID */

#define PIN0_ID               0 /* Pin 0 ID */
#define PIN1_ID               1 /* Pin 1 ID */
#define PIN2_ID               2 /* Pin 2 ID */
#define PIN3_ID               3 /* Pin 3 ID */
#define PIN4_ID               4 /* Pin 4 ID */
#define PIN5_ID               5 /* Pin 5 ID */
#define PIN6_ID               6 /* Pin 6 ID */
#define PIN7_ID               7 /* Pin 7 ID */

/***** Types Declaration *****/
typedef enum /* Enum to define pin direction types */
{
    PIN_INPUT, /* Pin is configured as input */
    PIN_OUTPUT /* Pin is configured as output */
} GPIO_PinDirectionType;

/* Enum to define port direction types */
typedef enum
{
    PORT_INPUT, /* pins in the port are configured as i/p */
    PORT_OUTPUT = 0xFF /* pins in the port are configured as o/p */
} GPIO_PortDirectionType;

/***** Functions Prototypes *****/
void GPIO_setupPinDirection(uint8 port_num, uint8 pin_num, GPIO_PinDirectionType
direction);
void GPIO_writePin(uint8 port_num, uint8 pin_num, uint8 value);
uint8 GPIO_readPin(uint8 port_num, uint8 pin_num);
void GPIO_setupPortDirection(uint8 port_num, GPIO_PortDirectionType direction);
void GPIO_writePort(uint8 port_num, uint8 value);
uint8 GPIO_readPort(uint8 port_num);
```

- **Universal Asynchronous Receiver-Transmitter (UART):**

```
#ifndef MCAL_UART_UART_H_
#define MCAL_UART_UART_H_

#include <avr/io.h>
#include <avr/interrupt.h>
#include "../LIB/std_types.h"

/* Configuration structure */
typedef struct
{
    uint32 baudRate;
    uint8 dataBits;           /* 5, 6, 7, 8, or 9 */
    uint8 parity;             /* 0 = None, 1 = Even, 2 = Odd */
    uint8 stopBits;           /* 1 or 2 */
} UART_ConfigType;

/* Function prototypes */
void UART_Init(UART_ConfigType *config);
void UART_Transmit(uint8 data);
uint8 UART_Receive(void);

/* Optional: Callbacks for RX and TX interrupts */
void UART_SetRxCallback(void (*callback)(uint8));
void UART_SetTxCallback(void (*callback)(void));

#endif /* MCAL_UART6_UART_H_ */
```

- **Timer:**

```
#ifndef MCAL_TIMER_H_
#define MCAL_TIMER_H_

#include "../LIB/std_types.h"
#include <avr/interrupt.h>
#include <avr/io.h>

/*****
 *                                     Types Declaration                                     *
 *****/
/* Enum to define Timer IDs */
typedef enum
{
    TIMER0_ID, TIMER1_ID, TIMER2_ID
} Timer_ID_Type;
```

```

/* Enum to define Timer clock sources */

/* Enum to define Timer modes */
typedef enum
{
    NORMAL_MODE, COMPARE_MODE
} Timer_ModeType;

/* Structure to configure the Timer */
typedef struct
{
    uint16 timer_InitialValue;          /* Initial value for the timer */
    uint16 timer_compare_MatchValue;    /* Compare match value, used in compare
mode only */
    Timer_ID_Type timer_ID;             /* Timer ID */
    Timer_ClockType timer_clock;        /* Clock source for the timer */
    Timer_ModeType timer_mode;          /* Mode of the timer */
} Timer_ConfigType;

/*****
 *                      Functions Prototypes                      *
 *****/
/*
 * Function to initialize the Timer driver.
 * Config_Ptr: Pointer to the Timer configuration structure.
 */
void Timer_init(const Timer_ConfigType * Config_Ptr);

/*
 * Function to disable the Timer via Timer_ID.
 * timer_type: The ID of the timer to be disabled.
 */
void Timer_deInit(Timer_ID_Type timer_type);

/*
 * Function to get the Timer Value.
 * timer_type: The ID of the timer to get the value from.
 * Returns: The current value of the timer.
 */
uint16 Timer_getTimerValue(Timer_ID_Type timer_type);

/*
 * Function to set the Call Back function address to the required Timer.
 * a_ptr: Pointer to the callback function.
 * a_timer_ID: The ID of the timer to set the callback for.
 */
void Timer_setCallBack(void(*a_ptr)(void), Timer_ID_Type a_timer_ID);

#endif /* MCAL_TIMER_H_ */

```

- **External Interrupt (EXT_INT):**

```

/***** Types Declaration *****/
typedef enum{ /* Enum to define the external interrupt types */
    INT_0, INT_1, INT_2
} EXT_INT_Type;

typedef enum{ /* Enum to define the sense control for external interrupts */
    FALLING_EDGE = 2, RISING_EDGE, FALLING_EDGE_INT2 = 0, RISING_EDGE_INT2
} EXT_INT_Sense;

typedef struct{ /* Structure to configure the external interrupt */
    EXT_INT_Type INT_ID; /* Interrupt ID (INT0, INT1, INT2) */
    EXT_INT_Sense INT_Sense; /* Interrupt sense control (Falling, Rising) */
} EXT_INT_ConfigType;

/***** Functions Prototypes *****/

void external_interrupt_init(const EXT_INT_ConfigType * a_ptr);
void external_interrupt_deinit(EXT_INT_Type INT_ID);
void external_interrupt_setCallBack(void(*a_ptr)(void), EXT_INT_Type INT_ID);

```

- **Pulse Width Modulation (PWM):**

```

/***** Types Declaration *****/
/* Enum to define PWM modes */
typedef enum
{
    NON_INVERTING = 2, /* Non-inverting (clear OC0/OC2 on compare match) */
    INVERTING /* Inverting (set OC0/OC2 on compare match) */
} PWM_Mode;

/* Structure to hold Timer configuration parameters */
typedef struct
{
    PWM_Mode mode; /* PWM mode (Non-inverting or Inverting) */
    Timer_ClockType timer_clock; /* Timer clock type (prescaler) */
    uint8 duty_cycle; /* Duty cycle percentage (0% to 100%) */
} Timer_ConfigType;

/***** Functions Prototypes *****/

void PWM_Timer0_Start(const Timer_ConfigType * Config_Ptr);
void PWM_Timer2_Start(const Timer_ConfigType * Config_Ptr);

```

In summary, the **MCAL (Microcontroller Abstraction Layer) drivers** serve as the foundation for low-level hardware interactions, enabling efficient communication between the microcontroller and various peripherals. By implementing key modules such as **GPIO, Timer, External Interrupts, PWM, and UART**, we establish a robust framework for handling input/output operations, signal processing, and data transmission.

Furthermore, these drivers ensure code modularity, reusability, and portability, making system development more scalable and maintainable. As we move forward, we will explore the **HAL (Hardware Abstraction Layer) and Application Drivers**, which build upon MCAL to further simplify hardware interaction and enhance overall system functionality.

Chapter 6: Control

6.1 Motor



Fig. 6.1. DC Motor

6.1.1 Overview about DC Motor

A DC motor (Direct Current motor) is an electromechanical device that converts electrical energy into mechanical motion. It operates on direct current (DC) and is widely used in various applications due to its efficiency, controllability, and reliability.

An H-Bridge circuit allows controlling the motor's direction (forward or backward).

▪ Working Principle

A DC motor works based on electromagnetic principles, specifically Lorentz force, which states that when a current-carrying conductor is placed in a magnetic field, it experiences a force. The interaction between the magnetic field of the stator and the current flowing through the rotor (armature) generates rotational motion.

▪ Main Components

- **Stator:** The stationary part that provides the magnetic field.
- **Rotor (Armature):** The rotating part that carries current through windings.
- **Power Supply:** Provides the necessary voltage and current to operate the motor.

▪ Advantages of DC Motors

- **Precise Speed Control:** Can be easily controlled by adjusting voltage or PWM signals.
- **High Starting Torque:** Ideal for applications requiring strong initial movement.

6.1.2 DC Motor Calculation

Small car model with a calculated speed of 5 km/h and a tire radius of 4.5 cm. I need to determine the angular velocity (ω) in RPM. The applied voltage is 12V, Front area of the car is 5 m².

Sol

1. Convert given values to standard units:

$$\text{tire radius } (r) = 4.5 \text{ cm} \rightarrow \text{So, } r = 0.045 \text{ m}$$

$$\text{Speed } (V) = 5 \text{ km/h} \rightarrow \text{So, } V = 1.388 \text{ m/s}$$

2. Calculate angular velocity (ω):

$$\omega = \frac{V}{r} = 30.8 \text{ rad/sec} = 30.8 * \frac{60}{2\pi} \approx 294.54 \text{ RPM} \rightarrow \text{So, use } \omega = 300 \text{ RPM}$$

3. Determine rolling resistance force (F_r)

$$F_r = C_r * m * g$$

Assuming $C_r = 0.02$, $m = 10 \text{ kg}$, and $g = 9.8 \text{ m/s}^2$:

$$F_r = 0.02 * 10 * 9.8 = 1.96 \text{ N}$$

4. Calculate air drag resistance (F_D):

$$F_D = \frac{1}{2} * \rho * A * C_D * V^2$$

The drag coefficient or air resistance of the car is its default value (C_D) = 0.35 for simple design

$$F_D = \frac{1}{2} * 1.225 * 5 * 0.35 * 1.388^2 = 2.065 \text{ N}$$

5. Total force (F_T):

$$F_T = F_r + F_D = 4.025 \text{ N}$$

6. Required power (P):

$$P = F_T * V = 4.025 * 1.388 = 5.5867 \text{ watt}$$

7. Adding Safety margin:

$$P_{max} = 2 * P = 11.1734 \text{ watt} \rightarrow \text{So, use } I = 1 \text{ A and } V = 12 \text{ volt}$$

6.1.3 Code of DC Motor

The vehicle model is equipped with four motors, with two positioned on the right side and two on the left side.

The movement of the vehicle is controlled by selectively operating these motors as follows:

To move forward, all four motors rotate in the same direction.

To move backward, all four motors rotate in the opposite direction.

To turn right, only the two left-side motors are activated, while the right-side motors remain off. This creates a pivoting effect that steers the vehicle to the right.

To turn left, only the two right-side motors are activated, while the left-side motors remain off, causing the vehicle to turn left.

For a sharper turn or rotation in place, the left-side motors can rotate forward while the right-side motors rotate in reverse, or vice versa, enabling the vehicle to spin in place.

This motor control strategy allows smooth and precise movement in all directions, ensuring efficient navigation based on the commands sent from the mobile application.

Code of DC Motor (file.c):

```
uint8 max_Speed = 0;          /* Variable to store maximum speed */

/*****
 *                               Functions Definitions                               *
 *****/
/* Static function to handle maximum speed */
static void Handel_Max_Speed(uint8 speed)
{
    max_Speed = speed;
}

void DcMotor_Init(uint8 MAXSPEED)
{
    Handel_Max_Speed(MAXSPEED);

    /* For motor 1 */
    GPIO_setupPinDirection(MOTOR_PORT_CONNECTION, PIN_INT1, PIN_OUTPUT);
    GPIO_setupPinDirection(MOTOR_PORT_CONNECTION, PIN_INT2, PIN_OUTPUT);
    GPIO_setupPinDirection(ENABLE1_PORT_CONNECTION, ENABLE1_PIN, PIN_OUTPUT);

    /* For motor 2 */
    GPIO_setupPinDirection(MOTOR_PORT_CONNECTION, PIN_INT3, PIN_OUTPUT);
    GPIO_setupPinDirection(MOTOR_PORT_CONNECTION, PIN_INT4, PIN_OUTPUT);
    GPIO_setupPinDirection(ENABLE2_PORT_CONNECTION, ENABLE2_PIN, PIN_OUTPUT);
}
```

```

    /* Stop the motor at the beginning. */
    GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT1, LOGIC_LOW);
    GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT2, LOGIC_LOW);
    GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT3, LOGIC_LOW);
    GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT4, LOGIC_LOW);
}

```

```

void DcMotor1_Rotate(DcMotor_State state, uint8 speed)
{
    Timer_ConfigType configurations = {NON_INVERTING, F_CPU_CLOCK, speed};
    PWM_Timer0_Start(&configurations);

    switch (state)
    {
        case STOP:
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT1, LOGIC_LOW);
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT2, LOGIC_LOW);
            break;

        case CW:
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT1, LOGIC_HIGH);
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT2, LOGIC_LOW);
            break;

        case CCW:
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT1, LOGIC_LOW);
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT2, LOGIC_HIGH);
            break;
    }
}

```

```

void DcMotor2_Rotate(DcMotor_State state, uint8 speed)
{
    Timer_ConfigType configurations = {NON_INVERTING, F_CPU_CLOCK, speed};
    PWM_Timer2_Start(&configurations);

    switch (state)
    {
        case STOP:
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT3, LOGIC_LOW);
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT4, LOGIC_LOW);
            break;

        case CW:
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT3, LOGIC_HIGH);
            GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT4, LOGIC_LOW);
            break;
    }
}

```

```

    case CCW:
        GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT3, LOGIC_LOW);
        GPIO_writePin(MOTOR_PORT_CONNECTION, PIN_INT4, LOGIC_HIGH);
        break;
    }
}

```

```

void Forward(){
    for(int i = 0; i <= max_Speed; i = i + 10){
        DcMotor1_Rotate(CCW, i);
        DcMotor2_Rotate(CCW, i);
        _delay_ms(DELAY_MOTOR);
    }
}

void Backward(){
    for(int i = 0; i <= max_Speed; i = i + 10){
        DcMotor1_Rotate(CW, i);
        DcMotor2_Rotate(CW, i);
        _delay_ms(DELAY_MOTOR);
    }
}

```

```

void Stop(void){
    DcMotor1_Rotate(STOP, MOTOR_STOP);
    DcMotor2_Rotate(STOP, MOTOR_STOP);
}

```

```

void Right_Forward(){
    for(int i = 0; i <= max_Speed; i = i + 10){
        DcMotor1_Rotate(STOP, MOTOR_STOP);
        DcMotor2_Rotate(CCW, i);
        _delay_ms(DELAY_MOTOR);
    }
}

void Left_Forward(){
    for(int i = 0; i <= max_Speed; i = i + 10){
        DcMotor1_Rotate(CCW, i);
        DcMotor2_Rotate(STOP, MOTOR_STOP);
        _delay_ms(DELAY_MOTOR);
    }
}

```

```

void Right_Backward(){
    for(int i = 0; i <= max_Speed; i = i + 10){
        DcMotor1_Rotate(STOP, MOTOR_STOP);
        DcMotor2_Rotate(CW, i);
        _delay_ms(DELAY_MOTOR);
    }
}

void Left_Backward(){
    for(int i = 0; i <= max_Speed; i = i + 10){
        DcMotor1_Rotate(CW, i);
        DcMotor2_Rotate(STOP, MOTOR_STOP);
        _delay_ms(DELAY_MOTOR);
    }
}

```

6.2 H-Bridge “L298”

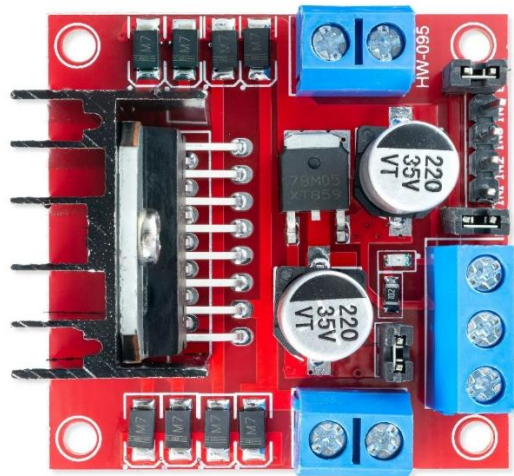


Fig. 6.2. L298

▪ Overview about L298

The H-Bridge L298 is a high-power dual full-bridge motor driver IC widely used for controlling the direction and speed of DC motors in various applications.

The L298 operates with a motor supply voltage of up to 46V and can deliver a continuous current of up to 2A per channel, allowing it to handle motors with moderate power requirements efficiently. The IC features separate logic and motor power supplies, providing flexibility in different operating conditions.

Due to its robust design and ease of use, the L298 is commonly integrated into microcontroller-based systems, where it is controlled via PWM (Pulse Width Modulation) signals to regulate motor speed and direction.

▪ Main Features of L298

- **Dual H-Bridge:** To control two DC motors.
- **High Voltage & Current Handling:** Up to 46V and 2A per channel.
- **Thermal Protection:** Built-in over-temperature protection.
- **Separate Logic & Motor Power Supply:** Allows flexible operation with different voltages.

▪ How the H-Bridge Works:

An H-Bridge is a circuit that allows a motor to rotate in both directions by controlling the polarity of the voltage applied to its terminals. L298 uses transistors arranged in an "H" shape to achieve this control.

- **Forward Rotation:** Activating one pair of transistors causes the motor to spin in one direction.
- **Reverse Rotation:** Activating the opposite pair of transistors reverses the direction.
- **Braking & Stopping:** Turning off all transistors stops the motor.

▪ Control Pins Explanation:

- **Enable Pins (ENA, ENB):** Enable or disable each motor.
- **Input Pins (IN1, IN2, IN3, IN4):** Control the direction of the motors.
- **Power Supply Pins (VCC, GND, VS):** Provide power to the logic and motor.

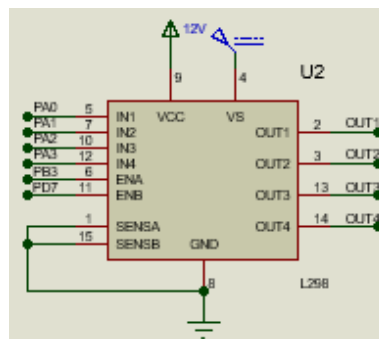


Fig. 6.3. L298 connections

6.3 Bluetooth module “HC-05”

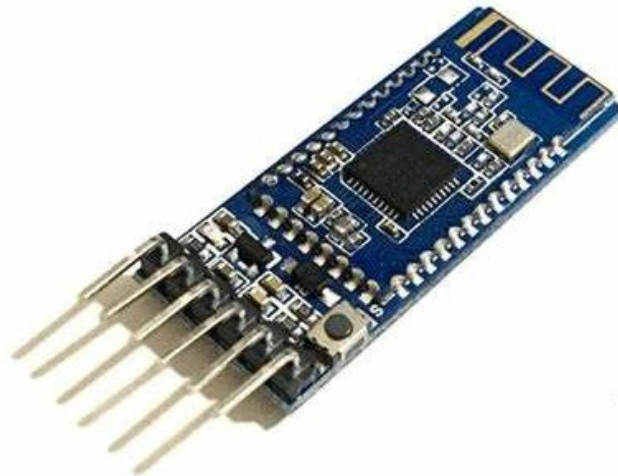


Fig. 6.4. HC-05

▪ Overview of HC-05

The HC-05 is a Bluetooth SPP (Serial Port Protocol) module designed for establishing wireless serial communication between microcontrollers and our application. It operates in the 2.4 GHz ISM (Industrial, Scientific, and Medical) frequency band, which is the standard for Bluetooth communication.

▪ Key Features

- **Bluetooth Version:** The HC-05 supports Bluetooth 2.0, which allows it to operate with most devices that have Bluetooth capability.
- **Serial Communication:** It communicates with microcontrollers (like ESP32, ATmega32, etc.) via UART protocol. This allows simple data exchange between the microcontroller and Bluetooth-connected devices.
- **Range:** It provides a range of up to 10 meters depending on the surrounding environment and obstacles.

▪ Connections and Pins

The HC-05 module has a small set of pins that make it easy to integrate into your circuit:

- **VCC:** This is the power pin. It typically needs a 3.3V to 5V supply, depending on your system’s voltage.
- **GND:** Ground pin, always connected to the system’s ground.
- **Tx:** This is the pin through which the HC-05 sends data to the microcontroller.
- **Rx:** This is the pin through which the HC-05 receives data from the microcontroller.

- **State:** Used to indicate the Bluetooth connection status. If it's low, the module is unconnected; if high, it's connected.
- **EN:** Used to enter AT command mode when held HIGH during startup, allowing you to configure the module.

6.4 Code of control

This code is responsible for controlling the motor operation, including direction and speed management using the H-Bridge L298. The application has dedicated buttons that send a character representing the desired movement, whether forward, backward, or any other action. This character is transmitted via the Bluetooth module, which then forwards it to the ATmega32 microcontroller using the UART protocol through Rx and Tx.

Before the application sends the character, the microcontroller operates in the loop. Once the character is received, an interrupt occurs, and the corresponding command is executed. I have also implemented three speed levels, allowing precise control over the model's movement using the mobile application. This provides full control over the vehicle's motion.

```
#include "Application.h"

/***** Global Variables *****/
volatile uint8 g_selection = 0;

int main(void)
{
    SREG |= (1 << 7);          /* Enable global interrupts */

    UART_Init(&config);
    UART_SetRxCallback(App_Receive);

    DcMotor_Init(100);

    while (1)
    {
        .
        .
    }
}
```

```
/***** Functions Definitions *****/
void App_Receive(uint8 recievedMSG)
{
```



```

g_selection = recievedMSG ;
switch (recievedMSG)
{
case 'F':
    Forward();          /* Move forward */
    break;
case 'B':
    Backward();          /* Move backward */
    break;
case 'S':
    Stop();              /* Stop movement */
    break;
case 'R':
    Right_Forward(); /* Turn right and move forward */
    break;
case 'L':
    Left_Forward();      /* Turn left and move forward */
    break;
case 'A':
    Right_Backward();    /* Turn right and move backward */
    break;
case 'H':
    Left_Backward(); /* Turn left and move backward */
    break;
case 'M':
    DcMotor_Init(motorSpeed(UART_Receive()));
    break;
}
}

```

```

uint8 motorSpeed(uint8 speed)
{
    switch(speed)
    {
    case '1':
        return MOTOR_SPEED_ONE;          /* Speed level 1 (70%) */
        break;
    case '2':
        return MOTOR_SPEED_TWO;           /* Speed level 2 (85%) */
        break;
    case '3':
        return MOTOR_MAX_SPEED;           /* Speed level 3 (100%) */
        break;
    }

    return MOTOR_MAX_SPEED; /* Default speed */
}

```

6.5 Simulation on proteus

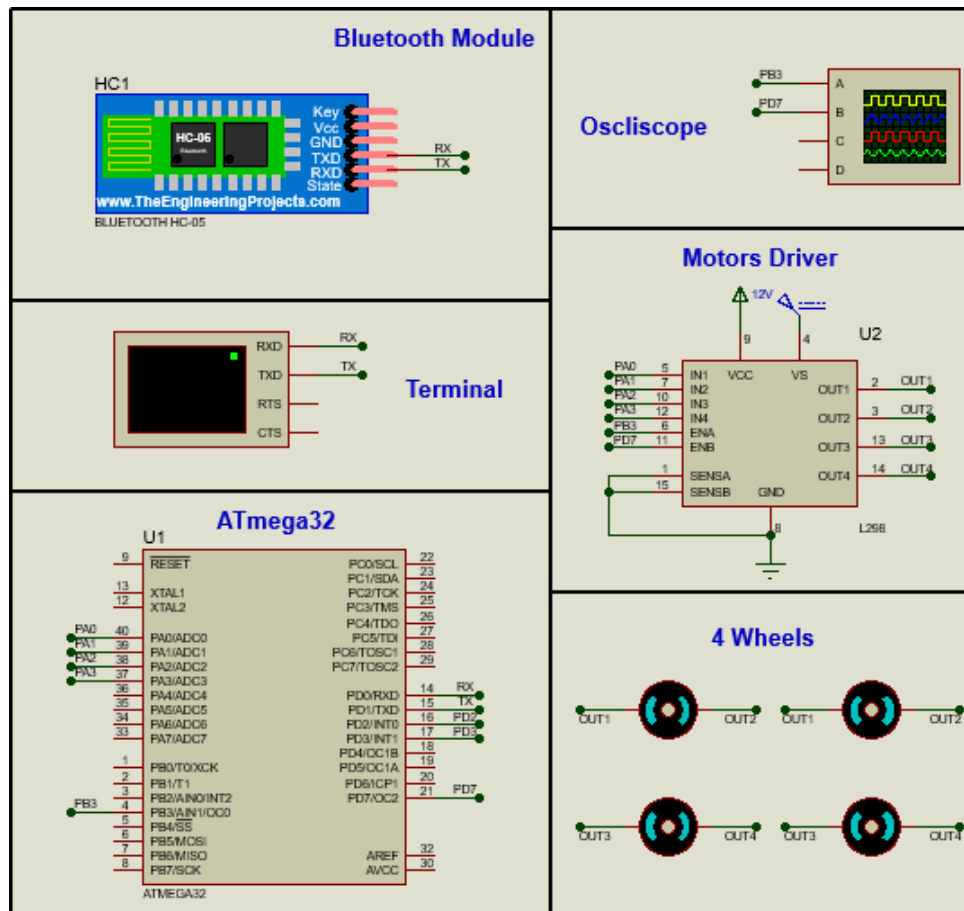


Fig. 6.5. Proteus control

Based on the previous circuit diagram, here is a detailed explanation of each connection and its function:

- **Bluetooth Module (HC-05)**

The Bluetooth module enables wireless communication between the mobile application and the ATmega32 microcontroller, allowing remote control commands to be sent and received.

- **ATmega32 Microcontroller**

The ATmega32 processes the received commands from the Bluetooth module and controls the motor driver accordingly. It serves as the central control unit for executing movement and operational logic.

- **Motor Driver (L298)**

The L298 motor driver controls the direction and speed of the four wheels based on input signals from the ATmega32. It allows independent control of each side, enabling forward, backward, and turning movements.

- **Wheels (Motors)**

The wheels receive power from the motor driver to facilitate movement. The differential control (activating only one side for turns) provides basic steering capability.

- **Terminal**

Provides a visual or textual interface for debugging or displaying system status.

Connects via UART, similar to the Bluetooth module, allowing serial communication for monitoring.

- **Oscilloscope**

Connected to observing signal waveforms, such as PWM signals

Useful for analyzing data transmission, motor control signals, or troubleshooting circuit behavior.

- **Overall System Functionality**

The system enables wireless control of a vehicle model through a mobile application. Commands are transmitted via Bluetooth to the ATmega32, which interprets them and controls the motors via the L298 driver. The vehicle can move forward, backward, and turn left or right based on motor activation patterns. Additional tools like the terminal and oscilloscope assist in monitoring and debugging.

- **Setting of Atmega32 and Bluetooth in proteus**

To change some settings on the ATmega32, we need to double-click on it. First, place the code file with the .hex extension into the program file. Then, configure the microcontroller's frequency settings by selecting "External Crystal High Frequency" in the CKSEL fuses section. Finally, set the clock frequency to 16 MHz.

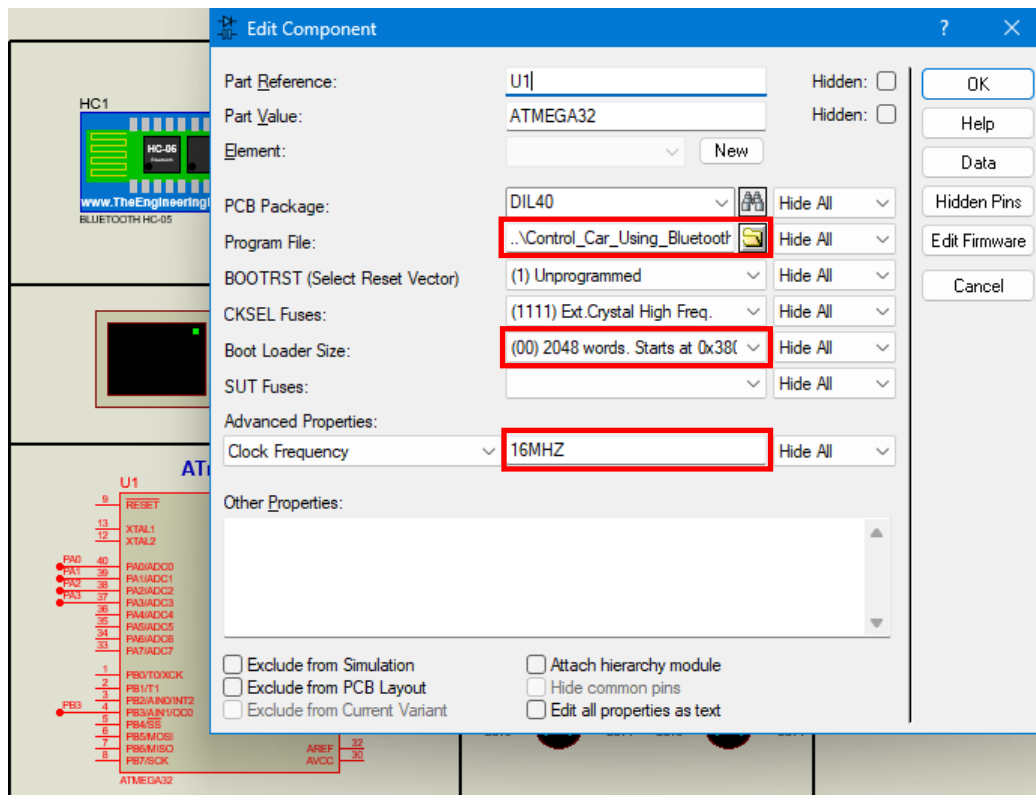


Fig. 6.6. Setting of Atmega32 in proteus

We must ensure that both the HC-05 module and our microcontroller (ATmega32) have the same baud rate setting (9600) in the code and simulation settings for UART communication

`UART_ConfigType uart_configratin = {9600, UART_Parity_NONE, UART_STOP_1_BIT, MODE_EIGHT_BIT};`

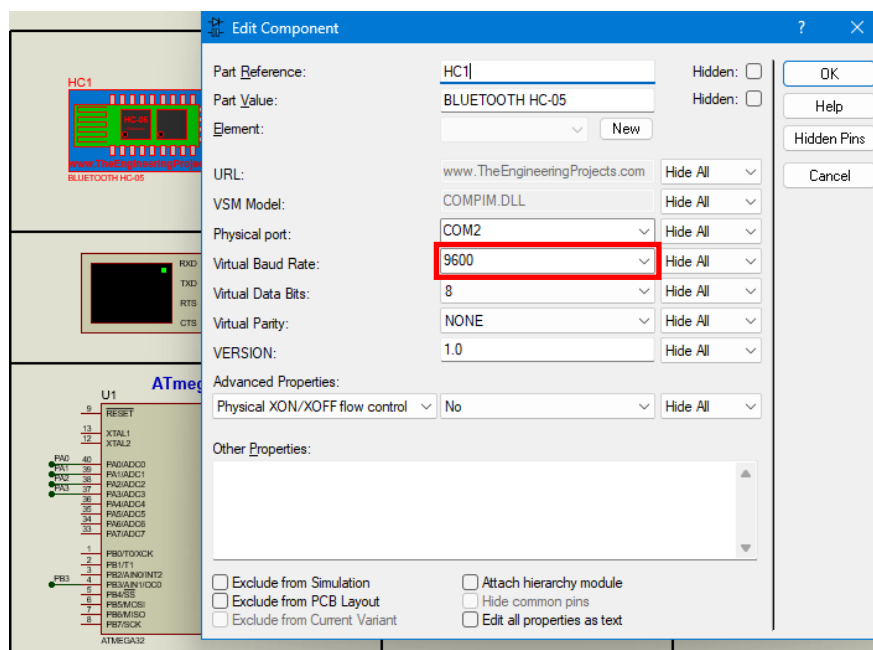


Fig. 6.7. Setting of HC-05 in proteus

Chapter 7: Auto Parking

7.1 Ultrasonic “HC-SR04”

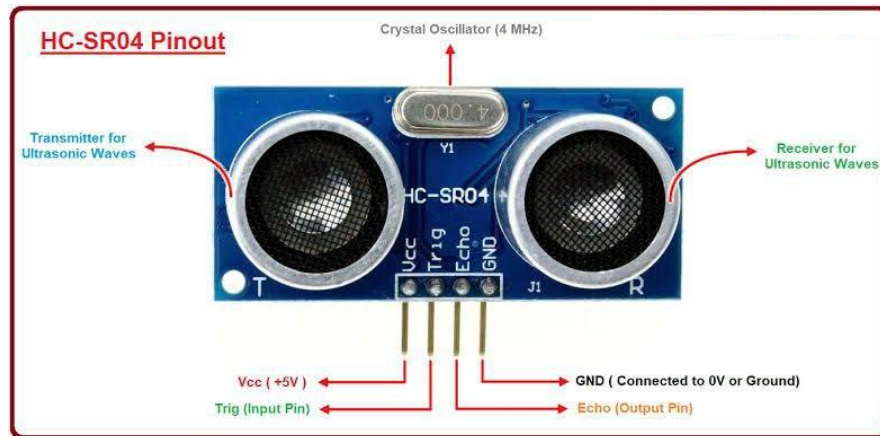


Fig. 7.1. HC-SR04

7.1.1 Overview about ultrasonic Sensor

The ultrasonic sensor works on the principle of SONAR and RADAR system which is used to determine the distance to an object. An ultrasonic sensor generates the high-frequency sound (ultrasound) waves. When this ultrasound hits the object, it reflects as echo which is sensed by the receiver as shown in below figure.

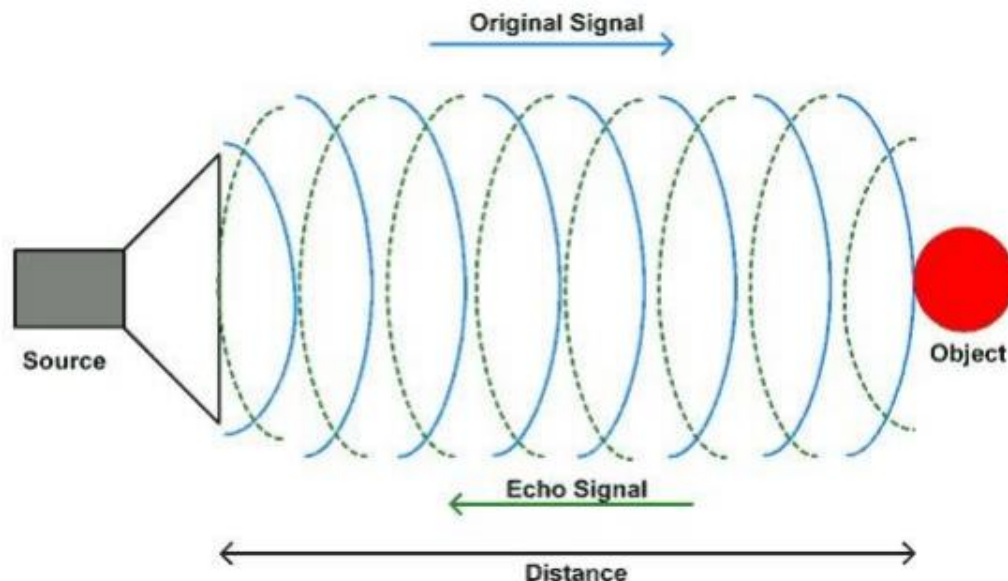


Fig. 7.2. Wave of ultrasonic

■ Ultrasonic Working Principle

By measuring the time required for the echo to reach to the receiver, we can calculate the distance. This is the basic working principle of Ultrasonic module to measure distance.

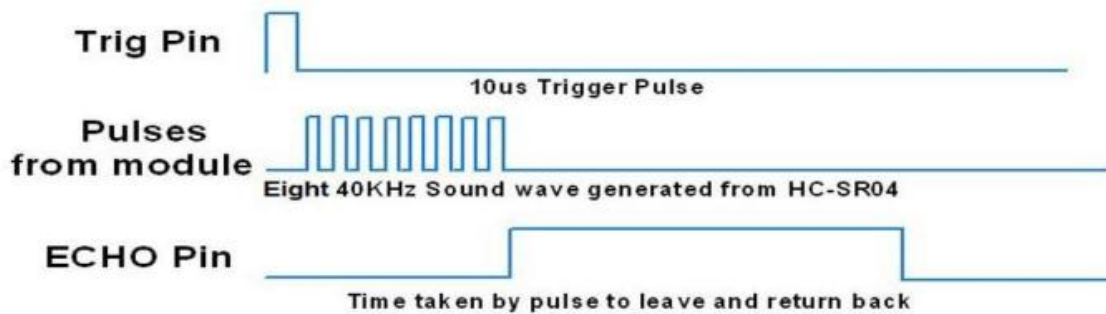


Fig. 7.3. HC-SR04 Ultrasonic Module Timing Diagram

1. We need to transmit a trigger pulse of at least 10 us to the HC-SR04 Trig Pin.
2. Then the HC-SR04 automatically sends Eight 40 kHz sound wave and waits for rising edge output at Echo pin.
3. When the rising edge capture occurs at Echo pin, start the Timer and wait for falling edge on Echo pin.
4. As soon as the falling edge is captured at the Echo pin, read the count of the Timer. This time count is the time required by the sensor to detect an object and return back from an object.

■ How does HC-SR04 Sensors Work ?

HC-SR-04 has an ultrasonic transmitter, receiver and control circuit. In ultrasonic module HCSR04, we have to give trigger pulse, so that it will generate ultrasound of frequency 40 kHz. After generating ultrasound i.e. 8 pulses of 40 kHz, it makes echo pin high. The echo pin remains high until it does not get the echo sound back. So the width of echo pin will be the time for sound to travel to the object and return back. Once we get the time we can calculate distance, as we know the speed of sound. HC-SR04 can measure up to range from 2 cm - 400 cm. The module has only 4 pins, Vcc, Gnd, Trig, and Echo. When a pulse of 10μsec or more is given to the Trigerrg pin, 8 pulses of 40 kHz are generated. After this, the Echo pin is made high by the control circuit in the module. The echo pin remains high till it gets an echo signal of the transmitted pulses back. The time for which the echo pin remains high, i.e. the width of the Echo pin gives the time taken for generated ultrasonic sound to travel towards the object and return. Using this time and the speed of sound in air, we can find the distance of the object using a simple formula for distance using speed and time.

- **How to calculate distance?**

$$\text{Distance} = \text{Speed} * \text{Time}$$

The speed of sound waves is 340 m/s. So,

$$\text{Total Distance} = \frac{340 * \text{Time of high Echo pulse}}{2}$$

Total distance is divided by 2 because signal travels from HC-SR04 to object and returns to the module HC-SR-04.

- **Steps of Programming**

1. ATmega32 microcontroller needs to transmit at least 10 us trigger pulse to the HC-SR04 Trig Pin.
2. After getting a trigger pulse, HC-SR04 automatically sends eight 40 kHz sound waves, and the microcontroller waits for rising edge output at the Echo pin.
3. When the rising edge capture occurs at the Echo pin which is connected to an input of ATmega16, start Timer of ATmega32 and again wait for a falling edge on the Echo pin.
4. As soon as the falling edge is captured at the Echo pin, the microcontroller reads the count of the Timer. This time count is used to calculate the distance to an object.

$$\text{The distance of Object (in cm)} = 17000 * \text{Time}$$

Now, here we have selected a 16MHz oscillator frequency for ATmega32, with Prescaler F_CPU/8 for timer frequency, then time to execute 1 instruction is 0.5 us.

So, the timer gets incremented after 0.5 us time elapses.

$$\text{The distance of Object (in cm)} = 17000 \times (\text{TIMER value}) \times 5 \times 10^{-7} = 0.0085 \times (\text{TIMER value})$$

Note: TIMER VALUE is the pulse width time calculated by the IC

7.1.2 Code of Ultrasonic Sensor

```
/*
 * Global Variables
 */
static uint8 g_edgeTime = 0, g_highTime = 0;

/*
 * Functions Prototypes
 */
static void Ultrasonic_edgeProcessing(void);

/*
 * Functions Definitions
 */
void Ultrasonic_init(void){
    ICU_ConfigType ICU_Configurations = {F_CPU_8, RAISING};
    ICU_init(&ICU_Configurations);
    ICU_setCallBack(Ultrasonic_edgeProcessing);
    GPIO_setupPinDirection(TRIGGER_PORT_CONNECTION, TRIGGER_PIN1, PIN_OUTPUT);
}

static void Ultrasonic_Trigger(void){
    GPIO_writePin(TRIGGER_PORT_CONNECTION, TRIGGER_PIN1, LOGIC_HIGH);
    _delay_us(10);
    GPIO_writePin(TRIGGER_PORT_CONNECTION, TRIGGER_PIN1, LOGIC_LOW);
}

uint16 Ultrasonic_readDistance(void){
    Ultrasonic_Trigger();
    return (g_highTime / 117.6) + 1;
}

static void Ultrasonic_edgeProcessing(void){
    g_edgeTime++;
    if(g_edgeTime == 1){
        ICU_clearTimerValue();
        ICU_setEdgeDetectionType(FALLING); /* Detect falling edge */
    }
    else if(2 == g_edgeTime){
        g_highTime = ICU_getInputCaptureValue(); /* Get the high time */
        ICU_setEdgeDetectionType(RAISING); /* Detect falling edge */
        ICU_clearTimerValue();
        g_edgeTime = 0; /* Clear the counter to start from zero again. */
    }
}
```


7.1.3 Auto Parking Motion calculation

At the beginning, when the letter "P" is sent from the application to the ATmega32 microcontroller, the system waits for one second and then begins taking readings from the ultrasonic sensor. It checks whether the detected distance is greater than 40 cm (the width of the car). If the distance is less than 40 cm, the LCD displays the measured distance and indicates that parking is not possible by showing "**No Space**", after which the system returns to the main routine.

However, if the distance is greater than 40 cm, the car moves forward for half a second while performing a check every 100 ms to determine if the available space still allows for parking. If the half-second interval passes without interruptions, the LCD displays the measured distance and indicates that parking is possible by showing "**Space Available**" and the parking maneuver is initiated. The car then performs the following sequence of actions:

1. Stop for one second.
2. Move forward to the left for half a second (**Left_Forward**).
3. Stop for half a second.
4. Reverse straight for one second (**Backward**).
5. Adjust its orientation for proper parking by reversing to the left for 1.4 seconds (**Left_Backward**).
6. Stop completely.

At this point, the car has completed the parking process and enters an idle state, waiting for a new command via Bluetooth.

Throughout this entire sequence, **the car operates at its minimum speed (70%)**, except for the first movement when determining whether the parking space is sufficient. This movement, which involves forwarding to the left (100%), is handled separately, as explained in the subsequent section where the time calculations for auto-parking are detailed:

The speed is 70% of 300 RPM = 210 RPM $\rightarrow \omega = 22 \text{ rad/sec}$

$$\omega = \frac{v}{r} \rightarrow v = 0.045 * 22 = 1 \text{ m/sec}$$

To motion (distance = 0.5 m) need to **delay** 0.5 sec

7.1.4 Liquid Crystal Display (LCD)

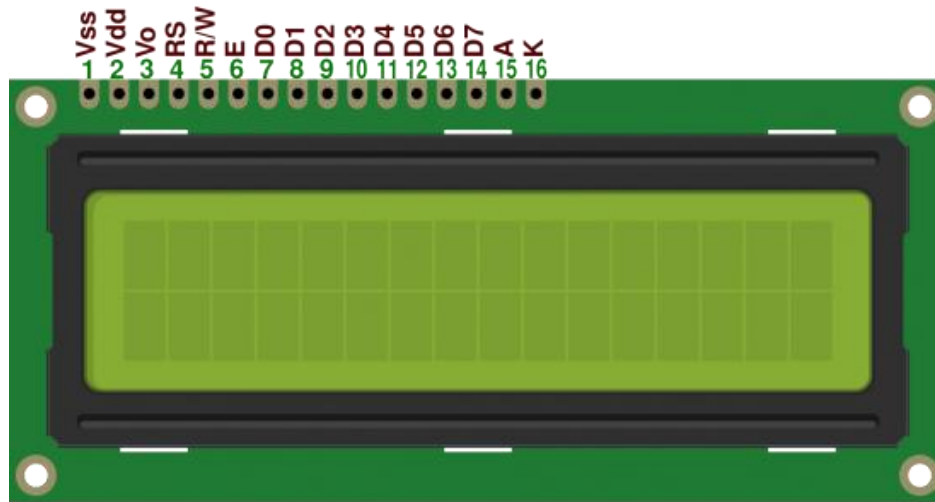


Fig. 7.4. LCD 16x2

The **16x2 LCD (Liquid Crystal Display)** is a widely used display module in electronics projects. It features a **16-character by 2-line layout**, meaning it can display up to 16 characters per line and has 2 lines in total. Each character is displayed in a **5x8 pixel matrix**, making it suitable for showing alphanumeric characters and some custom symbols.

▪ Key Features:

1. **Compact Size:** Ideal for small projects where space is limited.
2. **Interface:** Supports both **4-bit** and **8-bit modes**, allowing flexibility in communication with microcontrollers.
3. **Built-in Controller:** Uses the **HD44780 controller**, which simplifies interfacing with microcontrollers.
4. **Backlight and Contrast:** Comes with an optional backlight for readability in low-light conditions and a potentiometer to adjust the contrast.

▪ Pin Configuration:

The 16x2 LCD typically has 16 pins:

- **VSS (Pin 1):** Ground pin.
- **VDD (Pin 2):** Power supply pin (typically 5V).
- **V0 (Pin 3):** Contrast control pin.

- **RS (Pin 4):** Register Select – used to switch between command and data modes.
- **RW (Pin 5):** Read/Write – set to Write (0) for most projects.
- **E (Pin 6):** Enable – activates the LCD for data transfer.
- **D0-D7 (Pins 7-14):** Data pins for sending commands or data.
- **A and K (Pins 15-16):** Power for the backlight (Anode and Cathode).

▪ **How It Works:**

The LCD operates by sending commands (instructions) and data (characters) from the microcontroller.

- **Commands** are used to initialize the LCD, clear the display, or set the cursor position.
- **Data** represents the characters or symbols that you want to display on the screen.

7.1.5 Auto Parking code

Add the initial of Ultrasonic and add case 'P' in switch inside while (1)

```
int main(void)
{
    .
    .
    Ultrasonic_init();
    .
    .

    while(1)
    {
        .
        .
        switch(recievedMSG)
        {
            .
            .
            case 'P':
                autoParking();
                break;
            .
            .
        }
    }
    return 0;
}
```

The code of autoParking and readDistance functions to read from Ultrasonic sensor.

```
volatile uint16 distance = 0;
void autoParking(void){
    uint8 counter = 0;

    LCD_clearScreen();
    _delay_ms(500);
    _delay_ms(500);
    readDistance();
    Forward();

    while(distance > 40){
        readDistance();
        _delay_ms(100);
        counter++;

        if(counter > 5){
            LCD_displayStringRowColumn(1, 0, "SPACE Available");
            Stop();
            _delay_ms(500);
            _delay_ms(500);

            Left_Forward_Parking();
            _delay_ms(500);

            Stop();
            _delay_ms(500);

            Backward();
            _delay_ms(500);
            _delay_ms(500);

            Stop();
            _delay_ms(500);

            Left_Backward();
            _delay_ms(700);
            _delay_ms(700);

            break;
        }
    }
    if(counter < 5){
        LCD_displayStringRowColumn(1, 0, "NO SPACE");
    }
    Stop();
}
```

```

void readDistance(void){
    distance = Ultrasonic_readDistance();
    LCD_displayStringRowColumn(0, 0, "D1 = ");
    LCD_intgerToString(distance);
    LCD_displayStringRowColumn(0, 8, "cm");

    if(distance < 10){
        LCD_displayStringRowColumn(0, 6, " ");
    }

    if(distance < 100){
        LCD_displayStringRowColumn(0, 7, " ");
    }

    if(distance >= 100){
        LCD_displayStringRowColumn(0, 5, "99 ");
    }
}

```

The previous function is designed to measure the distance using an ultrasonic sensor and display the result on an LCD screen. Here's a detailed explanation:

- **Function Explanation:**

1. **Reading the Distance:**

- The function begins by calling `Ultrasonic_readDistance()`, which retrieves the measured distance from the ultrasonic sensor. The result is stored in the global variable `distance`.

2. **Displaying Distance on the LCD:**

- The function uses `LCD_displayStringRowColumn()` to position and display text on the LCD.
- The first part of the display shows "D1 =" at row 0, column 0, followed by the value of `distance` in centimeters. The integer value of `distance` is converted to a string using the function `LCD_intgerToString()` and displayed next to "D1 =".
- The unit "cm" is added after the distance value, starting at column 8.

3. **Formatting the LCD Output:**

- Since the LCD display is static and doesn't clear characters automatically, the function ensures that no extra characters are left behind when the displayed value changes.

- If the distance is less than 10 cm:
 - A blank space is added at column 6 to overwrite any previously displayed digits.
- If the distance is less than 100 cm:
 - A blank space is added at column 7 to ensure the display is properly formatted.
- If the distance is 100 cm or more:
 - The function overwrites the display at column 5 with "99 " to cap the displayed value at 99 cm, ensuring that the display does not overflow or distort.

7.1.6 Auto Parking Simulation

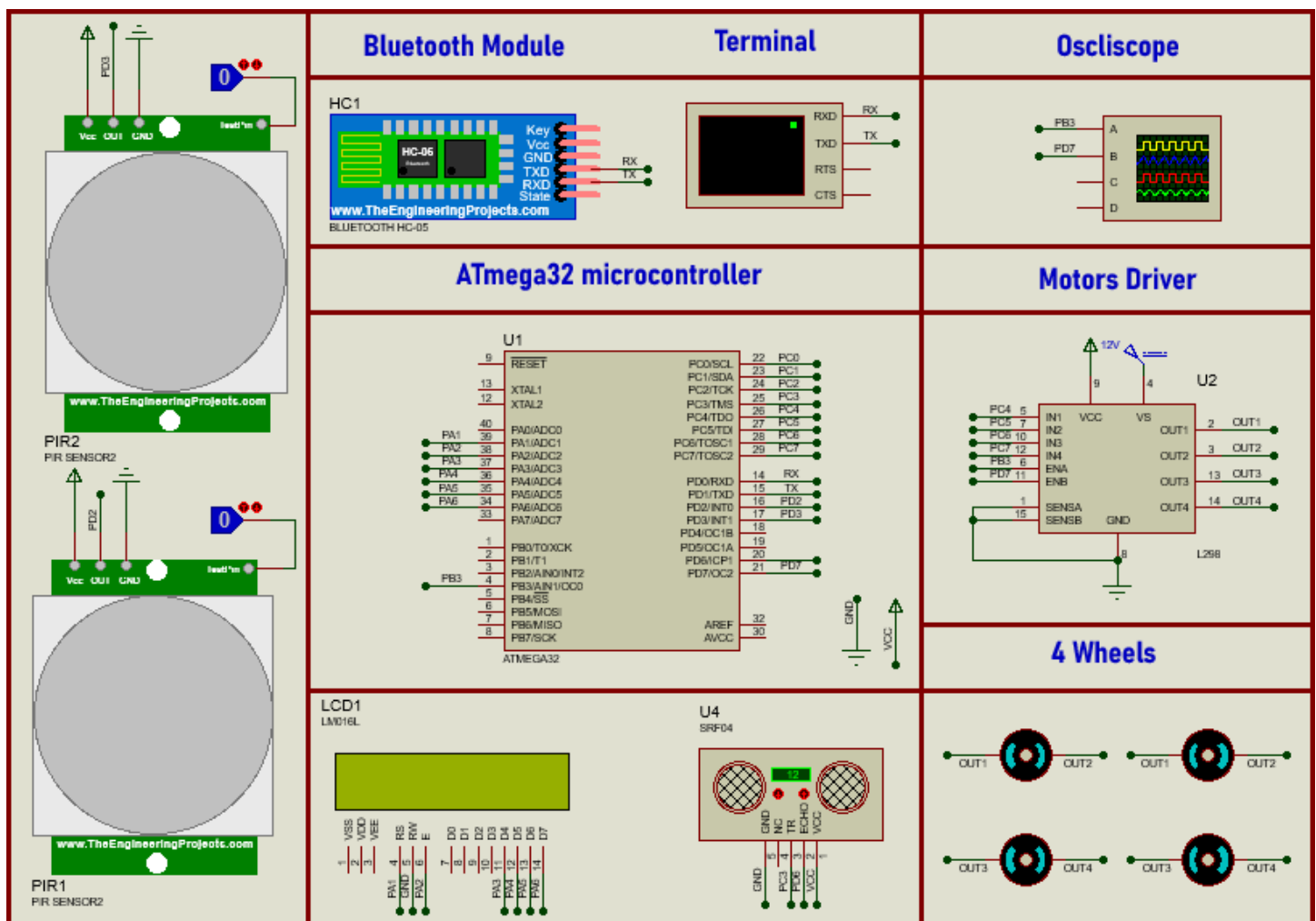


Fig. 7.5. Simulation of Car in proteus

Chapter 8: Optimizing Collision Avoidance (Methods and Improvements)

8.1 Collision avoidance using PIR



Fig. 8.1. Passive Infrared Sensor

8.1.1 Overview of Passive Infrared (PIR) Sensor

A Passive Infrared (PIR) Sensor is an electronic device that detects motion by sensing infrared (IR) radiation emitted by objects, particularly humans and animals.

▪ How does PIR Sensors Work ?

PIR sensors operate by detecting changes in infrared radiation in their field of view. Every warm object emits infrared radiation, and when a moving object (such as a person) passes in front of the sensor, it detects a change in IR levels. This triggers an output signal that can be used to activate alarms, lights, or other devices.

▪ Key Features of PIR Sensors

- **Passive Operation:** The sensor does not emit any radiation; it only detects infrared waves passively.
- **Wide Detection Range:** PIR sensors typically have a detection range of 3 to 12 meters, depending on the lens design.
- **Energy Efficiency:** They consume very little power, making them ideal for battery-operated devices.
- **Adjustable Sensitivity & Delay:** Many PIR sensors allow users to adjust sensitivity and delay time for more precise control.

▪ Limitations of PIR Sensors

No Object Identification: PIR sensors can detect motion but cannot distinguish between different objects.

8.1.2 Code of PIR

The code of PIR.c is:

```
#include "pir.h"

/*****
 *                               Functions Definitions                               *
 *****/

void PIR_init(void) {
    GPIO_setupPinDirection(PIR_PORT, PIR0_PIN, PIN_INPUT);
    GPIO_setupPinDirection(PIR_PORT, PIR1_PIN, PIN_INPUT);
}

uint8 PIR_ReadValue1() {
    return (GPIO_readPin(PIR_PORT, PIR0_PIN));
}

uint8 PIR_ReadValue2() {
    return (GPIO_readPin(PIR_PORT, PIR1_PIN));
}
```

8.1.3 Using Two PIR in our project

▪ Why did we use two PIR not one, explain?

- We used two PIR sensors, one at the front and one at the rear of the vehicle. When a person moves in front of or behind the vehicle, the corresponding PIR sensor detects the motion and takes action to stop the vehicle's movement.
- If the detected motion is in front of the vehicle, the PIR sensor cannot control forward movement, but it can prevent backward movement. Conversely, if the detected motion is behind the vehicle, the PIR sensor cannot control backward movement, but it can prevent forward movement.
- This ensures safety by restricting movement in the direction where an obstacle or person is detected

8.1.4 Interrupts and Their Importance

An interruption is a sign that temporarily halts the execution of the main program to handle an urgent task. Instead of continuously checking for a specific event using polling (which consumes processing time), interrupts allow the microcontroller to respond immediately when an event occurs.

▪ Why Use Interrupts ?

- **Efficiency:** The microcontroller can perform other tasks instead of constantly checking sensor status.
- **Real-time Response:** The system reacts instantly when an event occurs.
- **Power Saving:** The processor stays idle until an interruption is triggered, reducing unnecessary computations.

▪ Using Interrupts in the PIR Sensor Code

In our project, we used two PIR sensors (front and rear) and implemented interrupts to handle motion detection efficiently.

When the PIR sensor detects movement, it triggers an interruption, immediately executing the STOP command to halt the vehicle.

▪ Interrupt code handle:

```
void ISR_forward() { Stop(); }

void ISR_backward() { Stop(); }

int main(void)
{
    SREG |= (1<<7);           /* OR sei() for I-bit activate */

    PIR_init();

    external_interrupt_setCallback(ISR_forward, INT_0);
    external_interrupt_init(&EX_interrupt_0);

    external_interrupt_setCallback(ISR_backward, INT_1);
    external_interrupt_init(&EX_interrupt_1);

    .
    .
    .
}
```

▪ There are three inputs to the AND gate:

- **Global Interrupt Enable (GIE) - 1 bit:** This is the global interruption enable bit. If it's set to 1, interruptions are allowed. If it's 0, all interrupts are disabled.
- **Timer Interrupt Enable (MIE):** This bit enables or disables the timer interrupts specifically. If it's 1, the timer interrupt is enabled.
- **Timer Interrupt Flag (MIF):** This flag is set when the timer interrupt occurs. It indicates that the timer event has happened (in our project occurs when rising edge happen).

▪ How It Works:

The AND gate produces an output only when all three inputs are 1 (GIE = 1, MIE = 1, MIF = 1).

When this happens, the microcontroller recognizes that a timer interrupt has occurred and executes the corresponding interruption service routine (ISR).

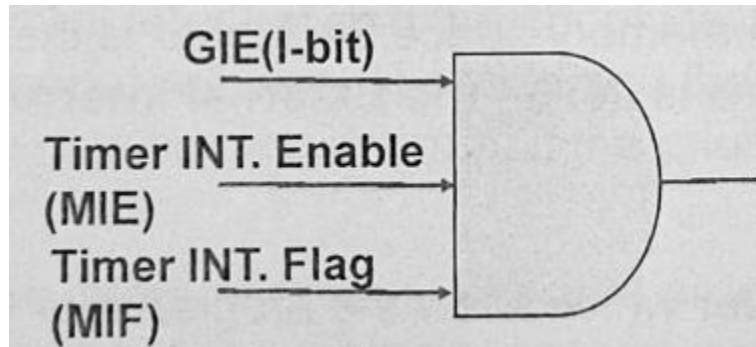


Fig. 8.2. AND Gate of interrupt service

8.1.5 Check motion of objects

After stopping, a condition checks whether the PIR sensor still detects an object:

If the front PIR sensor still detects an obstacle, the vehicle cannot move forward but can move backward.

If the rear PIR sensor still detects an obstacle, the vehicle cannot move backward but can move forward.

This ensures that the vehicle only moves when it is safe to do so. The use of interrupts allows for immediate reaction to motion detection, making the system more reliable and responsive

```
int main(void)
{
.
.
switch(recievedMSG)
{
case 'F':
    if(!pir_forward_recieve){ Forward(); }
    break;
case 'B':
    if(!pir_backward_recieve){ Backward(); }
    break;
case 'S':
    Stop(); /* Stop movement */
    break;
case 'R':
    if(!pir_forward_recieve){ Right_Forward(); }
    break;
}
```

```

case 'L':
    if(!pir_forward_recieve){ Left_Forward(); }
    break;
case 'A':
    if(!pir_backward_recieve){ Right_Backward(); }
    break;
case 'H':
    if(!pir_backward_recieve){ Left_Backward(); }
    break;
case 'M':
    DcMotor_Init(motorSpeed(UART_receiveByte()));
    break;
}

```

Simulation of PIR in proteus

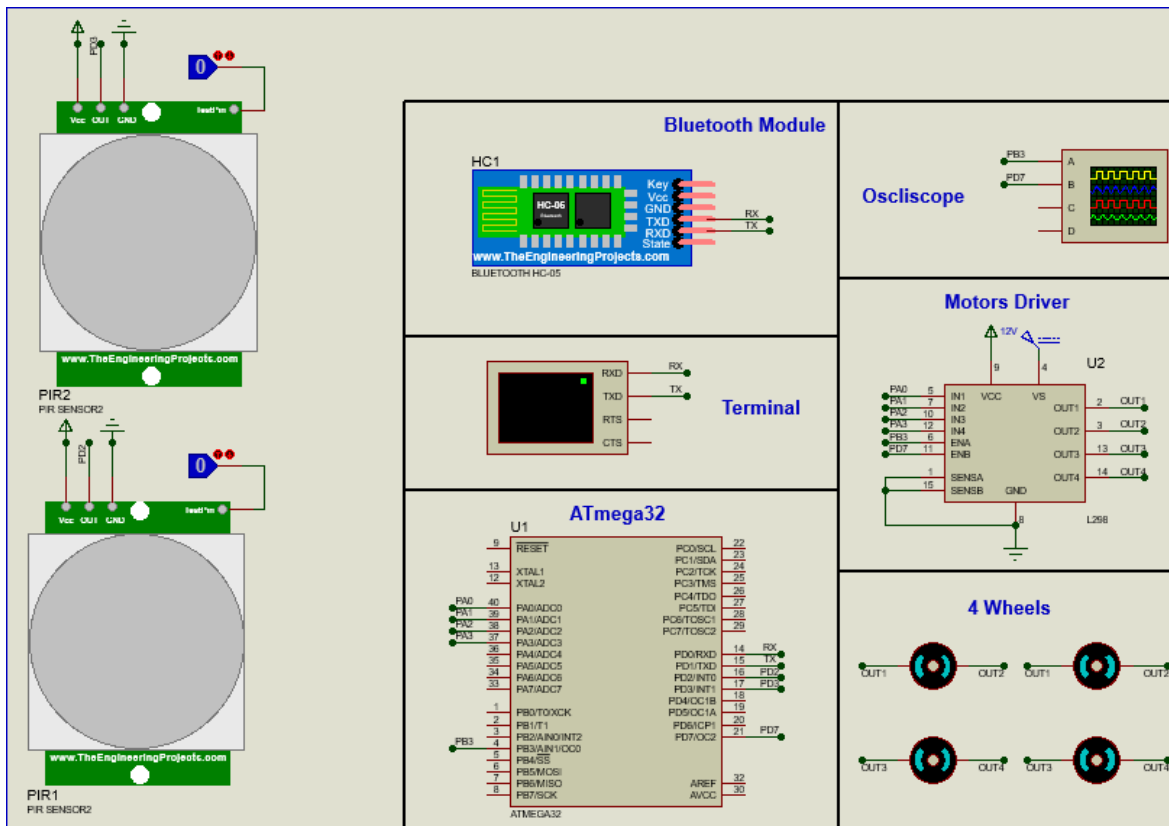


Fig. 8.3. Simulation of PIR in proteus

8.2 Collision avoidance using ultrasonic (The Improvement)

Due to the limitations of the PIR sensor, which can only detect moving objects and is unable to sense stationary obstacles or measure distances, it proved ineffective for collision avoidance. To address this shortcoming, we enhanced the system by replacing the PIR sensor with an Ultrasonic sensor. Unlike the PIR, the Ultrasonic sensor can detect both moving and stationary objects while also providing accurate distance measurements between the vehicle and obstacles. As previously discussed in Chapter (7), where we outlined its advantages and functionality, we will utilize this sensor to alert the driver through a buzzer when the vehicle reaches a predefined distance from an obstacle. If the distance decreases beyond a critical threshold, the system will automatically halt the vehicle to prevent collisions.

When using more than two Ultrasonic sensors with the ATmega32, the system failed to operate accurately, producing identical readings, which was not the intended outcome. Since our design requires three Ultrasonic sensors, one on the right side for Auto Parking, as previously discussed in Chapter (7), and two replacing the PIR sensors (one at the front and another at the rear) we implemented Timer 1 to measure the wave propagation time. The timer functions as a switch, alternating between the three Ultrasonic sensors, with two linked to Interrupt-0 and the third connected to Interrupt-1. The measured distances are then displayed on the LCD and consolidated into a single function for reading values, which is subsequently called within the main function inside the while loop for continuous monitoring and processing.

- **Ultrasonic Code**

```

/*****
 * Module      : Ultrasonic Sensor
 * File Name   : ultrasonic_sensor.c
 * Author      : A71a Team :)
 * Created on  : 28/01/2025 --> "Edited in 24/02/2025"
 * Description : Source file for the Ultrasonic Sensor driver
 *****/
#include "ultrasonic_sensor.h" /* Include Ultrasonic Sensor header file */

/*****
 *                               Global Variables                               *
 *****/
volatile static uint16 g_highTime_INT0 = 0; /* Stores the high time (pulse width) */
volatile static uint8 g_edgeTime_INT0 = 0; /* Tracks the number of edges detected */

volatile static uint16 g_highTime_INT1 = 0; /* Stores the high time (pulse width) */
volatile static uint8 g_edgeTime_INT1 = 0; /* Tracks the number of edges detected */

```

```

/*****
 *                               Functions Prototypes                               *
 *****/
static void Ultrasonic_edgeProcessing_INT0(void);
static void Ultrasonic_edgeProcessing_INT1(void);

/*****
 *                               Functions Definitions                               *
 *****/
void Ultrasonic_init(void)
{
    EXT_INT_ConfigType EXT_INT0_Configurations = {INT_0, RISING_EDGE};
    external_interrupt_setCallBack(Ultrasonic_edgeProcessing_INT0, INT_0);
    external_interrupt_init(&EXT_INT0_Configurations);

    EXT_INT_ConfigType EXT_INT1_Configurations = {INT_1, RISING_EDGE};
    external_interrupt_setCallBack(Ultrasonic_edgeProcessing_INT1, INT_1);
    external_interrupt_init(&EXT_INT2_Configurations);

    /* Set up pin direction for trigger pin as output */
    GPIO_setupPinDirection(TRIGGERS_PORT_CONNECTION, TRIGGER1_PIN, PIN_OUTPUT);
    GPIO_setupPinDirection(TRIGGERS_PORT_CONNECTION, TRIGGER2_PIN, PIN_OUTPUT);
    GPIO_setupPinDirection(TRIGGERS_PORT_CONNECTION, TRIGGER3_PIN, PIN_OUTPUT);
}

static void Ultrasonic_Trigger(Ultrasonic ultrasonic)
{
    switch(ultrasonic)
    {
    case U_right:
        /* Set the trigger pin to high for 10us */
        GPIO_writePin(TRIGGERS_PORT_CONNECTION, TRIGGER1_PIN, LOGIC_HIGH);
        _delay_us(10);
        /* Set the trigger pin back to low */
        GPIO_writePin(TRIGGERS_PORT_CONNECTION, TRIGGER1_PIN, LOGIC_LOW);
        break;

    case U_forward:
        /* Set the trigger pin to high for 10us */
        GPIO_writePin(TRIGGERS_PORT_CONNECTION, TRIGGER2_PIN, LOGIC_HIGH);
        _delay_us(10);
        /* Set the trigger pin back to low */
        GPIO_writePin(TRIGGERS_PORT_CONNECTION, TRIGGER2_PIN, LOGIC_LOW);
        break;

    case U_backward:
        /* Set the trigger pin to high for 10us */
        GPIO_writePin(TRIGGERS_PORT_CONNECTION, TRIGGER3_PIN, LOGIC_HIGH);
        _delay_us(10);
    }
}

```

```

        /* Set the trigger pin back to low */
        GPIO_writePin(TRIGGERS_PORT_CONNECTION, TRIGGER3_PIN, LOGIC_LOW);
        break;
    }
}

uint16 Ultrasonic_readDistance (Ultrasonic ultrasonic)
{
    switch(ultrasonic)
    {
        case U_right:
            /* Send the trigger pulse to start the measurement */
            Ultrasonic_Trigger(U_right);

            /* Calculate the distance in centimeters using the high time */
            return (g_highTime_INT0 * 1.088) + 1;
            break;

        case U_forward:
            /* Send the trigger pulse to start the measurement */
            Ultrasonic_Trigger(U_forward);

            /* Calculate the distance in centimeters using the high time */
            return (g_highTime_INT1 * 1.088) + 1;
            break;

        case U_backward:
            /* Send the trigger pulse to start the measurement */
            Ultrasonic_Trigger(U_backward);

            /* Calculate the distance in centimeters using the high time */
            return (g_highTime_INT0 * 1.088) + 1;
            break;
    }
}

static void Ultrasonic_edgeProcessing_INT0(void)
{
    g_edgeTime_INT0++;    // Increment edge count

    if (1 == g_edgeTime_INT0) {
        /* Rising edge detected */
        Timer_deInit(TIMER1_ID); // De-initialize Timer1

        external_interrupt_deinit(INT_0); // De-initialize INT2
        EXT_INT_ConfigType EXT_INT0_Configurations = {INT_0, FALLING_EDGE};
        external_interrupt_init(&EXT_INT0_Configurations);
    }
}

```

```

    Timer_ConfigType Timer_Configurations =
    {0, 0, TIMER1_ID, F_CPU_1024, NORMAL_MODE}; // Configure Timer1
    Timer_init(&Timer_Configurations); // Initialize Timer1

} else if (2 == g_edgeTime_INT0) {
    /* Falling edge detected */
    g_highTime_INT0 = Timer_getTimerValue(TIMER1_ID);
    Timer_deInit(TIMER1_ID); /* De-initialize Timer1 */

    external_interrupt_deinit(INT_0); /* De-initialize INT2 */
    EXT_INT_ConfigType EXT_INT0_Configurations = {INT_0, RISING_EDGE};
    external_interrupt_init(&EXT_INT0_Configurations);

    g_edgeTime_INT0 = 0; /* Reset edge count */
}
}

static void Ultrasonic_edgeProcessing_INT1(void)
{
    g_edgeTime_INT1++; /* Increment edge count

    if (1 == g_edgeTime_INT1) {
        /* Rising edge detected */
        Timer_deInit(TIMER1_ID); // De-initialize Timer1

        external_interrupt_deinit(INT_1); // De-initialize INT1
        EXT_INT_ConfigType EXT_INT1_Configurations = {INT_1, FALLING_EDGE};
        external_interrupt_init(&EXT_INT1_Configurations);

        Timer_ConfigType Timer_Configurations =
        {0, 0, TIMER1_ID, F_CPU_1024, NORMAL_MODE}; // Configure Timer1
        Timer_init(&Timer_Configurations); // Initialize Timer1

    } else if (2 == g_edgeTime_INT1) {
        /* Falling edge detected */
        g_highTime_INT1 = Timer_getTimerValue(TIMER1_ID);
        Timer_deInit(TIMER1_ID); /* De-initialize Timer1 */

        external_interrupt_deinit(INT_1); /* De-initialize INT2 */
        EXT_INT_ConfigType EXT_INT2_Configurations = {INT_1, RISING_EDGE};
        external_interrupt_init(&EXT_INT2_Configurations);

        g_edgeTime_INT1 = 0; /* Reset edge count */
    }
}

```

- Application Code

```
#include "Application.h"

/***** Global Variables *****/
volatile uint16 g_distanceRight    = 0;
volatile uint16 g_distanceForward = 0;
volatile uint16 g_distanceBackward = 0;

/***** Interrupt Service Routines *****/
void ISR0(void) { Stop(); }

void ISR1(void) { Stop(); }

int main(void)
{
    SREG |= (1 << 7);          /* Enable global interrupts */

    .
    .

    LCD_init();
    Buzzer_init();
    Ultrasonic_init();

    while (1)
    {
        readDistance();        /* Read distances from Three Ultrasonics */
        .
    }
}
```

```
/***** Functions Definitions *****/
void readDistance(void)
{
    g_distanceRight = Ultrasonic_readDistance(U_right);

    LCD_displayStringRowColumn(0, 0, "R:");
    LCD_moveCursor(0, 2);
    LCD_intgerToString(g_distanceRight);

    if (g_distanceRight < 100)
    {
        LCD_displayStringRowColumn(0, 5, " ");
    }
    if (g_distanceRight > 100)
    {
        LCD_displayStringRowColumn(0, 2, "99 ");
    }
}
```



```

if (g_distanceRight < 10)
{
    LCD_displayStringRowColumn(0, 3, " ");
}

_delay_ms(100);

g_distanceForward = Ultrasonic_readDistance(U_forward);

LCD_displayStringRowColumn(1, 0, "F:");
LCD_moveCursor(1, 2);
LCD_intgerToString(g_distanceForward);

if (g_distanceForward < 100)
{
    LCD_displayStringRowColumn(1, 5, " ");
}
if (g_distanceForward > 100)
{
    LCD_displayStringRowColumn(1, 2, "99 ");
}
if (g_distanceForward < 10)
{
    LCD_displayStringRowColumn(1, 3, " ");
}

_delay_ms(100);

g_distanceBackward = Ultrasonic_readDistance(U_backward);

LCD_displayStringRowColumn(1, 6, "B:");
LCD_moveCursor(1, 9);
LCD_intgerToString(g_distanceBackward);

if (g_distanceBackward < 100)
{
    LCD_displayStringRowColumn(1, 12, " ");
}
if (g_distanceBackward > 100)
{
    LCD_displayStringRowColumn(1, 9, "99 ");
}
if (g_distanceBackward < 10)
{
    LCD_displayStringRowColumn(1, 10, " ");
}

_delay_ms(100);
}

```

```

void collisionAvoidance(void){
    if(g_distanceForward <= 20 && (g_selection == 'F' || g_selection == 'R' ||
g_selection == 'L')){
        if(g_distanceForward <= 10){
            Stop();
            Buzzer_off();
        }
        else{
            Buzzer_on();
        }
    }

    if(g_distanceBackward <= 20 && (g_selection == 'B' || g_selection == 'A' ||
g_selection == 'H')){
        if(g_distanceBackward <= 10){
            Stop();
            Buzzer_off();
        }
        else{
            Buzzer_on();
        }
    }
}

```

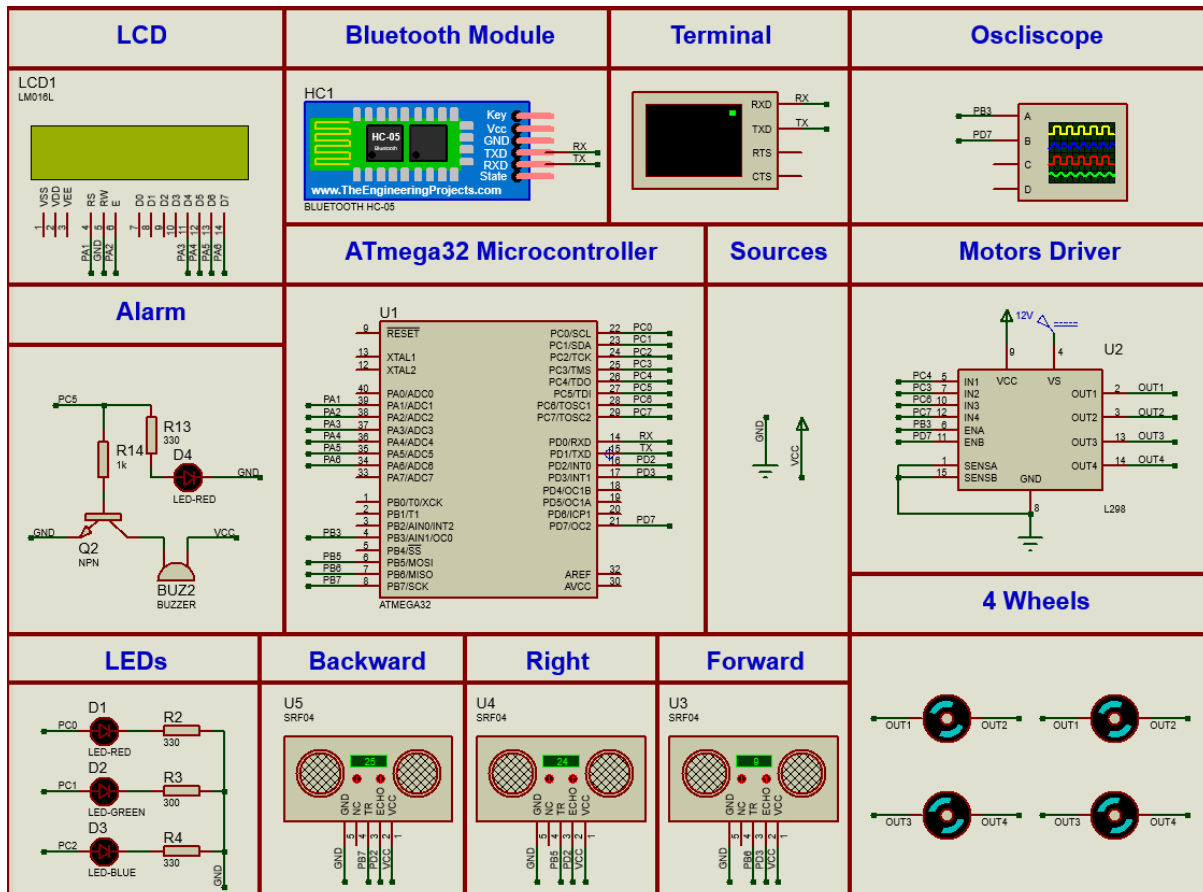


Fig. 8.4. Total Project Simulation in proteus

References

1. Atmega32 Microcontroller:

- Microchip AVR Atmega32 Overview and Documentation:
 - [Microchip Technology Atmega32 Product Page](#)

2. Proteus Simulation for Microcontrollers:

- [Proteus Official Software for Simulation](#)

3. ESP32-WROOM-32:

- ESP32 IoT Applications and Programming:
 - [Espressif ESP32 Datasheet](#)
 - [Programming ESP32 with Arduino IDE](#)

4. IoT and MQTT Integration:

- [Mosquitto MQTT Documentation](#)

5. Node-RED:

- IoT Integration with Node-RED:
 - [Official Node-RED Documentation](#)

6. MIT App Inventor:

- Building Mobile Applications for Bluetooth Control:
 - [MIT App Inventor Documentation](#)

7. L298 Motor Driver:

- L298 H-Bridge Overview and Datasheet:
 - [L298 Datasheet](#)

8. Ultrasonic Sensors (HC-SR04):

- HC-SR04 Sensor Datasheet and Working Principles:
 - [HC-SR04 Datasheet](#)